

AI Privacy

Privacy Threats and Attack Fundamentals

Machine learning models are supposed to learn patterns, not memorize individuals. Yet every model trained on personal data carries a hidden risk: it can reveal which specific individuals were in its training set. This privacy violation, known as `membership inference`, represents one of the core threats to ML systems deployed on sensitive data.

What is a Membership Inference Attack?

A `Membership Inference Attack (MIA)` targets a straightforward question: was a specific data point used to train a given model? We provide a sample and receive a binary answer (member or non-member) based on how the model responds to that sample. Despite its simplicity, this binary classification has serious privacy implications because membership itself can reveal sensitive information.

Privacy Attacks on Machine Learning

`MIA` belongs to a broader family of privacy attacks targeting ML systems, each extracting different types of information.

`Model inversion attacks` attempt to reconstruct training data features from model outputs. Given a model that predicts disease risk from genetic markers, an attacker might infer genetic information about training subjects. A related threat comes from `attribute inference attacks`, where we deduce sensitive attributes not directly predicted by the model. If a model predicts income and was trained on complete records, querying it strategically might reveal education levels or employment status of training members. The most direct form of leakage occurs in `training data extraction attacks`, where we recover verbatim training examples from models. Large language models that memorize specific sequences are particularly vulnerable to this approach.

Where does `MIA` fit in this landscape? It represents a basic privacy violation because it requires the least information to execute and often appears alongside other attacks. Even if attackers do not run a separate membership test first, a model that leaks membership information tends to be easier to exploit with `model inversion attacks` or `attribute inference attacks`. `MIA` success also serves as a privacy audit metric: if a model leaks membership, it likely leaks other information too. We focus on `MIA` because it establishes a baseline vulnerability that other attacks often build upon.

Consider a medical diagnosis model trained on patient records. An attacker with access only to the model's predictions could determine whether a particular patient's data was used for training. If the model was trained exclusively on cancer patients, successfully identifying someone as a training member reveals their medical status. Similar scenarios arise in

financial services, where membership in a credit scoring model's training set might expose loan history, or in social platforms where presence in a content moderation dataset could imply previous violations.

This module establishes the baseline vulnerability of ML models to membership inference before we introduce any defenses. We implement the core attack methodology from [Shokri et al.'s 2017 paper](#), which demonstrated that standard neural networks leak substantial membership information through their prediction behavior.

Why Membership Inference Works

Membership inference exploits the distinction between memorization and generalization. When we train a model, we want it to learn general patterns that apply to new data. In practice, models also memorize specific training examples, particularly those that are unusual, repeated, or near decision boundaries. This memorization creates detectable behavioral differences between how models treat data they have seen versus data they have not.

Consider what happens during gradient descent. We adjust model weights to reduce loss on training samples. After many iterations, we fit training data very closely, sometimes perfectly classifying every training example. But this tight fit does not transfer to new data. We have learned idiosyncrasies of specific training samples instead of underlying patterns. This phenomenon, commonly called overfitting, is the root cause of membership leakage.

Our attack model extracts two distinct signals from model predictions. The first is prediction confidence: how certain is the model about its prediction? Members tend to receive higher-confidence predictions because the model optimized specifically for them during training. A member might receive $[0.05, 0.95]$ (95% confidence in class 1) while a non-member gets $[0.20, 0.80]$ (80% confidence). The second signal is prediction correctness: does the predicted class match the true label? Models are more accurate on their training data because they have seen those exact feature combinations during training.

These signals interact in interesting ways. A high-confidence correct prediction is a strong membership indicator because both signals agree. A high-confidence incorrect prediction suggests non-membership. The challenging cases are low-confidence predictions where neither signal is decisive, and these form the bulk of attack errors. Sometimes the signals conflict: a member receives a low-confidence prediction on a genuinely ambiguous sample, or a non-member receives high confidence on a very typical sample. Our attack model learns to weight these signals based on how often each pattern corresponds to membership.

Model capacity also matters. Larger models with more parameters can memorize more training data, making them more vulnerable to MIA. A model with 10 million parameters stores more training examples in its weights than one with 100,000 parameters. Regularization techniques like dropout, weight decay, and early stopping reduce

memorization but cannot eliminate it entirely. The optimization process inherently treats training data differently from unseen data, creating tension between model utility and privacy.

Our goal as attackers is to learn a binary classifier that exploits this memorization gap. Given a specific sample and its true label, we determine whether that sample was in the target model's training set. A successful attack correctly identifies training members with accuracy significantly better than random guessing (50%).

Industry Security Frameworks

Three frameworks catalogue ML privacy risks. The OWASP ML Security Top 10 (draft v0.3) lists membership inference as ML04:2023, alongside model inversion (ML03:2023) and model theft (ML05:2023). It assigns membership inference moderate exploitability (4/5) and moderate impact (4/5), recommending differential privacy and regularization as primary defenses.

The OWASP Top 10 for LLM Applications (2025) addresses generative AI with LLM02: Sensitive Information Disclosure covering training data leakage, and LLM04: Data and Model Poisoning addressing the memorization patterns that enable inference attacks.

Google SAIF takes an architectural approach, categorizing membership inference under Sensitive Data Disclosure and Inferred Sensitive Data. SAIF assigns responsibility to both model creators (implement privacy-preserving training) and model consumers (filter outputs, monitor queries).

All three frameworks converge on the same conclusion: if a model leaks membership, it likely leaks other information too. Differential privacy is the recommended countermeasure, which is why we focus on DP-SGD and PATE in subsequent sections.

The Shadow Model Attack

The shadow model attack, introduced by Shokri et al. in 2017, remains the primary approach for membership inference. The core challenge is that we cannot directly observe membership patterns in the target model because we do not know its training set. We use shadow models to solve this by generating labeled training data for the attack classifier.

We train multiple shadow models that approximate the target model's architecture and training procedure. Since we control these shadow models, we know exactly which samples were in their training sets and which were not. We collect each shadow model's predictions on its training data (members) and held-out data (non-members), labeling each prediction with its membership status. This labeled dataset trains an attack model to classify predictions as coming from members or non-members. We then apply this trained attack model to the target model's predictions.

The intuition is straightforward: if shadow models exhibit similar overfitting patterns to the target model, then a classifier trained to detect membership in shadow models will

generalize to detecting membership in the target.

Why Shadow Models Work

Consider what training an attack classifier requires. We want to learn a function that takes a prediction vector and true label, then outputs whether the sample was a member. Learning this function requires examples of predictions on members and non-members, each labeled with ground truth.

The target model's training set is secret, so we cannot directly obtain this data. But we can create our own models where we know the training membership. If these `shadow models` exhibit the same overfitting patterns as the target (higher confidence on training samples, different error distributions), then an attack classifier trained on `shadow model` predictions will generalize to the target.

The similarity assumption is reasonable when the attacker can approximate three key aspects of the target. First, the model architecture should have similar capacity and structure, as this determines the degree and pattern of overfitting. Second, the training procedure should use similar optimization settings (optimizer type, learning rate, number of epochs), since these affect how much the model memorizes versus generalizes. Third, the data distribution should match the target's training data population, because overfitting patterns depend on the statistical properties of the data.

What Makes Attacks Succeed or Fail

Not all models are equally vulnerable to `membership inference`. Several factors determine attack success, and understanding them helps predict when `MIA` will be effective.

`Model complexity and capacity` directly correlate with vulnerability. Larger models with more parameters can memorize more training examples, making them more susceptible to attack. A 10-layer network with 1 million parameters memorizes more than a 2-layer network with 10,000 parameters. This explains why large language models and deep neural networks face greater `MIA` risk than simple logistic regression models.

We also find an inverse relationship between `training data size` and vulnerability. When we train models on small datasets, they memorize a larger fraction of their training data because each example has more influence on the final weights. A model trained on 1,000 samples is more vulnerable than the same architecture trained on 1 million samples.

With `training duration and regularization`, we have direct control over memorization. When we train models for more epochs without regularization, they overfit more severely. Models with strong dropout (0.5), weight decay, and early stopping exhibit smaller overfitting gaps and resist `MIA` more effectively.

The `number of output classes` changes the attack signal we can exploit. Binary classification provides less information per prediction than 100-class classification. With

more classes, the softmax distribution reveals finer-grained confidence patterns. Attacks on ImageNet classifiers (1000 classes) often achieve higher accuracy than attacks on binary classifiers because the membership signal is richer.

Finally, consider data heterogeneity and which samples become vulnerable. Samples near decision boundaries or with unusual feature combinations are memorized more than typical samples. An attack might achieve 80% accuracy on outlier samples but only 55% on typical samples. MIA vulnerability is not uniform across the training set; some individuals are at higher risk than others.

Shadow model mismatch causes attack failure when the attacker's models differ too much from the target in architecture, training procedure, or data distribution. The learned membership patterns may not transfer. An attack trained on shadow CNNs will likely fail against a target transformer. Attacks trained on CIFAR-10 shadows will fail against a target trained on medical images. The shadow-target similarity assumption is the key limitation of this approach.

Attack Model Architecture

Because shadow models serve as proxies, we should match the target's architecture as closely as possible since overfitting behavior depends on model capacity and structure. We train multiple shadow models on different random subsets of our available data. This diversity helps the attack model learn robust membership signals that generalize across different training sets instead of memorizing artifacts of any single model.

Our attack model takes as input the target model's prediction vector and the true class label, then outputs a binary classification. We concatenate the softmax probabilities with a one-hot encoding of the true label, allowing the attack model to learn class-specific membership patterns. Different classes may exhibit different overfitting characteristics, so conditioning on the true label improves attack accuracy. We typically implement the attack model as a simple neural network because the membership signal, while subtle, is relatively low-dimensional.

Threat Model Assumptions

The membership inference attacker we consider has black-box access to the target model: they can submit inputs and observe corresponding prediction outputs, but cannot inspect model parameters, gradients, or internal activations. This constraint reflects realistic deployment scenarios where models are served through APIs or embedded in applications. The attacker receives prediction probability vectors (the full softmax output) rather than just class labels, which provides richer information about the model's confidence.

Query limits pose minimal obstacles. In our analysis, we assume the attacker can query the model as many times as needed without rate limiting or detection. Real-world APIs often impose query limits (1000 requests per minute, 10000 per day), but MIA typically requires relatively few queries per target sample (just one query to get the prediction). The expensive

part is training `shadow models`, which happens offline and does not touch the target. An attacker targeting 100 individuals needs only 100 queries to the target model, well within typical API limits.

Model owners cannot easily detect the attack. `MIA` queries look identical to legitimate inference requests. The attacker submits a normal input and receives a normal prediction. Without knowing which specific individuals the attacker targets, the model owner cannot distinguish attack queries from benign usage. This makes `MIA` harder to detect compared to attacks that require unusual query patterns (like model extraction attacks that systematically probe the input space).

The attacker also knows the training data distribution. They might possess data from the same population (medical records from the same hospital system, financial records from the same region) without knowing the exact samples used for training. This assumption is realistic in many scenarios. Consider a hospital that trains a model on patient records. An attacker working at a nearby hospital has access to similar patient demographics, disease distributions, and treatment patterns. A competitor building a similar product has collected their own dataset from the same population. Research datasets like Adult Census are publicly available, so anyone can obtain data matching the training distribution. Training data rarely comes from secret sources; it typically comes from identifiable populations, making distribution matching feasible.

Attackers with more knowledge achieve stronger results. With `white-box access` (full model parameters), attacks become significantly easier because the attacker can compute exact loss values on target samples. With knowledge of the training procedure (learning rate, epochs, batch size), shadow models can more closely replicate target behavior. We focus on the minimal black-box setting because it represents the hardest case for attackers and the most realistic deployment scenario.

Alternative Attack Approaches

The `shadow model attack` requires substantial effort: training multiple models, collecting predictions, building an attack classifier. Simpler alternatives exist that trade some accuracy for reduced complexity.

The simplest alternative is `metric-based attacks`, which skip shadow model training entirely by using statistical thresholds on prediction confidence. The intuition is direct: if the model is highly confident about a prediction, the sample was probably in training. Setting a threshold at 0.9 confidence and classifying all samples above it as members achieves surprisingly good results on vulnerable models. This approach requires no training whatsoever, just a single threshold chosen based on expected model behavior.

`Loss-based attacks` take a slightly different approach by computing the model's loss on each target sample. Members should have lower loss because the model was optimized to minimize loss on exactly these examples. Given a sample with true label and the model's

prediction, we compute cross-entropy loss and threshold it. Samples with loss below the threshold are classified as members. This approach requires knowing the true label for each target sample, which the `shadow model attack` also assumes.

For those seeking optimal statistical efficiency, `likelihood ratio attacks` offer a Bayesian approach that computes probability ratios between training and reference distributions. Instead of learning a classifier, we model the distribution of predictions on members versus non-members and compute which is more likely for a new sample. This approach can achieve optimal statistical efficiency but requires careful distribution modeling.

We focus on the shadow model approach because it achieves strong empirical results without requiring access to the target's training procedure and generalizes across different model architectures. Shadow models learn nuanced membership patterns that simple thresholds cannot capture, making them effective even when the confidence gap is small.

Setup Overview

This module uses the `htb_ai_library` package for models, training utilities, data loading, and visualization. This section explains the setup and configuration choices that directly affect attack success.

Installing Dependencies

This module requires several Python packages for machine learning, privacy-preserving training, and visualization. Install all dependencies within your environment:

```
pip install torch torchvision numpy scikit-learn matplotlib tqdm safetensors opacus flask
```

These packages provide:

Package	Purpose
<code>torch</code> , <code>torchvision</code>	PyTorch deep learning framework and vision utilities
<code>numpy</code>	Numerical computing and array operations
<code>scikit-learn</code>	Data preprocessing, metrics, and train/test splitting
<code>matplotlib</code>	Visualization and plotting
<code>tqdm</code>	Progress bars for training loops
<code>safetensors</code>	Secure model weight serialization for challenge submissions
<code>opacus</code>	Differential privacy for PyTorch (DP-SGD implementation)
<code>flask</code>	Web API framework used by challenge evaluators

Next, install the HTB AI library from GitHub. This library provides pre-built models, training utilities, data loaders, and visualization functions used throughout the module:

```
pip install --upgrade git+https://github.com/PandaSt0rm/htb-ai-library
```

Imports and Configuration

Start your training script with the standard library imports and our custom library:

```
import os
import json
import numpy as np
import torch
import torch.nn.functional as F
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, precision_score, recall_score,
f1_score, classification_report
```

Next, import the attack-specific components from our library:

```
from htb_ai_library import (
    set_reproducibility, use_htb_style,
    MLP, AttackModel,
    load_adult_census,
    train_fixed_epochs, train_with_early_stopping, evaluate_model,
    get_model_predictions, prepare_attack_data, create_dataloader,
    plot_training_history, plot_overfitting_gap,
    plot_confidence_distributions,
    plot_shadow_confidence_distributions, plot_attack_roc_curve,
    plot_precision_recall_curve,
    plot_attack_accuracy_comparison, analyze_attack_decision_boundary,
    plot_decision_boundary,
)
```

Now configure the execution environment. Setting `RANDOM_SEED = 1337` and calling `set_reproducibility()` ensures identical results across runs:

```
RANDOM_SEED = 1337
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
set_reproducibility(RANDOM_SEED)
use_htb_style()
```


Finally, set up output directories for saving models and figures:

```
OUTPUT_DIR = "output"
MODEL_DIR = f"{OUTPUT_DIR}/models"
FIGS_DIR = "figs"
FIG_PREFIX = "Introduction_"
os.makedirs(MODEL_DIR, exist_ok=True)
os.makedirs(FIGS_DIR, exist_ok=True)

DATASET_CONFIG = {
    "num_classes": 2,
}
```

Library Components

We import several building blocks from `htb_ai_library` to construct our `membership inference` attack. Understanding how these components fit together clarifies the attack pipeline before we implement it.

To ensure our experiments produce identical results across runs, we call `set_reproducibility(seed)` at the start of every script. This function configures PyTorch, NumPy, and CUDA random number generators with our chosen seed value. Reproducibility matters because comparing attack performance under different conditions requires eliminating variation from randomness that would confound our measurements.

Two neural network classes power our attack. We use `MLP` (Multi-Layer Perceptron) in dual roles: it becomes both the target model we attack and the `shadow models` we train to learn membership patterns. When creating an `MLP`, we specify hidden layer sizes, dropout rates, and the number of output classes. We'll call `predict_proba()` to obtain softmax probabilities that our attack will analyze. We configure the target model with a larger architecture `[256, 128]` and zero dropout to maximize overfitting, while `shadow models` use a smaller `[128, 64]` architecture with moderate dropout since they only need to exhibit similar overfitting behavior.

We build the `AttackModel` to distinguish members from non-members. Its input structure differs from the `MLP`: instead of raw features, it receives prediction probabilities concatenated with one-hot encoded true labels in the format `[prob_class_0, prob_class_1, label_0, label_1]`. This design lets the `attack model` learn class-specific confidence patterns, since some classes may exhibit stronger overfitting signals than others.

To load our dataset, we call `load_adult_census()`, which fetches the Adult Census dataset from OpenML, preprocesses categorical features, and creates three disjoint splits. We receive scaled numpy arrays ready for training: target training data (members), shadow

training data, and attack evaluation data (non-members). This strict separation prevents data leakage that would artificially inflate attack success.

For training, we use two functions depending on our goals. To deliberately overfit the target model, we call `train_fixed_epochs()`, which runs training for a specified number of epochs without early stopping. For `shadow models` and the `attack model`, we use `train_with_early_stopping()`, which monitors validation loss and restores the best model weights when training stagnates. Both functions return a history dictionary containing per-epoch training and validation losses and accuracies for visualization.

To measure performance, we call `evaluate_model()` with a trained model and `DataLoader`, receiving accuracy, predictions, and probability outputs. We use this to compare performance on training data versus test data, revealing the overfitting gap. When we need predictions without setting up `DataLoaders` manually, we use `get_model_predictions()`, which handles batching and device transfers internally. Pass a trained model and numpy array, receive prediction probabilities back.

To prepare training data for the attack classifier, we call `prepare_attack_data()` with member predictions, non-member predictions, and their true labels. This function constructs feature vectors `[predictions, one_hot_labels]` and binary membership labels (1 for member, 0 for non-member). The output feeds directly into `attack model` training. We wrap our numpy arrays into PyTorch `DataLoaders` using `create_dataloader()`, which accepts configurable batch size and shuffling options.

To interpret our results, we use several visualization functions. We call `plot_training_history()` to render dual-panel training curves showing loss and accuracy over epochs for both training and validation sets. The divergence between curves reveals overfitting. To quantify this gap, `plot_overfitting_gap()` creates a bar chart comparing training accuracy (members) versus test accuracy (non-members). The gap between these bars measures the behavioral difference that enables `membership inference`.

We visualize confidence patterns using `plot_confidence_distributions()`, which overlays histograms of `prediction confidence` for members versus non-members. Members typically show higher confidence, and the separation between distributions indicates attack potential. A related function, `plot_shadow_confidence_distributions()`, shows these distributions across all `shadow models`.

Three complementary visualizations evaluate attack performance. We generate the receiver operating characteristic curve using `plot_attack_roc_curve()`, which includes the area under curve (AUC) metric where AUC above 0.5 indicates better-than-random performance. To examine the precision-recall tradeoff (useful when member and non-member classes are imbalanced), we use `plot_precision_recall_curve()`. For a summary view, `plot_attack_accuracy_comparison()` displays a bar chart comparing all metrics against the 0.5 random baseline.

To understand what the attack learned, we use `analyze_attack_decision_boundary()` to probe the `attack_model` and determine confidence thresholds for membership prediction. We then visualize these thresholds with `plot_decision_boundary()`, showing how membership probability varies with `prediction confidence` for each class.

Configuration: Maximizing the Overfitting Gap

The attack exploits the behavioral difference between how models treat training data versus unseen data. Our configuration deliberately maximizes this gap:

```
TARGET_MODEL_CONFIG = {
    "hidden_layers": [256, 128],
    "dropout": 0.0, # No dropout to maximize overfitting
    "epochs": 100,
    "batch_size": 32,
    "learning_rate": 0.001,
}
```

Zero dropout removes regularization that would prevent memorization. We train for a fixed number of epochs without early stopping to maximize overfitting. This allows the model to continue memorizing training data well past the optimal generalization point. In production, these would be mistakes. For our demonstration, they create a vulnerable model that MIA can exploit.

Shadow models use different settings because they serve a different purpose:

```
SHADOW_MODEL_CONFIG = {
    "num_shadow_models": 5,
    "hidden_layers": [128, 64],
    "dropout": 0.3,
    "epochs": 100,
    "batch_size": 64,
    "learning_rate": 0.001,
    "early_stopping_patience": 10,
    "shadow_data_size": 0.5,
}
```

Smaller architecture and moderate dropout make `shadow models` train faster while still exhibiting detectable overfitting patterns. The `attack_model` learns from `shadow model` behavior, so `shadow models` need only enough overfitting to generate representative membership signals.

The `attack_model` needs minimal capacity since it learns a relatively simple decision boundary (higher confidence suggests membership):

```

ATTACK_MODEL_CONFIG = {
    "hidden_layers": [64, 32],
    "dropout": 0.2,
    "epochs": 100,
    "batch_size": 128,
    "learning_rate": 0.001,
    "early_stopping_patience": 15,
}

```

A small architecture [64, 32] with light dropout (0.2) prevents the attack model from overfitting to quirks of specific shadow models. The larger batch size (128) provides stable gradients for the simpler 4-dimensional input, and extended patience (15 epochs) allows subtle membership patterns to emerge during training.

Data Splitting Strategy

Before loading data, we should understand how the dataset is partitioned. The `load_adult_census()` function creates three disjoint datasets:

```

Total Dataset (48,842 samples)
├─ Target Training (24,421) → Members we try to identify
├─ Holdout (24,421)
│   └─ Shadow Training (12,210) → Train shadow models
│   └─ Attack Evaluation (12,210) → Non-members for final testing

```

This separation is critical. If attack evaluation data overlapped with target training data, we would inflate attack success metrics by testing on samples we already know are members.

Loading Data

With the data splits understood, we load the Adult Census dataset:

```

print("Loading Adult Census dataset...")
X_target, y_target, X_shadow, y_shadow, X_attack_eval, y_attack_eval,
num_features = load_adult_census(
    random_state=RANDOM_SEED
)

print(f"Dataset loaded: {num_features} features")
print(f"  Target training (members): {len(X_target)} samples")
print(f"  Shadow training: {len(X_shadow)} samples")
print(f"  Attack evaluation (non-members): {len(X_attack_eval)} samples")

```

Training the Target Model

The target model is the victim we will attack. We train it to deliberately overfit, starting with data preparation:

```
print("\n" + "=" * 60)
print("Training Target Model")
print("=" * 60)

scaler = StandardScaler()
X_target_norm = scaler.fit_transform(X_target)
X_attack_eval_norm = scaler.transform(X_attack_eval)
```

We fit the `StandardScaler` on target training data and use `transform()` (not `fit_transform()`) on evaluation data. This ensures both datasets use identical normalization parameters.

Next, create `DataLoaders` without a validation split. We deliberately omit validation because we want maximum overfitting:

```
train_loader = create_dataloader(X_target_norm, y_target,
                                TARGET_MODEL_CONFIG['batch_size'])
test_loader = create_dataloader(X_attack_eval_norm, y_attack_eval,
                                TARGET_MODEL_CONFIG['batch_size'],
                                shuffle=False)
```

Initialize the target model with zero dropout to remove regularization:

```
target_model = MLP(
    input_size=num_features,
    hidden_layers=TARGET_MODEL_CONFIG['hidden_layers'],
    num_classes=DATASET_CONFIG['num_classes'],
    dropout=TARGET_MODEL_CONFIG['dropout']
)

print(f"Architecture: {num_features} ->
{TARGET_MODEL_CONFIG['hidden_layers']} -> 2")
print(f"Training for {TARGET_MODEL_CONFIG['epochs']} epochs (no early
stopping)")
```

Unlike typical training where we'd use early stopping, we intentionally train for the full 100 epochs to maximize overfitting:

```

history = train_fixed_epochs(
    target_model, train_loader, test_loader,
    device=DEVICE,
    epochs=TARGET_MODEL_CONFIG['epochs'],
    learning_rate=TARGET_MODEL_CONFIG['learning_rate']
)

```

The `history` dictionary captures per-epoch metrics: training loss, validation loss, training accuracy, and validation accuracy. Watching these diverge over time reveals how the model progressively memorizes training data. By epoch 30-40, training loss typically continues decreasing while validation loss starts climbing, the classic overfitting signature.

Now we quantify the overfitting gap:

```

train_acc, _, _ = evaluate_model(target_model, train_loader, DEVICE)
test_acc, _, _ = evaluate_model(target_model, test_loader, DEVICE)

print(f"\nTarget Model Performance:")
print(f"  Training Accuracy: {train_acc:.4f}")
print(f"  Test Accuracy:      {test_acc:.4f}")
print(f"  Overfitting Gap:    {train_acc - test_acc:.4f}")

plot_overfitting_gap(train_acc, test_acc,
                     save_path=os.path.join(FIGS_DIR, f"
{FIG_PREFIX}overfitting_gap.png"))

```

Model Architectures

We use the `MLP` class from `htb_ai_library` for both target and shadow models. The key method for our attack is `predict_proba()`:

```

def predict_proba(self, x):
    logits = self.forward(x)
    return F.softmax(logits, dim=1)

```

This returns calibrated probabilities, not raw logits. The attack analyzes these confidence values because members tend to receive higher-confidence predictions. A member might get `[0.05, 0.95]` while a non-member with identical features gets `[0.15, 0.85]`. This confidence gap is the signal our attack exploits.

The `AttackModel` class takes a different input structure: prediction probabilities concatenated with one-hot encoded true labels. This design lets the `attack_model` learn

class-specific confidence patterns. We cover the exact feature format when we prepare attack training data in the shadow model training section.

The Overfitting Gap

When training the target model, you will see output like:

Target Model Performance:

Training Accuracy: 0.9012

Test Accuracy: 0.8456

Overfitting Gap: 0.0556

This 5.5% gap means the model correctly classifies 90% of training samples but only 85% of unseen samples. The model behaves differently on data it has seen versus data it has not. This behavioral difference, consistent across tens of thousands of samples, provides the statistical foundation for membership inference.

Let's examine how this gap develops over time:



Notice the divergence around epoch 20: training loss continues decreasing while validation loss starts climbing. Training accuracy reaches 90% while validation accuracy stagnates near 83%. This classic overfitting pattern shows the model memorizing training examples instead of learning generalizable patterns.



We can quantify this gap directly: 90.2% accuracy on training data (members) versus 83.2% on test data (non-members). This 7.1% difference represents the vulnerability our attack will exploit. The model treats members and non-members measurably differently.

The next sections use these components to implement the attack: training shadow models to generate labeled membership data, building an attack classifier that learns membership patterns, and executing the attack against the target model.

Training Shadow Models

The previous sections established why shadow models work and what factors affect attack success. Now we implement the training pipeline. Each shadow model trains on a different random subset of shadow data, and we collect predictions on both member and non-member samples to build the attack training dataset.

We train each shadow model on a different random subset of our shadow data. This diversity matters because training all shadow models on identical data would cause the

attack classifier to learn patterns specific to that particular split instead of general membership signals that transfer to the target. Five models strikes the right balance: fewer models produce insufficient diversity in overfitting patterns, causing the attack classifier to memorize artifacts of individual models, while more models provide diminishing returns and increase computational cost linearly.

Creating Shadow Model Data Splits

```
print("\n" + "=" * 60)
print("Training Shadow Models")
print("=" * 60)

shadow_splits = []
for i in range(SHADOW_MODEL_CONFIG['num_shadow_models']):
    seed = RANDOM_SEED + i
    X_train_s, X_out_s, y_train_s, y_out_s = train_test_split(
        X_shadow, y_shadow,
        train_size=SHADOW_MODEL_CONFIG['shadow_data_size'],
        random_state=seed, stratify=y_shadow
    )
    shadow_splits.append((X_train_s, X_out_s, y_train_s, y_out_s))

print(f"\nCreated {len(shadow_splits)} shadow model data splits")
print(f"Samples per shadow model: ~{len(shadow_splits[0][0])} in, ~
{len(shadow_splits[0][1])} out")
```

Training and Collecting Predictions

Now we iterate through each split, training a shadow model and collecting its predictions on both in-training (member) and out-of-training (non-member) samples.

Our task is to accumulate attack training data from all `shadow models` into combined arrays. Combining data from multiple models (rather than training on each separately) exposes the attack classifier to diverse overfitting patterns, preventing it from memorizing quirks of any single model. The variety acts as implicit regularization, producing an attack that generalizes better to the unseen target model.

We use Python lists instead of pre-allocated NumPy arrays because we don't know the exact sample counts upfront (each `shadow model`'s train/validation split varies slightly). Lists handle dynamic appending efficiently, and we convert to NumPy only after all 5 `shadow models` complete.

With each `shadow model` contributing approximately 12,210 samples (6,105 members + 6,105 non-members), we expect around 61,050 total attack training examples. The separate

`all_preds_in` and `all_preds_out` lists store raw 2D probability arrays (`samples, 2`) for the confidence distribution visualization, while `all_attack_X` stores the transformed 4D attack features (`samples, 4`).

We follow the same lifecycle for each shadow model: normalize its data, create train/validation splits, train with early stopping, and collect predictions on both member and non-member samples. Pay particular attention to normalization for attack transferability. We use the same scaler fitted on target data (calling `transform`, not `fit_transform`) so that identical raw feature values produce identical normalized values across all models. If we fitted separate scalers, prediction differences would partly reflect normalization differences instead of pure membership signals.

```
all_attack_X = []
all_attack_y = []
all_preds_in = []
all_preds_out = []

for i, (X_train_s, X_out_s, y_train_s, y_out_s) in enumerate(shadow_splits):
    print(f"\nTraining Shadow Model
{i+1}/{SHADOW_MODEL_CONFIG['num_shadow_models']}")

    # Normalize using target scaler for transferability
    X_train_s_norm = scaler.transform(X_train_s)
    X_out_s_norm = scaler.transform(X_out_s)

    # Create validation split for early stopping
    X_tr_s, X_val_s, y_tr_s, y_val_s = train_test_split(
        X_train_s_norm, y_train_s, test_size=0.2,
        random_state=RANDOM_SEED + i, stratify=y_train_s
    )
    train_loader_s = create_dataloader(X_tr_s, y_tr_s,
SHADOW_MODEL_CONFIG['batch_size'])
    val_loader_s = create_dataloader(X_val_s, y_val_s,
SHADOW_MODEL_CONFIG['batch_size'], shuffle=False)
```

We allocate 20% of each shadow model's training data for validation, leaving approximately 4,884 samples for actual training and 1,221 for validation. Using different random seeds for each split (`RANDOM_SEED + i`) ensures variety across shadow models.

```
# Initialize and train shadow model
shadow_model = MLP(
    input_size=num_features,
    hidden_layers=SHADOW_MODEL_CONFIG['hidden_layers'],
```

```

        num_classes=DATASET_CONFIG['num_classes'],
        dropout=SHADOW_MODEL_CONFIG['dropout']
    )
    train_with_early_stopping(
        shadow_model, train_loader_s, val_loader_s,
        device=DEVICE,
        epochs=SHADOW_MODEL_CONFIG['epochs'],
        learning_rate=SHADOW_MODEL_CONFIG['learning_rate'],
        patience=SHADOW_MODEL_CONFIG['early_stopping_patience'],
        verbose=False
    )

```

After training completes, we collect predictions on both member and non-member samples.

```

# Collect predictions on members and non-members
preds_in = get_model_predictions(shadow_model, X_train_s_norm, DEVICE)
preds_out = get_model_predictions(shadow_model, X_out_s_norm, DEVICE)

# Transform to attack features and accumulate
attack_X_s, attack_y_s = prepare_attack_data(preds_in, preds_out,
y_train_s, y_out_s)
all_attack_X.append(attack_X_s)
all_attack_y.append(attack_y_s)
all_preds_in.append(preds_in)
all_preds_out.append(preds_out)

```

`preds_in` contains predictions on samples the model optimized for during training (members), while `preds_out` contains predictions on samples the model never saw (non-members). These two prediction sets exhibit the behavioral difference our attack will learn to detect. Each call to `get_model_predictions()` returns softmax probability vectors for all samples in the input array. For binary classification, each prediction is a 2-element array like `[0.15, 0.85]`.

We also verify that each shadow model exhibits an overfitting gap:

```

# Verify overfitting gap exists
full_train_loader_s = create_dataloader(X_train_s_norm, y_train_s,
SHADOW_MODEL_CONFIG['batch_size'], shuffle=False)
full_out_loader_s = create_dataloader(X_out_s_norm, y_out_s,
SHADOW_MODEL_CONFIG['batch_size'],
shuffle=False)
train_acc_s, _, _ = evaluate_model(shadow_model, full_train_loader_s,

```

```

DEVICE)
    out_acc_s, _, _ = evaluate_model(shadow_model, full_out_loader_s,
DEVICE)
    print(f" Shadow {i+1} - Train Acc: {train_acc_s:.4f}, Out Acc:
{out_acc_s:.4f}")

```

The `prepare_attack_data()` function concatenates prediction probabilities with one-hot encoded true labels, creating 4-dimensional feature vectors. It also assigns membership ground truth: label 1 for in-training samples (members) and label 0 for out-of-training samples (non-members).

Running this code produces output like `Shadow 1 - Train Acc: 0.8612, Out Acc: 0.8423`, showing training accuracy around 86% and holdout accuracy around 84%. Each model exhibits a 1-2% overfitting gap similar to the target, confirming that our shadow models capture representative behavior.

Combining Attack Training Data

With all five `shadow models` trained and their predictions collected, we now merge the accumulated data into single arrays for `attack model` training. Each shadow model contributed approximately 12,210 samples (6,105 members + 6,105 non-members), so our combined dataset contains about 61,050 total samples.

We use `np.concatenate` to stack arrays along the first axis (samples), preserving the feature dimension. The resulting `attack_X` has shape `(61050, 4)` and `attack_y` has shape `(61050,)`. This combined dataset exposes the `attack model` to diverse overfitting patterns from multiple `shadow models`, preventing it from overfitting to quirks of any single model.

```

attack_X = np.concatenate(all_attack_X, axis=0)
attack_y = np.concatenate(all_attack_y, axis=0)

print(f"\nTotal attack training samples: {len(attack_X)}")
print(f" Members: {np.sum(attack_y == 1)}")
print(f" Non-members: {np.sum(attack_y == 0)}")

```

The dataset is balanced: roughly 30,525 members and 30,525 non-members. This balance matters because an imbalanced attack dataset would bias the classifier toward the majority class, causing it to predict one label regardless of actual confidence patterns.

Understanding Attack Features

The attack feature vector structure was introduced in the Setup Overview. Each 4D vector contains softmax probabilities concatenated with one-hot encoded true labels: `[prob_0,`

prob_1, label_0, label_1] . Let's verify the data:

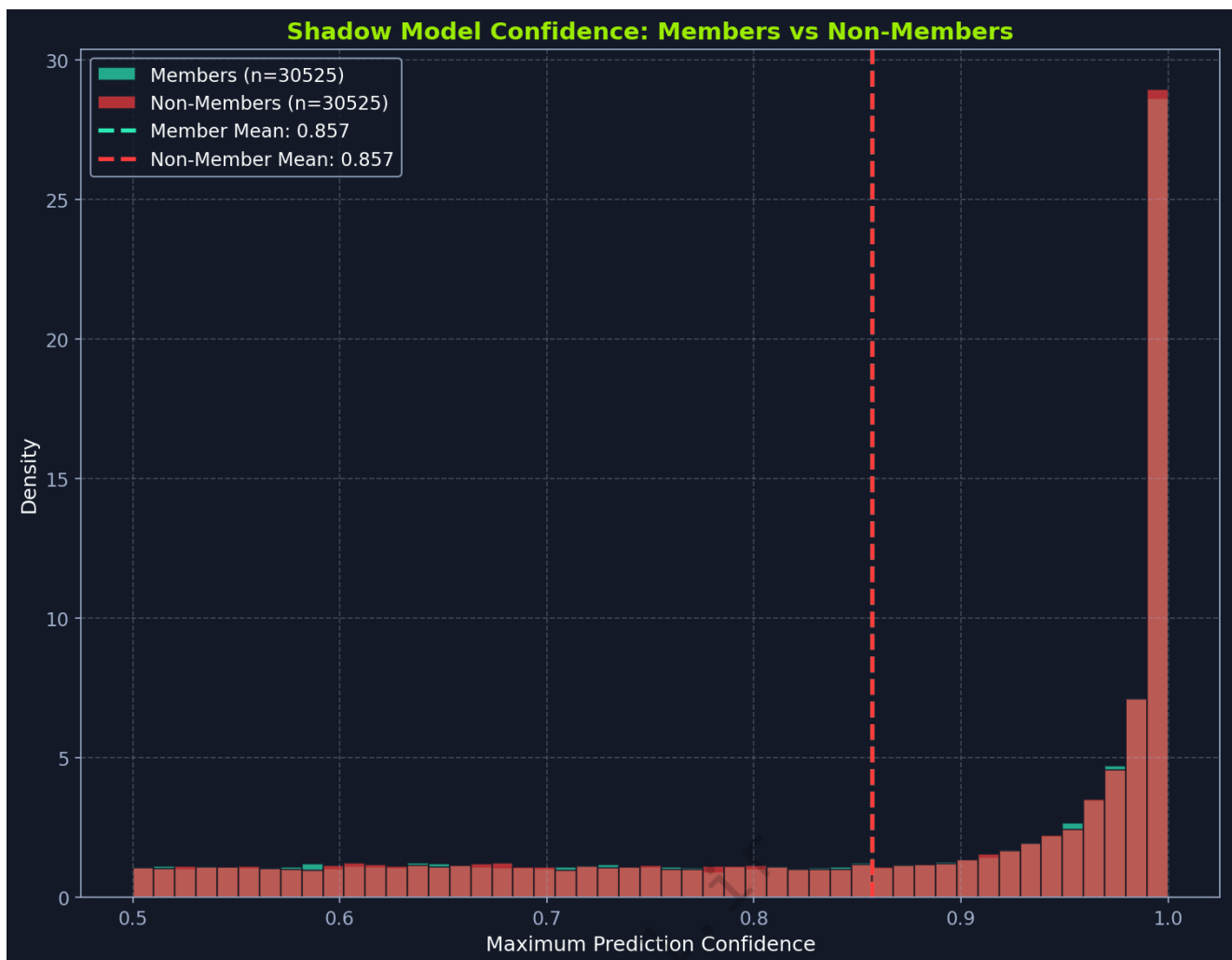
```
print(f"\nAttack feature dimensions: {attack_X.shape[1]}")
print(f"Example member feature: {attack_X[0].round(3)}")
print(f"Example non-member feature: {attack_X[len(attack_X)//2].round(3)}")
```

We typically see members show higher confidence for the true class (e.g., [0.12, 0.88, 0.0, 1.0]) while non-members with the same true label show lower confidence (e.g., [0.25, 0.75, 0.0, 1.0]). This confidence gap is the signal our attack exploits. As discussed in the first section, our `attack_model` extracts two distinct signals from these features: `prediction confidence` and `prediction correctness` . These signals combine to create the membership fingerprint our attack learns to detect.

Visualizing Shadow Model Behavior

We use `plot_shadow_confidence_distributions()` to visualize what the membership signal looks like, showing confidence distributions for members versus non-members across all `shadow models` .

```
plot_shadow_confidence_distributions(
    all_preds_in, all_preds_out,
    save_path=os.path.join(FIGS_DIR, f"{FIG_PREFIX}shadow_confidence.png")
)
```

The histogram reveals the challenge our attack faces. Both distributions cluster heavily at high confidence values (0.85+) with nearly identical means, often within 0.5% of each other. Because shadow models use dropout regularization and early stopping, they exhibit minimal overfitting and the expected confidence gap essentially disappears. This substantial overlap explains why membership inference on shadow data is difficult: well-regularized models treat members and non-members almost identically. However, the slight distributional differences, especially in the tails, still provide a learnable signal. The attack model will learn to exploit these subtle patterns across the 61,050 training samples.

Attack Data Statistics

Before moving to attack model training, let's examine the statistics of our attack dataset.

```
member_confidences = attack_X[attack_y == 1, :2].max(axis=1)
non_member_confidences = attack_X[attack_y == 0, :2].max(axis=1)

print(f"\nAttack Data Statistics:")
print(f"  Member confidence - Mean: {member_confidences.mean():.4f}, Std: {member_confidences.std():.4f}")
print(f"  Non-member confidence - Mean: {non_member_confidences.mean():.4f}, Std: {non_member_confidences.std():.4f}")
```

```
print(f" Confidence gap: {member_confidences.mean() -  
non_member_confidences.mean():.4f}")
```

The confidence gap (typically 0.03-0.05) represents the signal our attack will amplify. While this difference seems small, it is consistent across tens of thousands of samples. The `attack_model` learns to detect this subtle but reliable pattern.

With our attack training data prepared, the next section builds the attack classifier that learns to distinguish members from non-members based on these prediction patterns.

Building the Attack Classifier

We now have 61,050 labeled examples of shadow model behavior on members and non-members. The next step is training a classifier that learns to distinguish these two groups based on their prediction patterns. This `attack_model` will then be applied to the target model's predictions to infer membership.

Our `attack_model` must overcome a key generalization challenge: it needs to learn membership patterns from `shadow models` that transfer to the target model, which has different weights and potentially different overfitting characteristics. If we overfit the `attack_model` to shadow-specific artifacts (peculiarities of individual `shadow model` training runs), performance on the target will suffer. We address this through three mechanisms: training on diverse data from multiple shadow models, using a small architecture (64, 32 neurons) that cannot memorize shadow-specific patterns, and validating on held-out shadow data to detect overfitting before deployment.

Splitting Attack Data

We use three splits instead of two because transfer learning requires careful validation. The training set teaches membership patterns, the validation set guides early stopping to prevent memorizing shadow-specific artifacts, and the test set estimates performance on the genuinely unseen target. With only train/test, we'd either overfit to `shadow models` (no early stopping) or waste valuable data on validation (smaller training set).

We allocate 20% of attack data for testing, approximately 12,210 samples. Stratification ensures both members and non-members are proportionally represented in each split. The test set remains untouched during training, providing an unbiased estimate of attack performance.

We further subdivide the training portion to create a validation set for early stopping. Without validation data, we could not detect when the `attack_model` starts overfitting to the specific `shadow model` predictions in the training set. We use the same 80/20 split ratio as for other models in our pipeline. This yields approximately 39,072 training samples, 9,768 validation samples, and 12,210 test samples, providing sufficient data at each stage while maintaining held-out test integrity.

We configure a larger batch size of 128 (versus 64 for target/ shadow models) because the attack model has fewer parameters and processes smaller input features (4 dimensions versus 14). Larger batches provide more stable gradient estimates and faster training without exceeding memory limits.

```
print("\n" + "=" * 60)
print("Training Attack Model")
print("=" * 60)

X_attack_train, X_attack_test, y_attack_train, y_attack_test =
train_test_split(
    attack_X, attack_y, test_size=0.2, random_state=RANDOM_SEED,
    stratify=attack_y
)

print(f"\nAttack data split:")
print(f"  Training + Validation: {len(X_attack_train)} samples")
print(f"  Test: {len(X_attack_test)} samples")
```

Now subdivide the training portion to create a validation set for early stopping:

```
X_attack_tr, X_attack_val, y_attack_tr, y_attack_val = train_test_split(
    X_attack_train, y_attack_train, test_size=0.2, random_state=RANDOM_SEED,
    stratify=y_attack_train
)

print(f"  Training: {len(X_attack_tr)} samples")
print(f"  Validation: {len(X_attack_val)} samples")
```

Finally, wrap the arrays in DataLoaders. We set `shuffle=False` for validation and test loaders to ensure consistent evaluation:

```
attack_train_loader = create_dataloader(X_attack_tr, y_attack_tr,
    ATTACK_MODEL_CONFIG['batch_size'])
attack_val_loader = create_dataloader(X_attack_val, y_attack_val,
    ATTACK_MODEL_CONFIG['batch_size'], shuffle=False)
attack_test_loader = create_dataloader(X_attack_test, y_attack_test,
    ATTACK_MODEL_CONFIG['batch_size'], shuffle=False)

print(f"\nDataLoaders created with batch size
{ATTACK_MODEL_CONFIG['batch_size']}")
```

Attack Model Architecture

The `attack_model` needs minimal capacity. Unlike the target model which must learn complex relationships between 14 demographic features and income, the `attack_model` distinguishes membership from a 4-dimensional input where the primary signal is simply confidence level. A smaller network reduces the risk of overfitting to idiosyncrasies of specific `shadow_models` that wouldn't transfer to the target.

```
attack_input_size = attack_X.shape[1]
attack_model = AttackModel(
    input_size=attack_input_size,
    hidden_layers=ATTACK_MODEL_CONFIG['hidden_layers'],
    dropout=ATTACK_MODEL_CONFIG['dropout']
)

print(f"\nAttack model architecture: {attack_input_size} ->
{ATTACK_MODEL_CONFIG['hidden_layers']} -> 2")
print(f"Dropout: {ATTACK_MODEL_CONFIG['dropout']}")
```

The resulting architecture has only 2,600 parameters: $4 \times 64 + 64$ bias = 320 for the first layer, $64 \times 32 + 32 = 2,080$ for the second, and $32 \times 2 + 2 = 66$ for the output. Compare this to the target model's 37,000 parameters. We use 0.2 dropout (versus 0.3 in `shadow_models`) because the simpler task has less overfitting risk, and we want to preserve signal in the already-small network.

The decision boundary is straightforward: higher confidence suggests membership. The true label conditioning allows class-specific thresholds, but the underlying pattern is simple. A more complex architecture would risk overfitting to noise in the shadow model predictions.

Training the Attack Model

We train our `attack_model` on the same infrastructure as target and `shadow_models`, using the `train_with_early_stopping()` function with attack-specific hyperparameters. We set a longer early stopping patience (15 epochs versus 10 for shadow models) to allow more time for the subtle membership patterns to emerge, since the confidence differences we're learning from are smaller than the class differences the classification models learn.

Training typically converges in 20-40 epochs with validation accuracy hovering near 50% throughout. This near-random performance on shadow data is expected: `shadow_models` use dropout regularization and early stopping, which reduces their overfitting and makes member/non-member predictions nearly indistinguishable. The validation accuracy on shadow data does not predict attack performance on the target.

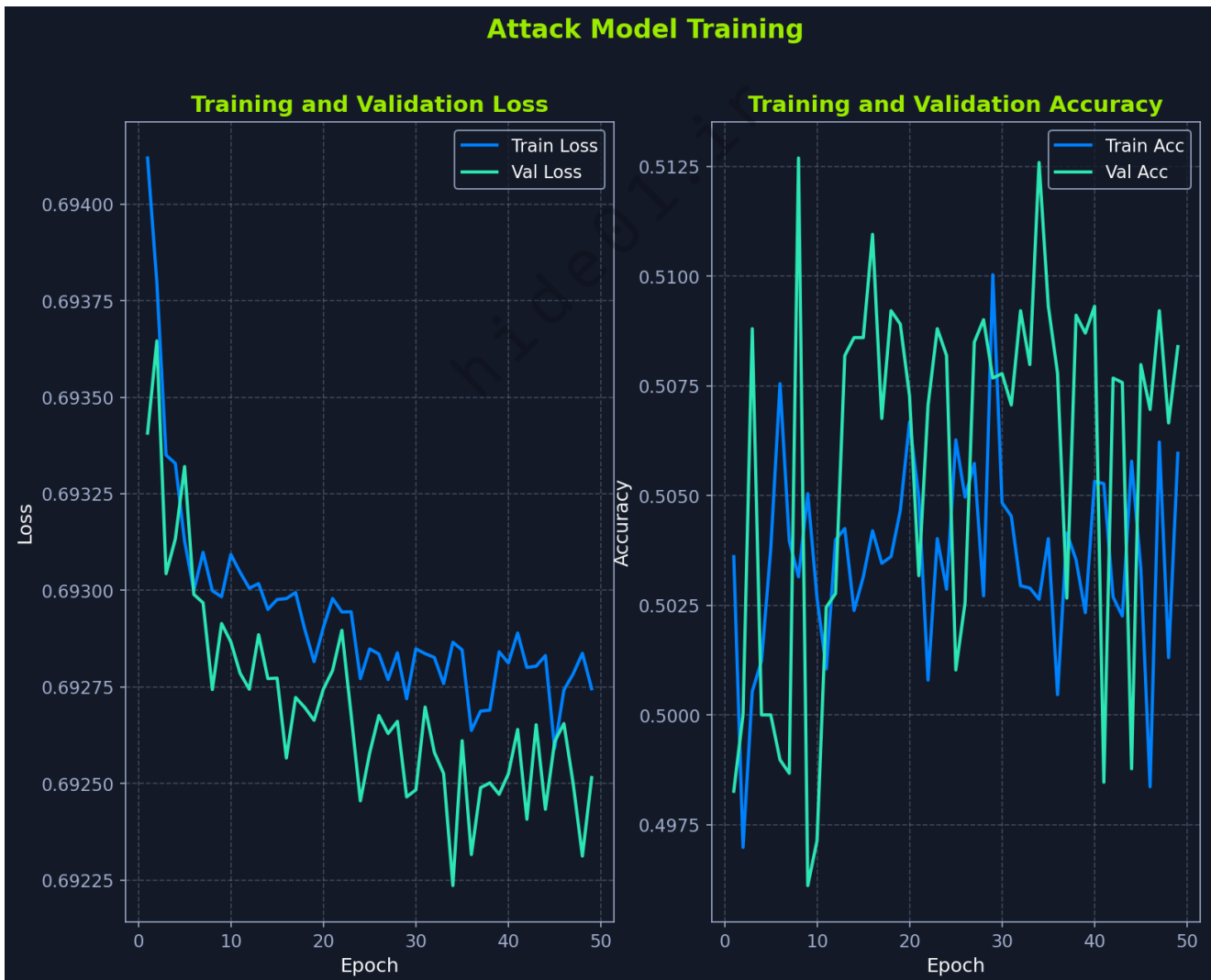
```

print("\nTraining attack model...")

history_attack = train_with_early_stopping(
    attack_model, attack_train_loader, attack_val_loader,
    device=DEVICE,
    epochs=ATTACK_MODEL_CONFIG['epochs'],
    learning_rate=ATTACK_MODEL_CONFIG['learning_rate'],
    patience=ATTACK_MODEL_CONFIG['early_stopping_patience']
)

plot_training_history(
    history_attack,
    "Attack Model Training",
    save_path=os.path.join(FIGS_DIR, f"{FIG_PREFIX}attack_training.png")
)

```



Looking at the training curves, we see the subtle nature of membership inference. Loss decreases only marginally (from 0.694 to 0.693), and accuracy hovers around 50-51% throughout training. This near-random performance reflects the weak membership signal in

shadow models : their dropout regularization and early stopping produce nearly identical confidence distributions for members and non-members. The attack model learns patterns that barely help on shadow data but will prove more effective on the target, which exhibits stronger overfitting.

Evaluating Attack Model Performance

Before applying the attack to the target model, we evaluate its performance on held-out shadow model data. The test accuracy measures how well the attack distinguishes members from non-members on shadow model data it never trained on. A typical result of 50-51% accuracy (essentially random guessing) reflects the weak overfitting signal in shadow models . This low accuracy does not indicate failure: the attack model has learned subtle patterns that will prove more effective on the target model, which overfits more strongly.

```
attack_test_acc, attack_test_predictions, attack_test_probs =
evaluate_model(attack_model, attack_test_loader, DEVICE)

print(f"\nAttack Model Test Performance:")
print(f"  Accuracy: {attack_test_acc:.4f}")
print(f"  Samples: {len(attack_test_predictions)}")

print("\nDetailed Classification Report:")
print(classification_report(
    y_attack_test,
    attack_test_predictions,
    target_names=['Non-Member', 'Member'],
    digits=4
))

# Save the attack model
attack_model_path = os.path.join(MODEL_DIR, "attack_model.pt")
torch.save(attack_model.state_dict(), attack_model_path)
print(f"\nAttack model saved to {attack_model_path}")
```

Detailed Performance Metrics

We examine performance for each class through three complementary metrics. With precision , we ask: of all samples we predicted as members, what fraction actually were members? If precision is 0.70, then 70% of our positive predictions were correct, while 30% were false positives (non-members incorrectly labeled as members). With recall , we ask the inverse: of all actual members, what fraction did we correctly identify? If recall is 0.85, we detected 85% of true members, while 15% escaped detection as false negatives. The F1 score gives us the harmonic mean of precision and recall, providing a single metric that penalizes extreme imbalances between the two.

Different applications prioritize different metrics based on their goals. Privacy audits emphasize recall to find all members even at the cost of false accusations, while legal contexts prioritize precision to avoid wrongly labeling someone as a training member. The relative values of precision and recall reveal the attack's bias: higher recall with lower precision means the attack aggressively labels samples as members, catching most true members but also generating false positives.

The saved file is small (about 20KB) because the `attack_model` has only approximately 2,600 parameters. This compact size reflects the simplicity of the membership classification task compared to the original income prediction task. Unlike the target model save, we don't create a dictionary with multiple components. A single `state_dict()` suffices because the `attack_model` needs no scaler or auxiliary data for inference.

Understanding What the Attack Learned

The `attack_model` essentially learned a confidence threshold with class-specific adjustments. We can examine its decision boundary by looking at predictions across the confidence range using `analyze_attack_decision_boundary()`.

```
boundary_analysis = analyze_attack_decision_boundary(attack_model, DEVICE)

print("\nDecision Boundary Analysis:")
for cls, data in boundary_analysis.items():
    threshold_idx = np.argmax(np.abs(data['membership_probs'] - 0.5))
    threshold_conf = data['confidences'][threshold_idx]
    print(f" Class {cls}: Membership threshold at confidence ~
{threshold_conf:.3f}")
```

Running this analysis reveals the confidence threshold above which our attack predicts membership. Typical thresholds are around 0.80-0.85: samples with higher confidence are classified as members, lower confidence as non-members. Notice how each class may have different thresholds, reflecting different overfitting patterns for each output class.

Threshold Selection and Attack Tuning

The `attack_model` outputs a membership probability between 0 and 1. Converting this to a binary decision requires choosing a threshold. Our implementation uses 0.5 by default: samples with membership probability above 0.5 are classified as members, below 0.5 as non-members. But this default is not always optimal.

Threshold choice depends on the relative costs of false positives versus false negatives. For privacy auditing, we want to catch all members even if we accidentally flag some non-members; a lower threshold like 0.3 increases recall at the cost of precision. For legal proceedings where false accusations are costly, a higher threshold like 0.7 increases

precision at the cost of recall. The ROC curve we generate later shows attack performance across all possible thresholds, letting us choose the operating point that matches our goals.

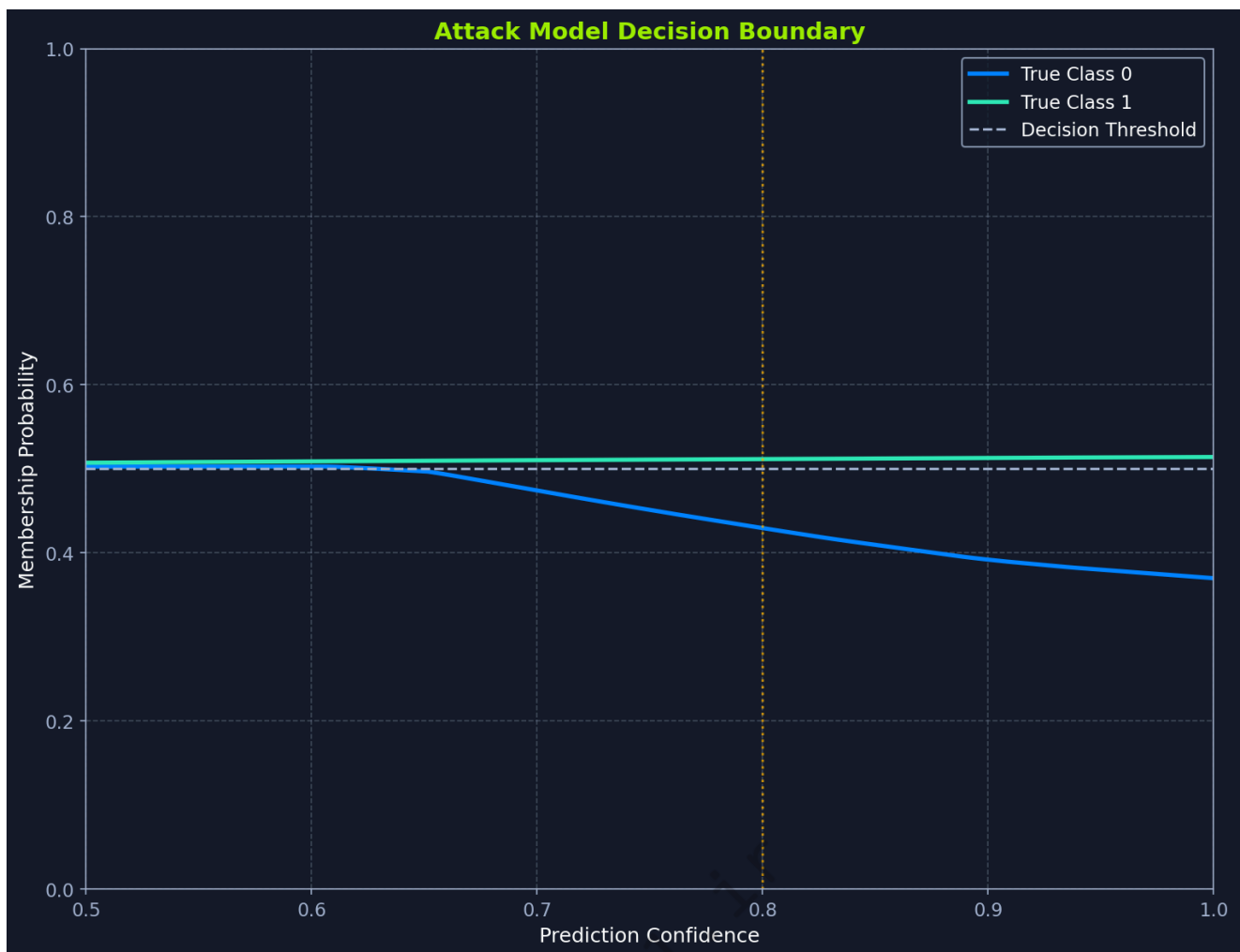
Finding the best threshold can use different approaches. One uses the validation set: try thresholds from 0.1 to 0.9 in steps of 0.05, compute F1 score for each, and select the threshold with highest F1. Another considers class balance: with 2:1 member to non-member ratio in our evaluation set, the 0.5 threshold tends to favor predicting membership because most samples are members. Adjusting the threshold to 0.6 or 0.7 can improve balanced accuracy.

The `attack_model` we train implicitly learns a threshold through its final layer bias. During training on balanced shadow data, it learns decision boundaries appropriate for 50/50 class balance. When we evaluate on imbalanced data (more members than non-members), the learned boundaries may not be optimal. This is why precision and recall can diverge significantly from each other.

Visualizing the Decision Boundary

To see how membership probability varies with prediction confidence for each class, we use `plot_decision_boundary()`.

```
plot_decision_boundary(  
    boundary_analysis,  
    save_path=os.path.join(FIGS_DIR, f"{FIG_PREFIX}decision_boundary.png")  
)
```



We see interesting class-specific patterns in this visualization. For True Class 1 samples (green), membership probability stays nearly flat around 0.5 across all confidence levels, indicating the attack cannot distinguish members from non-members in this class. For True Class 0 samples (blue), membership probability actually decreases with confidence, from 0.5 at low confidence to 0.38 at high confidence. This counterintuitive pattern suggests the attack learned that high-confidence Class 0 predictions are more likely non-members. The vertical dashed line marks the 0.8 confidence threshold where both classes cross.

Performance Expectations

Our shadow model evaluation reveals the challenge of membership inference. With shadow test accuracy around 50-51%, the attack appears to have learned almost nothing. This is expected: shadow models used dropout regularization (0.3) and early stopping (patience of 10), which reduced their overfitting to gaps of only 1-2% between training and holdout accuracy. Member and non-member predictions are nearly indistinguishable.

Our target model is different. With zero dropout and extended training (100 epochs without early stopping), it overfits significantly more, exhibiting a 7% gap between training accuracy (90%) and test accuracy (83%). This stronger overfitting creates a larger confidence gap between members and non-members. When we apply the attack to the target, expect accuracy to improve dramatically to 65-66%, yielding a 15-16% advantage over random guessing.

As a rough guide when evaluating the final attack on the target: accuracy above 60% indicates meaningful membership leakage; 55-60% suggests moderate vulnerability; and below 55% means the attack is largely ineffective. The shadow evaluation deliberately underestimates target performance because shadow models are designed to overfit less than the vulnerable target.

Executing and Evaluating the Attack

We have trained our attack classifier on shadow model predictions, though it achieved only marginal accuracy (around 50%) on that data due to weak membership signals. Now comes the real test: applying this attack to the actual target model, which exhibits stronger overfitting. This chapter executes the attack, evaluates its effectiveness with multiple metrics, and interprets what the results mean for privacy.

We evaluate our attack using genuinely held-out data: the target model's training set (members) and the attack evaluation set (non-members the target never saw). This mirrors the scenario a real attacker would face: querying an unknown model to determine membership.

Executing the Membership Inference Attack

We begin by collecting the target model's predictions on its training data (members) and on data it never trained on (non-members).

```
print("\n" + "=" * 60)
print("Executing Membership Inference Attack")
print("=" * 60)

preds_members = get_model_predictions(target_model, X_target_norm, DEVICE)
preds_non_members = get_model_predictions(target_model, X_attack_eval_norm,
DEVICe)

print(f"\nTarget model predictions collected:")
print(f"  Members: {len(preds_members)} samples")
print(f"  Non-members: {len(preds_non_members)} samples")
```

We collect softmax probability vectors for all 24,421 target training samples (members) and all 12,210 attack evaluation samples (non-members). These represent the complete population for our attack evaluation.

Preparing Attack Input

We cannot concatenate member and non-member predictions before calling `prepare_attack_data()` because the function assigns membership labels based on position: the first argument gets labeled 1 (member), the second gets labeled 0 (non-

member). We must call it separately for each population, passing empty arrays as placeholders for the missing counterpart.

We construct the empty array `np.zeros((0, preds_members.shape[1]))` with the correct column dimension (2 for binary classification) but zero rows, satisfying the function's input requirements without contributing any samples. Each call produces features of shape `(num_samples, 4)` with the appropriate membership label, which we'll then concatenate for evaluation.

```
attack_X_members, attack_y_members = prepare_attack_data(
    preds_members, np.zeros((0, preds_members.shape[1])),
    y_target, np.array([], dtype=np.int64)
)

attack_X_non_members, attack_y_non_members = prepare_attack_data(
    np.zeros((0, preds_non_members.shape[1])), preds_non_members,
    np.array([], dtype=np.int64), y_attack_eval
)

print(f"\nAttack input prepared:")
print(f"  Member features: {attack_X_members.shape}")
print(f"  Non-member features: {attack_X_non_members.shape}")
```

Combining Attack Data

With features prepared for both populations, we merge them into single arrays for evaluation. Note the class imbalance: we have twice as many members (24,421) as non-members (12,210), which affects metric interpretation.

```
attack_X_eval = np.concatenate([attack_X_members, attack_X_non_members],
                                axis=0)
attack_y_eval = np.concatenate([attack_y_members, attack_y_non_members],
                                axis=0)

print(f"\nTotal attack evaluation samples: {len(attack_X_eval)}")
print(f"  Members: {np.sum(attack_y_eval == 1)}")
print(f"  Non-members: {np.sum(attack_y_eval == 0)}")
```

Running the Attack

We feed the combined attack features through our trained `attack_model` to obtain membership predictions and extract membership probabilities for ROC analysis.

```

attack_eval_loader = create_dataloader(attack_X_eval, attack_y_eval,
ATTACK_MODEL_CONFIG['batch_size'], shuffle=False)

_, attack_predictions, attack_probs = evaluate_model(attack_model,
attack_eval_loader, DEVICE)

membership_probs = attack_probs[:, 1]

print(f"\nAttack predictions generated")
print(f" Mean membership probability: {membership_probs.mean():.4f}")

```

We set `shuffle=False` to maintain alignment with ground truth labels. The `[:, 1]` indexing extracts membership probability (class 1) from the probability array, giving confidence scores from 0.0 (definitely non-member) to 1.0 (definitely member).

Computing Attack Metrics

Accuracy alone does not tell the full story. Different applications care about different aspects: privacy audits want high recall (find all members), while legal contexts want high precision (avoid false accusations). We compute accuracy, precision, recall, and F1 score to provide a complete picture.

These metrics use scikit-learn's scoring functions with the default 0.5 threshold on membership probabilities.

```

attack_accuracy = accuracy_score(attack_y_eval, attack_predictions)
attack_precision = precision_score(attack_y_eval, attack_predictions)
attack_recall = recall_score(attack_y_eval, attack_predictions)
attack_f1 = f1_score(attack_y_eval, attack_predictions)

print(f"\nMembership Inference Attack Results:")
print(f" Attack Accuracy: {attack_accuracy:.4f}")
print(f" Attack Precision: {attack_precision:.4f}")
print(f" Attack Recall: {attack_recall:.4f}")
print(f" Attack F1 Score: {attack_f1:.4f}")

```

Let's interpret each metric. With `accuracy` (around 69% typical), we measure overall correct predictions. With 36,632 samples, 69% accuracy means correctly classifying about 25,276 samples. With `precision` (around 69% typical), we see what fraction of predicted members actually were members. Given the class imbalance, precision tends to be higher because most samples are members. With `recall` (around 97% typical), we measure what fraction of actual members we identified, the probability of detecting a randomly chosen member. The `F1 score` balances precision and recall, providing a single metric for comparison.

Attack Advantage

The most interpretable metric is the advantage over random guessing: simply subtract 0.5 from the attack accuracy. An advantage of around 0.19 (typical for this setup) means the attack performs about 19% better than random guessing, correctly identifying membership for about 69% of queries instead of 50%.

As a rough guide: advantages above 0.15 indicate high vulnerability where the model significantly leaks membership information; advantages between 0.05 and 0.15 suggest moderate vulnerability with detectable leakage; and advantages below 0.05 mean the attack performs near random. For privacy-sensitive applications, even a 5% advantage is concerning because it enables systematic exploitation at scale.

Storing Results

We collect all metrics and intermediate data for visualization and analysis.

```
results = {
    'attack_accuracy': attack_accuracy,
    'attack_precision': attack_precision,
    'attack_recall': attack_recall,
    'attack_f1': attack_f1,
    'attack_y_true': attack_y_eval,
    'attack_y_pred': attack_predictions,
    'attack_probs': membership_probs,
    'confidence_members': np.max(preds_members, axis=1),
    'confidence_non_members': np.max(preds_non_members, axis=1),
}

print("\nResults stored for visualization")
```

We store both scalar metrics and arrays for detailed analysis. The confidence arrays contain maximum prediction values for each sample, which we'll visualize to show the overfitting signal the attack exploited.

Generating Visualizations

Visualizations help us understand why the attack succeeded and communicate results effectively. We'll create several plots that reveal different aspects of the attack.

ROC Curve

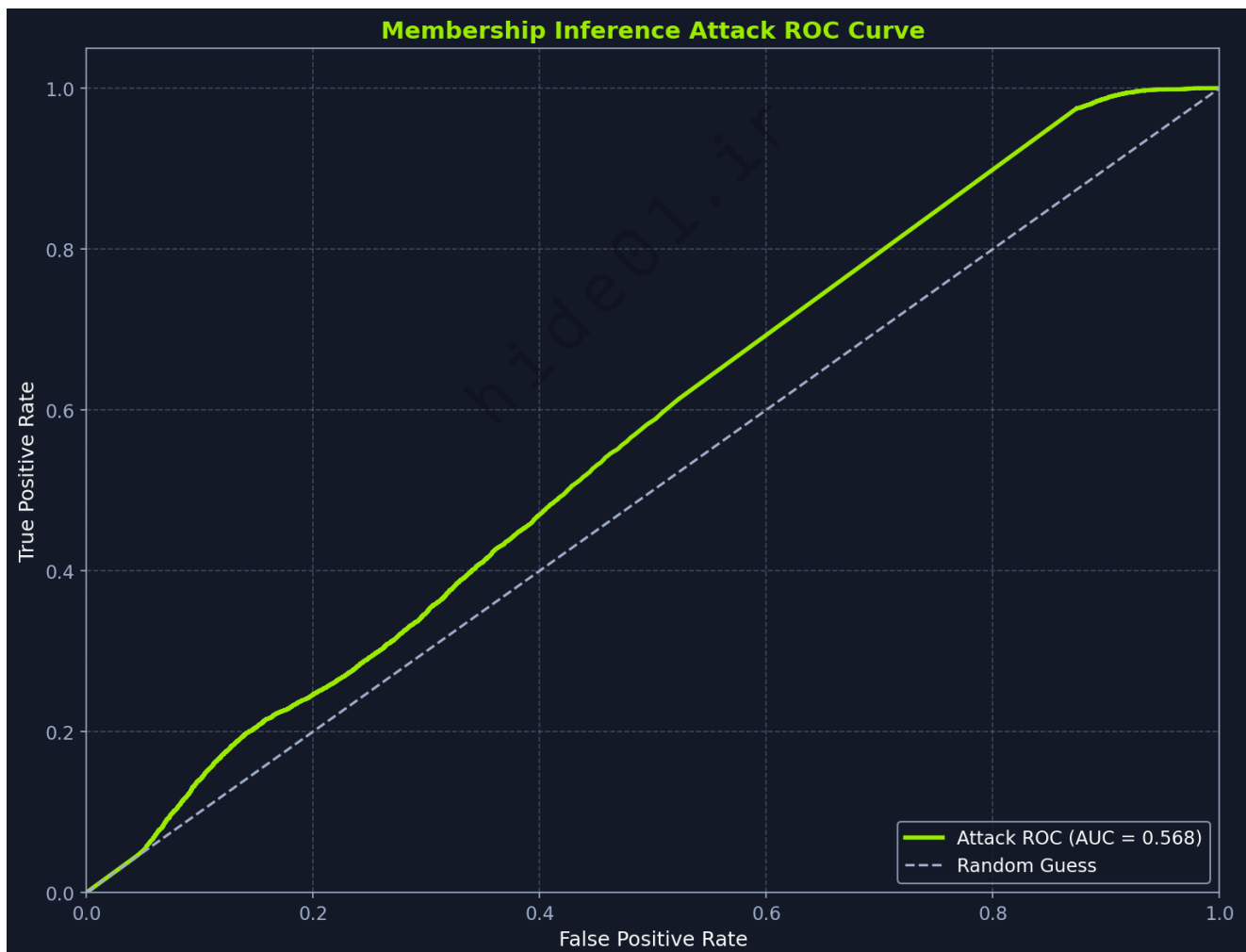
We visualize attack performance across all decision thresholds using the ROC curve. The area under this curve (AUC) summarizes overall attack quality: 0.5 means random guessing,

1.0 means perfect discrimination. We use `plot_attack_roc_curve()` to generate this visualization.

```
print("\n" + "=" * 60)
print("Generating Visualizations")
print("=" * 60)

auc_score = plot_attack_roc_curve(
    results['attack_y_true'],
    results['attack_probs'],
    save_path=os.path.join(FIGS_DIR, f"{FIG_PREFIX}attack_roc.png")
)
results['attack_auc'] = auc_score

print(f"Attack AUC: {auc_score:.4f}")
```

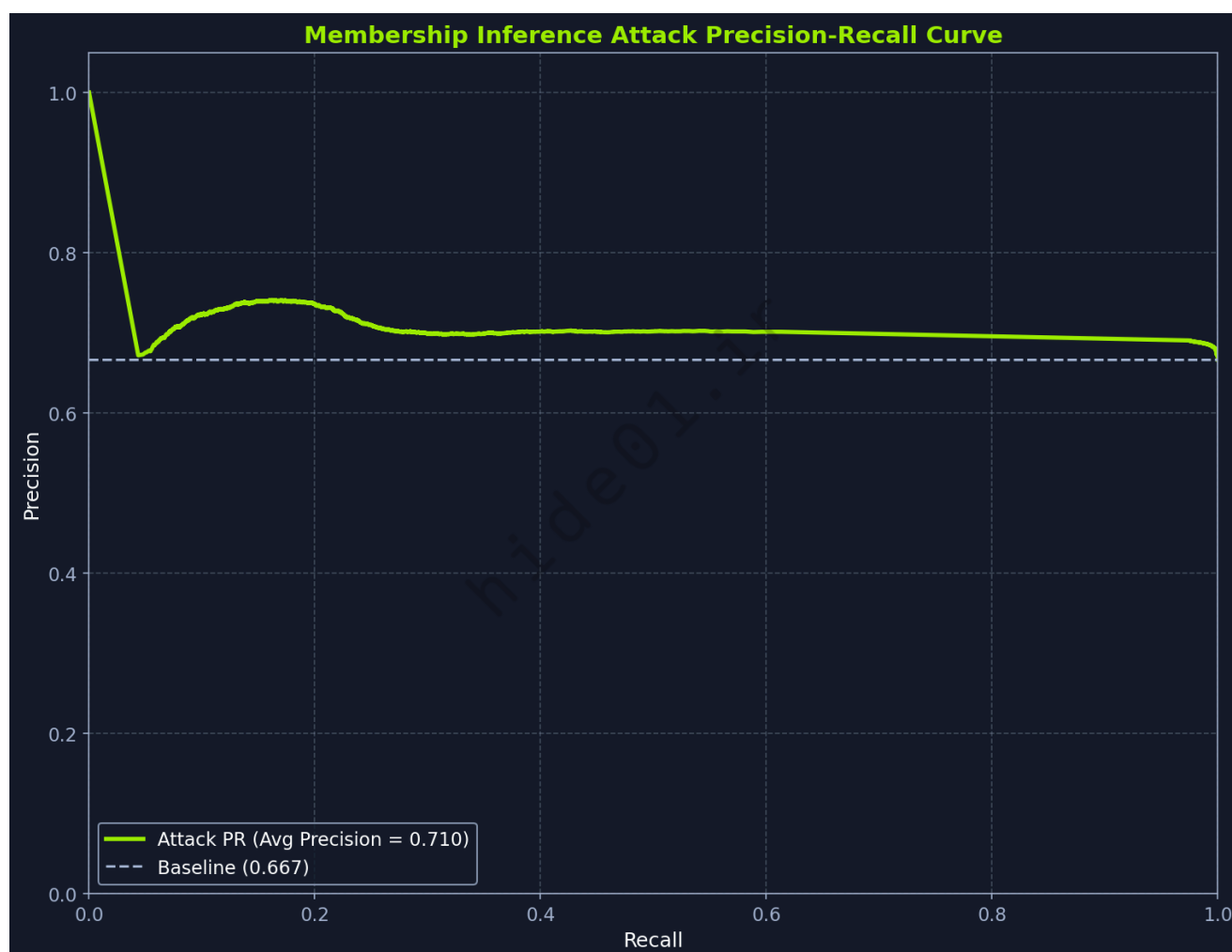


Looking at the ROC curve, we see AUC of 0.568, just slightly above the 0.5 random baseline (dashed diagonal). While this seems modest, remember that the `attack_model` was trained on `shadow_model` data with nearly identical member/non-member distributions. Notice how the curve stays above the diagonal throughout, confirming the attack performs better than random guessing across all threshold choices.

PR Curve

The precision-recall curve is particularly informative for our imbalanced dataset (2:1 member to non-member ratio). Unlike ROC where the baseline is 0.5, the PR baseline is the fraction of positive samples (0.667).

```
plot_precision_recall_curve(
    results['attack_y_true'],
    results['attack_probs'],
    save_path=os.path.join(FIGS_DIR, f"{FIG_PREFIX}attack_pr.png")
)
```



Our PR curve achieves average precision of 0.710, above the 0.667 baseline. Notice how precision stays above 0.7 for most recall values, meaning 70% of samples we predict as members actually are members. This modest but consistent improvement over the baseline demonstrates the attack learns real membership signals.

Confidence Distribution

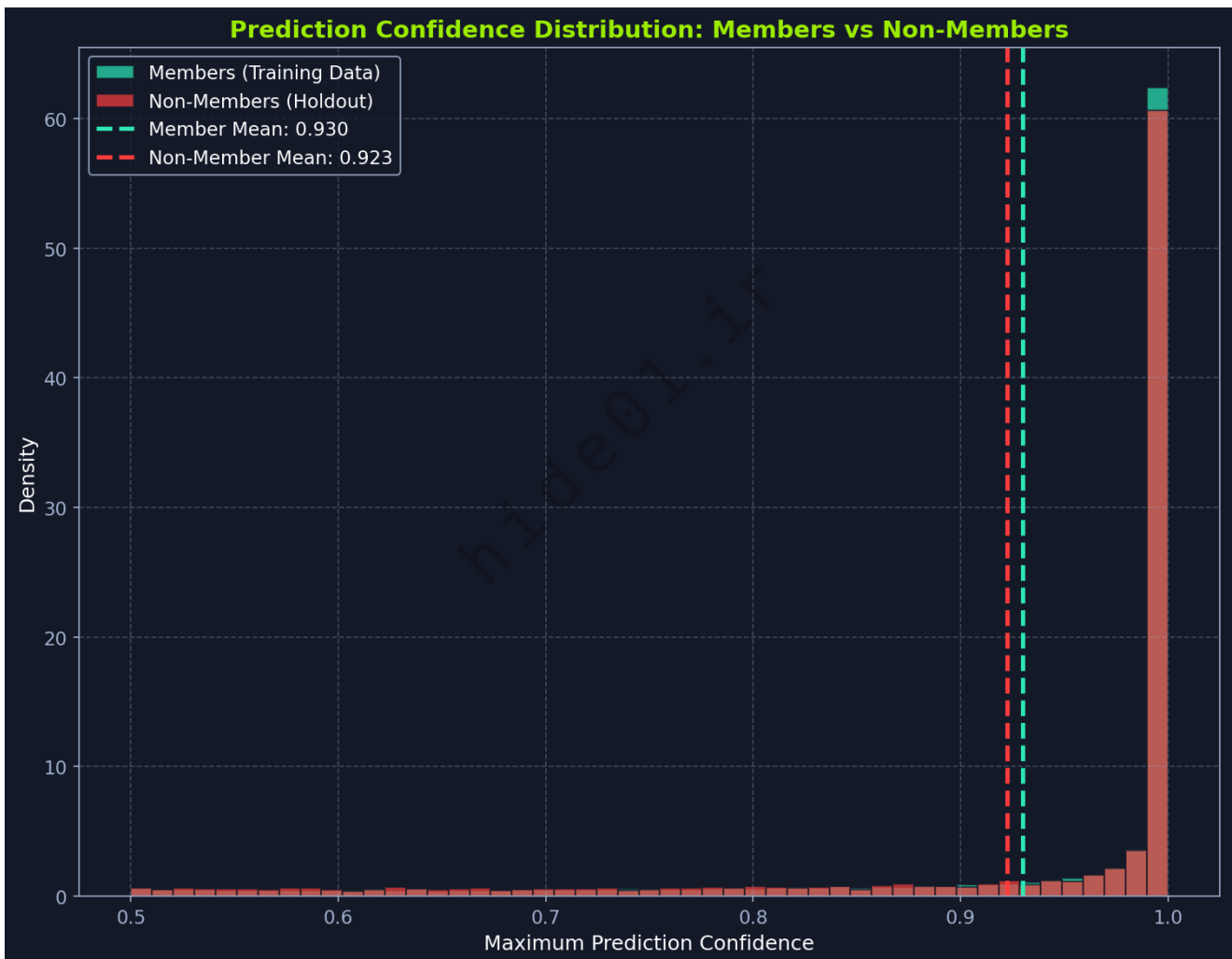
The confidence distribution visualization reveals the underlying signal our attack exploited: the difference in prediction confidence between members and non-members.

```

plot_confidence_distributions(
    results['confidence_members'],
    results['confidence_non_members'],
    save_path=os.path.join(FIGS_DIR, f"
{FIG_PREFIX}confidence_distributions.png")
)

print(f"\nMean confidence - Members:
{np.mean(results['confidence_members']):.4f}")
print(f"Mean confidence - Non-Members:
{np.mean(results['confidence_non_members']):.4f}")

```



Here we see the core signal our attack exploits: a small but consistent gap where members have mean confidence 0.9301 while non-members have 0.9226. This 0.75% difference, while subtle, persists across tens of thousands of samples. Both distributions cluster heavily at high confidence (0.95+), explaining why the attack must rely on statistical patterns instead of easily separating the populations. The substantial overlap in the middle range causes the attack errors.

Attack Metrics

Finally, a bar chart summarizes all attack metrics in one view, making it easy to compare performance across measures and against the 0.5 random baseline.

```
plot_attack_accuracy_comparison(  
    results,  
    save_path=os.path.join(FIGS_DIR, f"{FIG_PREFIX}attack_metrics.png")  
)
```



Our attack achieves 69.15% accuracy (19.15% advantage over random), 68.99% precision, and notably high 97.58% recall. The high recall indicates the attack aggressively predicts membership, catching most true members but also generating false positives among non-members. Notice that AUC at 0.5675 appears lower because it measures discrimination across all thresholds, while the other metrics use a fixed 0.5 threshold that happens to work well for this class-imbalanced dataset.

Saving Results

We save all results to a JSON file for future reference and comparison. The output dictionary has three sections: target model performance, attack results, and configuration.

```
output = {  
    'target_model': {
```

```

        'train_accuracy': float(train_acc),
        'test_accuracy': float(test_acc),
        'overfitting_gap': float(train_acc - test_acc),
    },
    'attack_results': {
        'accuracy': float(results['attack_accuracy']),
        'precision': float(results['attack_precision']),
        'recall': float(results['attack_recall']),
        'f1_score': float(results['attack_f1']),
        'auc': float(results['attack_auc']),
        'advantage': float(results['attack_accuracy'] - 0.5),
    },
    'configuration': {
        'random_seed': RANDOM_SEED,
        'num_shadow_models': SHADOW_MODEL_CONFIG['num_shadow_models'],
        'target_architecture': TARGET_MODEL_CONFIG['hidden_layers'],
        'attack_architecture': ATTACK_MODEL_CONFIG['hidden_layers'],
    }
}

```

We wrap NumPy floats with `float()` to ensure JSON compatibility. Now write the dictionary to disk:

```

results_path = os.path.join(FIGS_DIR, f"{FIG_PREFIX}attack_results.json")
with open(results_path, 'w') as f:
    json.dump(output, f, indent=2)

print(f"\nResults saved to {results_path}")

```

The three sections capture target model performance (showing the overfitting gap), attack results (all computed metrics), and configuration (for reproducibility).

Interpreting the Results

Running the complete pipeline produces output like this:

```

Target Model Performance:
  Training Accuracy: 0.9360
  Test Accuracy:    0.8247
  Overfitting Gap:  0.1113

Membership Inference Attack Results:
  Attack Accuracy:  0.6915

```

```
Attack Precision: 0.6899
Attack Recall:    0.9758
Attack F1 Score:  0.8083
Attack AUC:       0.5675
```

Confidence Analysis:

```
Mean confidence (members):    0.9301
Mean confidence (non-members): 0.9226
Confidence gap:               0.0075
```

What do these numbers mean in practice? Consider a healthcare organization whose model was trained on 24,000 patient records. An attacker with API access can now determine, with 69% accuracy, whether any specific individual's data was used for training. Given 1,000 queries, random guessing yields 500 correct answers. Our attack gets 692, which translates to 192 additional patients whose presence in the training set is revealed. For sensitive domains (medical records, financial data, criminal justice), this systematic leakage enables targeted identification at scale.

Notice the relationship between metrics. High recall (97.58%) with moderate precision (68.99%) means the attack aggressively labels samples as members. It catches nearly all true members but includes false positives. This bias emerges from the 2:1 class imbalance and the 0.5 decision threshold. A privacy auditor might prefer this configuration (find all potential leaks), while a legal context might demand higher precision (avoid false accusations).

Why does AUC (0.5675) appear so much lower than accuracy (69.15%)? AUC measures discrimination across all possible thresholds, while accuracy uses a fixed 0.5 cutoff. Our class-imbalanced dataset happens to work well with that default threshold. At other thresholds, performance degrades, which the modest AUC reflects. This gap between metrics is characteristic of membership inference attacks on imbalanced data.

Differential Privacy Fundamentals

We've seen how membership inference attacks exploit `overfitting` to recover information about individual training samples. Those attacks rely on a simple but powerful signal: models tend to be more confident on points they have seen during training than on points they have only seen at test time. Instead of defending at the model output level, we can attack the root cause during training itself.

Instead of redesigning the model architecture or hiding internal logits, DP-SGD modifies the training procedure itself. Think of it as controlled amnesia. During optimization we clip every `per-sample gradient` to a fixed L_2 norm bound and then add carefully calibrated random noise before each parameter update. Over many training steps, those noisy, clipped

gradients ensure that the model's final parameters do not depend too strongly on any single training example.

We use the `CIFAR-10` image classification dataset as our running example. `CIFAR-10` contains fifty thousand training images and ten thousand test images spread across ten classes such as `airplane`, `cat`, and `ship`. We first train a non-private convolutional neural network on this dataset, evaluate how vulnerable it is to a confidence-based membership inference attack, and then retrain the same architecture using DP-SGD with different privacy budgets.

Differential Privacy: Formal Definition

A randomized algorithm M satisfies (ϵ, δ) -differential privacy if for any two datasets D and D' that differ in exactly one record, and for any subset of outputs S :

$$P[M(D) \in S] \leq e^\epsilon \cdot P[M(D') \in S] + \delta$$

This inequality bounds how much adding or removing a single person's data can change the algorithm's output distribution. The term e^ϵ acts as a multiplicative bound: if $\epsilon = 1$, outputs can be at most $e \approx 2.7$ times more likely with your data than without. The δ term allows a small probability of complete failure, where the guarantee does not hold at all.

Why does this definition matter for machine learning? Consider an attacker who observes a trained model and tries to determine whether Alice's record was in the training set. Without DP, the model might behave very differently with and without Alice's data, revealing her membership with high confidence. With DP, the model's behavior is nearly identical regardless of Alice's presence, limiting what any attacker can learn.

We control privacy strength through ϵ , where smaller values mean stronger privacy. At $\epsilon = 1$, the guarantee is very strong, providing near-complete indifference to any one person's data. At $\epsilon = 3$, privacy remains strong with individual influence heavily suppressed. At $\epsilon = 10$, privacy is modest and the model might reveal aggregate patterns.

The parameter δ captures the probability that the guarantee might fail, and it is usually set to a very small value relative to the dataset size. For `CIFAR-10`'s fifty thousand training examples, setting $\delta = 10^{-5}$ means the privacy guarantee holds with probability 0.99999.

How DP-SGD Works

In standard stochastic gradient descent, we sample a batch of examples, compute the gradient of the loss with respect to each parameter, average these gradients over the batch, and then step the parameters in the negative gradient direction. Baseline SGD has perfect information: every per-sample gradient flows directly into parameter updates, unclipped and uncorrupted. DP-SGD deliberately corrupts this process through three mechanisms:

`gradient clipping` bounds each sample's influence, `noise addition` obscures individual contributions, and `privacy composition` tracks cumulative budget across training steps.

Gradient Clipping and Sensitivity

The key to adding privacy-preserving noise is knowing how much any single sample can affect the computation. This quantity is called `sensitivity`. For a function f , sensitivity measures the maximum change in f 's output when we add or remove one record from the input dataset. If sensitivity is unbounded, we cannot calibrate noise appropriately.

In standard SGD, sensitivity is effectively infinite. A single outlier sample could produce an enormous gradient that dominates the entire batch update. DP-SGD solves this by clipping each per-sample gradient so that its L_2 norm never exceeds a fixed constant

`max_grad_norm`. If a sample produces a gradient with norm 5.2 and `max_grad_norm = 1.0`, DP-SGD scales that gradient down by a factor of 5.2, reducing its norm to exactly 1.0.

After clipping, the sensitivity of the gradient sum is exactly `max_grad_norm`. Adding or removing any single sample can change the sum by at most one clipped gradient, which has norm at most `max_grad_norm`. This bounded sensitivity is what makes noise calibration possible.

Clipping discards gradient information from samples with large gradients, which are often the most informative for learning. Aggressive clipping (small `max_grad_norm`) provides stronger privacy but loses more gradient signal, slowing convergence. Choosing the clipping threshold requires balancing privacy against learning efficiency.

Noise Addition and the Gaussian Mechanism

With sensitivity bounded, we can add calibrated noise using the `Gaussian mechanism`. For a function f with L_2 sensitivity Δf , adding Gaussian noise with standard deviation $\sigma = \Delta f \cdot \sqrt{2 \ln(1.25/\delta)}/\epsilon$ achieves (ϵ, δ) -differential privacy.

In DP-SGD, we add zero-mean Gaussian noise to the sum of clipped gradients before averaging. The noise standard deviation is $\sigma = \text{max_grad_norm} \times \text{noise_multiplier}$, where the noise multiplier depends on the target ϵ , δ , and number of training steps.

For $\epsilon = 10$ with batch size 256 and `max_grad_norm = 1.0` in our CIFAR-10 configuration, Opacus typically chooses a `noise_multiplier` around 1.2, meaning we add Gaussian noise with standard deviation 1.2 to each clipped gradient sum. For $\epsilon = 3$, the same setup usually requires a `noise_multiplier` around 3.8, injecting much more randomness and making learning harder. These values come from the `privacy accountant` for this specific dataset, batch size, and epoch count rather than from the closed-form Gaussian mechanism alone.

Privacy Accumulates Across Training

A single noisy gradient step provides strong privacy. But we take thousands of gradient steps during training, and each step reveals a little more about the training data. Privacy composes across steps: the total privacy loss grows with the number of updates.

Naive composition would add epsilon values: 1000 steps at $\epsilon = 0.01$ each would yield total $\epsilon = 10$. Advanced composition theorems provide tighter bounds, and Rényi Differential Privacy (RDP) accounting (used by Opacus) achieves even tighter tracking. We let Opacus handle this accounting automatically through its PrivacyEngine, which computes the noise multiplier needed to achieve a target ϵ after a specified number of epochs.

Why does more training require more noise per step? To maintain the same final privacy budget across more steps, each individual step must leak less. This means higher noise multiplier, which makes optimization harder. The privacy-utility tradeoff is not just about final epsilon but also about how that budget is spent across training.

Choosing Privacy Budgets

We'll train DP-SGD models at two epsilon values: $\epsilon = 10$ and $\epsilon = 3$. These represent points on the privacy-utility spectrum that demonstrate the tradeoff effectively.

$\epsilon = 10$ represents a modest privacy guarantee that preserves most model utility. In practice, many machine learning applications use epsilon values in the 1-10 range. Apple's differential privacy implementations use epsilon values around 2-8 for various features. Research benchmarks often use $\epsilon = 8$ or $\epsilon = 10$ as a "reasonable privacy" baseline. At this level, the model is provably more private than no protection, but the guarantee allows some membership inference success.

$\epsilon = 3$ represents stronger privacy with more noticeable utility impact. This level provides meaningful protection against membership inference while still producing a useful classifier. Academic research often targets $\epsilon = 1$ -3 for "strong privacy" demonstrations. The increased noise makes optimization harder, resulting in lower accuracy, but the privacy guarantee is substantially stronger.

Why not $\epsilon = 1$ (very strong privacy)? Achieving $\epsilon = 1$ on CIFAR-10 with reasonable accuracy requires careful architecture design, longer training with smaller learning rates, and often pre-training on public data. For a demonstration of the basic tradeoff, $\epsilon = 3$ shows the effect of stronger privacy without requiring advanced techniques.

Theoretical Guarantees vs Empirical Measurement

Why measure MIA empirically when differential privacy provides theoretical guarantees? The two perspectives are complementary and reveal different aspects of privacy.

Differential privacy provides a worst-case bound. The guarantee holds against any possible attack, including attacks we have not yet imagined. An (ϵ, δ) -DP mechanism bounds the success of the optimal adversary by limiting how much any single record can change the distribution of outputs. If $\epsilon = 1$, the likelihood of any particular outcome can increase by at most a factor of $e \approx 2.7$ when one record is added or removed. Mapping this likelihood bound to concrete membership advantage requires additional assumptions, so it is best viewed as

an upper bound on how much any attack can improve over random guessing, regardless of the adversary's computational power or auxiliary information.

Empirical MIA measures average-case vulnerability using a specific attack strategy. A confidence-threshold attack is simple and does not exploit all available information. A more sophisticated attack (like the shadow model approach from the previous section) might achieve higher accuracy. The empirical advantage we measure is a lower bound on true vulnerability, while the DP guarantee is an upper bound.

Why is empirical MIA often much lower than the theoretical bound? Several factors contribute. The theoretical bound assumes a worst-case sample that maximally affects the model, while most samples have modest influence. The bound also assumes an optimal adversary with unlimited computational resources, while our attack uses a simple threshold. Finally, the bound holds uniformly for all samples, while some samples are much more vulnerable than others.

For privacy auditing, both perspectives matter. Empirical MIA tells us how vulnerable the model is to known attacks today. The DP guarantee tells us how vulnerable it could be to any attack, including future ones. A model with low empirical MIA but no DP guarantee might be broken by a better attack tomorrow. A model with a DP guarantee provides protection regardless of attack improvements.

The Opacus Library

We use the Opacus library to implement DP-SGD. Opacus wraps your PyTorch optimizer and data loader with a PrivacyEngine that modifies gradient computation during each training step. Instead of computing the batch-average gradient, Opacus computes per-sample gradients and clips each to the configured max_grad_norm. The clipped gradients are then noised with Gaussian noise scaled to the desired privacy budget and tracked through a privacy accountant that monitors cumulative privacy expenditure.

To configure privacy, we call make_private_with_epsilon(), which automatically calculates the required noise multiplier for a target privacy budget. During training, get_epsilon() reports cumulative privacy expenditure at any point.

Per-Sample Gradients and Architecture Constraints

Standard PyTorch backpropagation computes the average gradient across all samples in a batch. DP-SGD requires something different: per-sample gradients, where each sample's gradient contribution is computed separately before clipping and noise addition. This is computationally more expensive because we cannot simply average gradients during the backward pass.

Opacus computes per-sample gradients using gradient hooks that intercept and transform the backward computation. For a batch of 256 samples, instead of computing one gradient tensor per parameter, we get 256 gradient tensors per parameter. Each gets clipped

to `max_grad_norm`, then summed, noised, and averaged. This increases memory usage proportionally to batch size and slows training by roughly 2-5x compared to standard SGD.

Not all PyTorch layers support per-sample gradient computation. `BatchNorm` is incompatible because it computes statistics across the batch dimension, coupling sample gradients together in ways that prevent individual clipping. The gradient for one sample depends on all other samples in the batch, violating the independence assumption DP-SGD requires.

Compatible normalization layers include `GroupNorm` and `LayerNorm`, which normalize within each sample independently. `InstanceNorm` also works because it operates on individual samples. When adapting an existing architecture for DP-SGD, replace `BatchNorm` with `GroupNorm` (using groups equal to the number of channels for channel-wise normalization similar to `BatchNorm`'s effect).

Other incompatible operations include any layer that shares state across samples within a batch. Custom layers that accumulate batch statistics, certain attention mechanisms that mix samples before the final output, and operations that depend on batch size all require modification or replacement. Opacus provides `ModuleValidator.fix()` to attempt automatic fixes for common incompatibilities, but complex architectures may require manual adjustment.

Privacy Amplification by Subsampling

Rather than fixed-size batches, we use Poisson subsampling through Opacus. Each training example is included in a batch independently with probability $q = \text{batch_size} / \text{dataset_size}$. This randomness provides privacy amplification: because the attacker does not know which samples were included in any given batch, the effective privacy cost per step is reduced.

How much amplification do we get? It depends on the sampling rate q . For our CIFAR-10 setup with batch size 256 and 50,000 training samples, $q = 256 / 50000 = 0.00512$. This low sampling rate significantly amplifies privacy, allowing us to achieve reasonable epsilon values even over many training steps.

Larger batch sizes mean higher sampling rates and less amplification, which might seem counterintuitive. However, larger batches also mean fewer gradient steps per epoch, and the net effect depends on the specific privacy accountant calculations. In practice, moderate batch sizes (128-512) often work well for DP-SGD.

What does Poisson subsampling look like in practice? With standard batching, we shuffle the dataset once and iterate through fixed-size chunks. With Poisson subsampling, each sample has an independent 0.512% chance of appearing in any given batch. This means batch sizes vary slightly (averaging 256 but sometimes 240, sometimes 270), and the same sample might appear in multiple batches per epoch or skip an epoch entirely. Opacus handles this automatically when you call `make_private_with_epsilon()`.

Tuning DP-SGD Hyperparameters

Understanding how each hyperparameter affects the privacy-utility tradeoff helps when adapting to other datasets or stricter privacy requirements.

The `clipping threshold` determines how much gradient information survives each step. Too aggressive (say, 0.1) and most gradients get truncated, starving the model of learning signal. Too permissive (say, 10.0) and we waste privacy budget adding noise scaled to rarely-used headroom. A practical calibration approach: run a few epochs without privacy, compute the 75th percentile of gradient norms, and use that value. For typical CNN architectures on `CIFAR-10`, gradient norms fall between 0.5 and 5.0, making `max_grad_norm = 1.0` a reasonable middle ground.

Batch size affects privacy through subsampling amplification. Smaller batches mean lower sampling rates per step, strengthening privacy amplification, but they also require more gradient steps per epoch (each consuming privacy budget). Larger batches weaken amplification but reduce total steps. Going below 64 often hurts convergence because gradient variance becomes too high.

Learning rate for DP-SGD typically matches or slightly undercuts the baseline rate. Since added noise already reduces effective gradient signal, further reduction may not help. If training diverges, lowering the learning rate can stabilize optimization, but start by matching baseline settings. Some practitioners find DP-SGD works better without momentum, though our experiments use momentum successfully.

The number of epochs creates an interesting tradeoff. More epochs allow longer learning but consume more privacy budget, requiring higher noise per step to maintain a target final epsilon. If convergence happens quickly, fewer epochs with proportionally less noise per step often yields better utility at the same privacy level.

Finally, the failure probability `DELTA` should satisfy $\delta < 1/n$ where n is the training set size. `CIFAR-10` has 50,000 training samples, so $\delta = 10^{-5} = 1/100000$ comfortably satisfies $\delta < 1/50000$. Violating this constraint weakens the privacy guarantee because the failure probability would exceed the probability of any single individual appearing in the dataset.

Setup Overview

With the theoretical foundation established, we now move to implementation. Before training DP-SGD models, we need to configure our environment and understand the library components that handle privacy accounting behind the scenes. We use the `htb_ai_library` package for model training, evaluation, and visualization, which lets us focus on the privacy mechanisms rather than boilerplate code.

Installing Dependencies

Install the HTB AI library from GitHub:

```
pip install --upgrade git+https://github.com/PandaSt0rm/htb-ai-library
```

Imports

Start your training script with these imports:

```
import os
import json
import torch
import torch.optim as optim
from safetensors.torch import save_file

from htb_ai_library import (
    set_reproducibility, use_htb_style,
    CIFAR10CNN,
    get_cifar10_loaders,
    train_baseline_sgd,
    train_dp_sgd,
    evaluate_accuracy,
    compute_mia_advantage,
    plot_accuracy_comparison,
    plot_privacy_utility_tradeoff,
)
```

Library Components

The `htb_ai_library` handles core functionality for training and evaluating DP-SGD models. Here's what each component provides.

Reproducibility matters for meaningful comparisons. `set_reproducibility(seed)` ensures comparable results when measuring the privacy-utility tradeoff across different epsilon values.

Our classifier is `CIFAR10CNN`, a convolutional neural network designed for CIFAR-10 image classification. It uses standard `Conv2d`, `ReLU`, `MaxPool2d`, and `Linear` layers, all compatible with Opacus per-sample gradient computation. `BatchNorm` layers are intentionally excluded because they would break DP-SGD's privacy guarantees (see the architecture constraints discussion in the previous section).

To load our data, we call `get_cifar10_loaders(batch_size, download)`, which fetches the CIFAR-10 dataset (60,000 32x32 color images across 10 classes), applies standard normalization transforms, and returns a tuple of `(train_dataset, test_dataset, train_loader, test_loader)`. Each DP model needs its own loader instance because Opacus wraps loaders to track sample access for privacy accounting. Sharing loaders

between models would corrupt privacy budget calculations, so we call `get_cifar10_loaders()` separately for each epsilon configuration.

Training happens through two functions. `train_baseline_sgd(model, train_loader, device, epochs, learning_rate, momentum)` runs standard SGD training over fixed epochs, returning the trained model. `train_dp_sgd(model, train_loader, privacy_engine, optimizer, device, epochs, delta)` trains with DP-SGD using an attached Opacus privacy engine. The `privacy_engine` and `optimizer` must both reference the same model since Opacus coordinates them internally to ensure gradient clipping and noise addition happen correctly. This function returns `(model, final_epsilon)`.

To evaluate our models, we use two metrics. We call `evaluate_accuracy(model, loader, device)` to compute classification accuracy on a dataset (returning a percentage from 0-100). To measure membership inference vulnerability, we use `compute_mia_advantage(model, train_loader, test_loader, device)`, which performs a confidence-threshold attack and returns `(attack_accuracy, advantage)` where advantage is accuracy minus 0.5.

To visualize our results, we use two functions. We call `plot_accuracy_comparison()` to generate a grouped bar chart comparing test accuracy across baseline, $\epsilon = 10$, and $\epsilon = 3$ models. For a comprehensive view of the tradeoff, we use `plot_privacy_utility_tradeoff()` to create a dual-axis plot showing accuracy and MIA advantage across privacy levels.

Configuration Parameters

```
RANDOM_SEED = 1337
BATCH_SIZE = 256
BASELINE_EPOCHS = 20
BASELINE_LR = 0.1
DP_EPOCHS = 20
DP_LR = 0.1
MAX_GRAD_NORM = 1.0
DELTA = 1e-5

set_reproducibility(RANDOM_SEED)
use_htb_style()
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
```

```
os.makedirs("figs", exist_ok=True)
os.makedirs("output", exist_ok=True)
os.makedirs("models", exist_ok=True)
```

We keep training conditions parallel between baseline and DP-SGD (same epochs, batch size, learning rate) so that differences in final accuracy reflect the privacy mechanism itself rather than training configuration. `RANDOM_SEED = 1337` ensures reproducibility, enabling meaningful comparisons between runs.

The DP-specific parameters are `MAX_GRAD_NORM = 1.0`, which bounds each sample's gradient contribution before noise addition, and `DELTA = 1e-5`, which represents the privacy failure probability in our (ϵ, δ) -differential privacy guarantee. For guidance on choosing these values, see the hyperparameter tuning discussion in the Overview.

Three directories organize our outputs: `OUTPUT_DIR` stores intermediate checkpoints and results, `FIGS_DIR` holds visualization figures, and `MODELS_DIR` stores the final safetensors model for validation server submission.

Simplified "MIA" Measurement

The previous section used the full shadow model attack methodology. Here we use a simplified confidence-threshold approach as a measurement tool. Using `compute_mia_advantage()`, we collect maximum softmax confidence for all training samples (members) and all test samples (non-members), then balance these samples (10,000 from each) to avoid class imbalance. Why 10,000? `CIFAR-10` has exactly 10,000 test samples, so we match that count from the 50,000 training samples for fair comparison. With unbalanced datasets, this approach would need adjustment (either subsampling the larger set or using stratified sampling).

It searches for the optimal threshold that maximizes attack accuracy, returning `(attack_accuracy, advantage)` where advantage is accuracy minus 0.5. This captures the same underlying signal (overfitted models are more confident on training data) without the overhead of training shadow models.

Training Models and Measuring Privacy

Before applying DP-SGD, we need to establish a baseline and measure how much membership information the unprotected model leaks. This section trains the non-private baseline, measures its vulnerability, and then trains DP-SGD protected models at different privacy budgets.

Setting Up the Script

With the imports and helper functions from the Setup Overview in place, begin the demonstration:

```
print("=" * 80)
print("  DP-SGD PRIVACY MITIGATION DEMONSTRATION")
print("=" * 80)
print(f"\nDevice: {device}")
print(f"Random seed: {RANDOM_SEED}")
```

Load the CIFAR-10 dataset. The first run downloads the data; subsequent runs use the cached version.

```
print("\nLoading CIFAR-10 dataset...")
train_dataset, test_dataset, train_loader, test_loader =
get_cifar10_loaders(batch_size=BATCH_SIZE, download=True)

print(f"Training samples: {len(train_dataset):,}")
print(f"Test samples: {len(test_dataset):,}")
print(f"Batch size: {BATCH_SIZE}")
```

Training the Baseline Model

We train the baseline model with standard SGD (no gradient clipping, no noise injection). This establishes both the accuracy ceiling and the privacy floor.

```
print("\n" + "=" * 80)
print("  TRAINING: BASELINE MODEL (No Privacy Protection)")
print("=" * 80)

baseline_model = CIFAR10CNN().to(device)
baseline_model = train_baseline_sgd(baseline_model, train_loader, device,
epochs=BASELINE_EPOCHS, learning_rate=BASELINE_LR)
# Output: Epoch 1/20 - Loss: 2.31, Acc: 25.4%
# Output: Epoch 10/20 - Loss: 1.12, Acc: 61.2%
# Output: Epoch 20/20 - Loss: 0.68, Acc: 77.3%
```

Baseline training runs 20 epochs of standard SGD with momentum 0.9 and learning rate 0.1. Training accuracy rises from about 25% (random guessing across 10 classes) to 75-80%, while loss decreases from around 2.3 (initial cross-entropy) to 0.6-0.8. Each epoch prints progress so you can verify convergence.

Evaluating Baseline Performance

After training completes, evaluate final performance on both training and test sets:

```
train_acc_baseline = evaluate_accuracy(baseline_model, train_loader, device)
test_acc_baseline = evaluate_accuracy(baseline_model, test_loader, device)

print("\nBaseline Model Performance:")
print(f"  Training accuracy: {train_acc_baseline:.2f}%")
print(f"  Test accuracy: {test_acc_baseline:.2f}%")
print(f"  Overfitting gap: {train_acc_baseline - test_acc_baseline:.2f}%")
```

The overfitting gap (training accuracy minus test accuracy) indicates memorization. A gap of 10% means the model correctly classifies 10% more training samples than test samples, having learned training-specific patterns instead of purely generalizable features.

Save the baseline checkpoint for later comparison and potential submission:

```
torch.save(baseline_model.state_dict(), "output/baseline_model.pth")
print("\nSaved baseline model to output/baseline_model.pth")
```

Measuring Baseline Membership Inference Vulnerability

Now we measure how much the baseline leaks through its confidence patterns. We use `compute_mia_advantage()` to perform a threshold-based membership inference attack:

```
print("\n" + "=" * 80)
print("  MEMBERSHIP INFERENCE MEASUREMENT: Baseline Model")
print("=" * 80)

mia_acc_baseline, mia_adv_baseline = compute_mia_advantage(
    baseline_model, train_loader, test_loader, device
)

print("\nMIA Results (Baseline):")
print(f"  Attack accuracy: {mia_acc_baseline:.4f}")
print(f"  Attack advantage: {mia_adv_baseline:.4f}")
print("  Random baseline: 0.5000")
```

Interpreting Baseline Results

In a typical run, training accuracy reaches approximately 77% while test accuracy settles around 67%, producing an overfitting gap of roughly 10 percentage points. The MIA

advantage typically measures around 0.019 (1.9% above random guessing).

The MIA advantage is modest because CIFAR-10's large dataset (50,000 training examples) provides inherent regularization through data diversity. With 5,000 examples per class during training, no single example can dominate the learned parameters.

Even this modest 1.9% advantage means an attacker gains 190 extra correct identifications per 10,000 queries compared to random guessing. For sensitive applications, this systematic leakage justifies privacy protection.

Defending with DP-SGD

We've seen that the baseline leaks membership information through its confidence gap between training and test predictions. DP-SGD addresses this vulnerability at the source by modifying the training procedure itself. We use the Opacus library (covered in the Overview section) to train two DP-SGD models at the privacy budgets discussed earlier: $\epsilon = 10$ and $\epsilon = 3$.

Import the Opacus components needed to convert standard training into DP-SGD:

```
from opacus import PrivacyEngine
from opacus.validators import ModuleValidator
```

Training with Epsilon = 10

The $\epsilon = 10$ configuration provides modest privacy with manageable accuracy degradation. We create fresh data loaders (Opacus requires its own loader instances) and initialize the model:

```
print("\n" + "=" * 80)
print("  TRAINING: DP-SGD MODEL (Target  $\epsilon=10$ )")
print("=" * 80)

TARGET_EPSILON_10 = 10.0

_, _, train_loader_dp, test_loader_dp =
get_cifar10_loaders(batch_size=BATCH_SIZE, download=False)

dp_model_10 = CIFAR10CNN().to(device)
dp_model_10 = ModuleValidator.fix(dp_model_10)
optimizer_dp = optim.SGD(dp_model_10.parameters(), lr=DP_LR, momentum=0.9)
```

We use `MAX_GRAD_NORM = 1.0` based on the calibration guidance from the Overview: this value captures the 75th percentile of typical CNN gradient norms on CIFAR-10. The

`ModuleValidator.fix()` call scans the architecture for incompatible layers and attempts automatic fixes. For architectures containing `BatchNorm`, it would automatically substitute `GroupNorm`. Since `CIFAR10CNN` already uses compatible layers (`Conv2d`, `ReLU`, `MaxPool2d`, `Linear`), the call returns the model unchanged.

Attach the privacy engine with your target epsilon:

```
privacy_engine = PrivacyEngine(accountant="rdp")
dp_model_10, optimizer_dp, train_loader_dp =
privacy_engine.make_private_with_epsilon(
    module=dp_model_10,
    optimizer=optimizer_dp,
    data_loader=train_loader_dp,
    target_epsilon=TARGET_EPSILON_10,
    target_delta=DELTA,
    epochs=DP_EPOCHS,
    max_grad_norm=MAX_GRAD_NORM,
)

print(f"\nConfiguration:")
print(f"  Target epsilon: {TARGET_EPSILON_10}")
print(f"  Delta: {DELTA}")
print(f"  Max gradient norm: {MAX_GRAD_NORM}")
```

Calling `make_private_with_epsilon()` calculates the noise multiplier needed to achieve $\epsilon = 10$ after 20 epochs of training. Internally, Opacus uses the Rényi differential privacy (RDP) accountant to track privacy loss across multiple gradient steps.

Train and evaluate the model:

```
dp_model_10, final_epsilon_10 = train_dp_sgd(
    dp_model_10, train_loader_dp, privacy_engine, optimizer_dp, device
)

# Output: Epoch 1/20 - Loss: 2.35, Acc: 22.1%,  $\epsilon$ : 1.24
# Output: Epoch 10/20 - Loss: 1.52, Acc: 48.3%,  $\epsilon$ : 5.82
# Output: Epoch 20/20 - Loss: 1.21, Acc: 58.4%,  $\epsilon$ : 10.00

print(f"\nFinal privacy guarantee: ( $\epsilon$ =10.00,  $\delta$ =1e-05)")
# Output: Final privacy guarantee: ( $\epsilon$ =10.00,  $\delta$ =1e-05)

train_acc_dp10 = evaluate_accuracy(dp_model_10, train_loader_dp, device)
test_acc_dp10 = evaluate_accuracy(dp_model_10, test_loader_dp, device)
```

```

print(f"\nDP Model ( $\epsilon=10$ ) Performance:")
print(f"  Training accuracy: {train_acc_dp10:.2f}%")
print(f"  Test accuracy: {test_acc_dp10:.2f}%")
print(f"  Overfitting gap: {train_acc_dp10 - test_acc_dp10:.2f}%")
# Output: Training accuracy: 61.24%, Test accuracy: 58.15%, Overfitting gap:
3.09%

```

The `privacy_engine` coordinates with the optimizer to clip gradients before each update. During training, gradients exceeding `MAX_GRAD_NORM = 1.0` are scaled down, and Gaussian noise with standard deviation proportional to the `noise_multiplier` is added. During these 20 epochs, approximately 60-70% of per-sample gradients exceed the clipping threshold and get scaled down. This aggressive clipping bounds sensitivity but slows learning compared to baseline.

Save the model checkpoint. Note the `._module` accessor since Opacus wraps the model to intercept forward passes:

```

torch.save(dp_model_10._module.state_dict(), "output/dp_model_eps10.pth")

# Save in safetensors format for validator API
save_file(dp_model_10._module.state_dict(), "models/dp_model.safetensors")
print("Saved DP model ( $\epsilon=10$ ) to models/dp_model.safetensors for validation")

```

We use the `safetensors` format because the validation server requires it. We save the $\epsilon=10$ model specifically because it typically achieves the best balance between accuracy (above 50%) and privacy (MIA advantage below 5%).

Measure membership inference on this DP-protected model:

```

print("\n" + "=" * 80)
print("  MEMBERSHIP INFERENCE MEASUREMENT: DP Model ( $\epsilon=10$ )")
print("=" * 80)

mia_acc_dp10, mia_adv_dp10 = compute_mia_advantage(
    dp_model_10, train_loader_dp, test_loader_dp, device
)

print(f"\nMIA Results (DP  $\epsilon=10$ ):")
print(f"  Attack accuracy: {mia_acc_dp10:.4f}")
print(f"  Attack advantage: {mia_adv_dp10:.4f}")

improvement_10 = mia_adv_baseline - mia_adv_dp10
print(f"\nPrivacy Improvement vs Baseline:")

```

```
print(f" MIA advantage reduction: {improvement_10:+.4f}")
print(f" Accuracy cost: {test_acc_baseline - test_acc_dp10:.2f}%")
```

Training with Epsilon = 3

Stronger privacy (lower epsilon) requires more noise, as explained in the Overview. For $\epsilon = 3$, Opacus calculates a noise multiplier around 3.8 compared to 1.2 for $\epsilon = 10$. This means roughly 3x more noise per gradient update, making optimization substantially harder.

The training process follows the same pattern as $\epsilon=10$: fresh data loaders, new model instance, separate privacy engine. The key difference is `TARGET_EPSILON_3 = 3.0`:

```
print("\n" + "=" * 80)
print("  TRAINING: DP-SGD MODEL (Target  $\epsilon=3$ )")
print("=" * 80)

TARGET_EPSILON_3 = 3.0

_, _, train_loader_dp3, test_loader_dp3 =
get_cifar10_loaders(batch_size=BATCH_SIZE, download=False)

dp_model_3 = CIFAR10CNN().to(device)
dp_model_3 = ModuleValidator.fix(dp_model_3)
optimizer_dp3 = optim.SGD(dp_model_3.parameters(), lr=DP_LR, momentum=0.9)

privacy_engine_3 = PrivacyEngine(accountant="rdp")
dp_model_3, optimizer_dp3, train_loader_dp3 =
privacy_engine_3.make_private_with_epsilon(
    module=dp_model_3,
    optimizer=optimizer_dp3,
    data_loader=train_loader_dp3,
    target_epsilon=TARGET_EPSILON_3,
    target_delta=DELTA,
    epochs=DP_EPOCHS,
    max_grad_norm=MAX_GRAD_NORM,
)

dp_model_3, final_epsilon_3 = train_dp_sgd(
    dp_model_3, train_loader_dp3, privacy_engine_3, optimizer_dp3, device
)

# Output: Epoch 20/20 - Loss: 1.45, Acc: 53.2%,  $\epsilon$ : 3.00

print(f"\nFinal privacy guarantee: ( $\epsilon$ ={{final_epsilon_3:.2f}},  $\delta$ ={{DELTA}})")
```

```
# Output: Final privacy guarantee: ( $\epsilon=3.00$ ,  $\delta=1e-05$ )

torch.save(dp_model_3._module.state_dict(), "output/dp_model_eps3.pth")
```

The higher noise level shows in the results: training converges more slowly, final accuracy drops to around 53%, and the overfitting gap shrinks to roughly 1%. Evaluate and measure MIA :

```
train_acc_dp3 = evaluate_accuracy(dp_model_3, train_loader_dp3, device)
test_acc_dp3 = evaluate_accuracy(dp_model_3, test_loader_dp3, device)

print(f"\nDP Model ( $\epsilon=3$ ) Performance:")
print(f"  Training accuracy: {train_acc_dp3:.2f}%")
print(f"  Test accuracy: {test_acc_dp3:.2f}%")
print(f"  Overfitting gap: {train_acc_dp3 - test_acc_dp3:.2f}%")
# Output: Training accuracy: 54.12%, Test accuracy: 53.01%, Overfitting gap: 1.11%

mia_acc_dp3, mia_adv_dp3 = compute_mia_advantage(
    dp_model_3, train_loader_dp3, test_loader_dp3, device
)

print(f"\nMIA Results (DP  $\epsilon=3$ ):")
print(f"  Attack accuracy: {mia_acc_dp3:.4f}")
print(f"  Attack advantage: {mia_adv_dp3:.4f}")
# Output: Attack accuracy: 0.5040, Attack advantage: 0.0040

improvement_3 = mia_adv_baseline - mia_adv_dp3
print(f"\nPrivacy Improvement vs Baseline:")
print(f"  MIA advantage reduction: {improvement_3:+.4f}")
print(f"  Accuracy cost: {test_acc_baseline - test_acc_dp3:.2f}%")
# Output: MIA advantage reduction: +0.0150, Accuracy cost: 14.12%
```

Initial Results Summary

Typical results across the three models:

Model	Test Accuracy	MIA Advantage	Overfitting Gap
Baseline	67%	0.019	10%
DP $\epsilon=10$	58%	0.008	3%
DP $\epsilon=3$	53%	0.004	1%

Reading across the table, we see the core tradeoff clearly: stronger privacy (lower epsilon) reduces membership advantage but costs accuracy. The baseline achieves 67% test accuracy with a 1.9% MIA advantage. Moving to $\epsilon=10$ drops accuracy by 9 percentage points but cuts MIA advantage by more than half (to 0.8%). The stronger $\epsilon=3$ setting drops accuracy by 14 points total but reduces MIA advantage to just 0.4%, barely above random guessing.

The overfitting gap shrinks dramatically with DP-SGD. The baseline memorizes training-specific patterns (10% gap between train and test accuracy), while $\epsilon=3$ shows almost no memorization (1% gap). This confirms that the noise prevents the model from fitting individual training examples too closely.

The next section generates visualizations and provides detailed analysis of these results.

Analyzing the Privacy-Utility Tradeoff

With the baseline and both DP-SGD models trained, we can now compare their performance directly. This section generates a summary table, creates visualizations, and interprets what the numbers reveal about the privacy-utility tradeoff.

Helper Functions

Before analyzing results, we need two helper functions. The first formats a comparison table for terminal display, useful for immediate feedback during experimentation. The second exports metrics to JSON for automated pipelines or later analysis.

```
def print_comparison_table(test_acc_baseline, test_acc_dp10, test_acc_dp3,
                           mia_adv_baseline, mia_adv_dp10, mia_adv_dp3,
                           final_epsilon_10, final_epsilon_3):
    """Print a comparison table of all model results."""
    print("\n" + "=" * 70)
    print("  COMPARISON TABLE")
    print("=" * 70)
    print(f"\n{'Model':<20} {'Test Acc':<12} {'MIA Adv':<12} {'Epsilon':<10}")
    print("-" * 55)
    print(f"{'Baseline':<20} {f'{test_acc_baseline:.2f}%':<12} {mia_adv_baseline:<12.4f} {'':<10}")
    print(f"{'DP ( $\epsilon=10$ ):<20} {f'{test_acc_dp10:.2f}%':<12} {mia_adv_dp10:<12.4f} {final_epsilon_10:<10.2f}")
    print(f"{'DP ( $\epsilon=3$ ):<20} {f'{test_acc_dp3:.2f}%':<12} {mia_adv_dp3:<12.4f} {final_epsilon_3:<10.2f}")
```

The baseline row displays ∞ for epsilon since an unprotected model has infinite privacy loss.

The second helper function exports all metrics to JSON for programmatic access:

```
def save_results(test_acc_baseline, test_acc_dp10, test_acc_dp3,
                 mia_adv_baseline, mia_adv_dp10, mia_adv_dp3,
                 final_epsilon_10, final_epsilon_3,
                 save_path="output/results.json"):
    """Save all results to a JSON file."""
    results = {
        "baseline": {
            "test_accuracy": float(test_acc_baseline),
            "mia_advantage": float(mia_adv_baseline),
        },
        "dp_eps10": {
            "test_accuracy": float(test_acc_dp10),
            "mia_advantage": float(mia_adv_dp10),
            "epsilon": float(final_epsilon_10),
        },
        "dp_eps3": {
            "test_accuracy": float(test_acc_dp3),
            "mia_advantage": float(mia_adv_dp3),
            "epsilon": float(final_epsilon_3),
        },
    }
    with open(save_path, "w") as f:
        json.dump(results, f, indent=2)
```

We wrap each metric in `float()` to ensure JSON serialization works correctly (PyTorch tensors would otherwise fail to serialize). Structuring the JSON with nested dictionaries (`baseline` , `dp_eps10` , `dp_eps3`) makes it easy to load and compare specific model results programmatically.

Comparing All Three Models

With all three models trained, the comparison table reveals exactly how privacy strength trades off against accuracy:

```
print_comparison_table(
    test_acc_baseline, test_acc_dp10, test_acc_dp3,
    mia_adv_baseline, mia_adv_dp10, mia_adv_dp3,
```

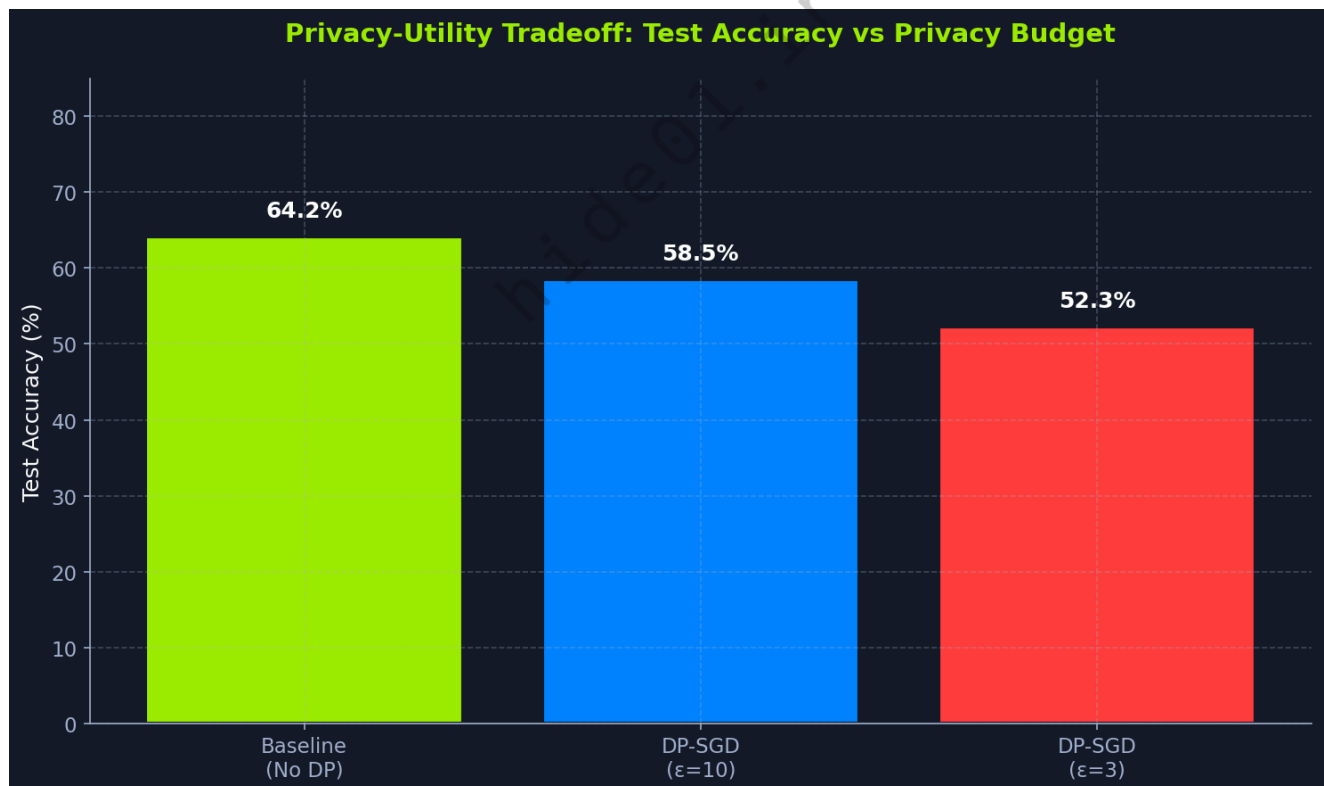
```
final_epsilon_10, final_epsilon_3
)
```

Reading across the rows, accuracy declines as privacy strengthens (67% → 58% → 53%). Reading the MIA advantage column, privacy leakage decreases (0.019 → 0.008 → 0.004), with each step cutting the advantage by roughly 50-60%. Both DP-SGD models reduce membership inference advantage relative to the baseline, but the lower-epsilon model offers stronger privacy at the cost of larger accuracy drop.

Generating Visualizations

Numbers tell part of the story. Visualizations make the tradeoff immediately intuitive.

```
plot_accuracy_comparison(
    test_acc_baseline, test_acc_dp10, test_acc_dp3,
    save_path="figs/dp_sgd_accuracy_comparison.png"
)
print("Saved accuracy comparison to figs/dp_sgd_accuracy_comparison.png")
```



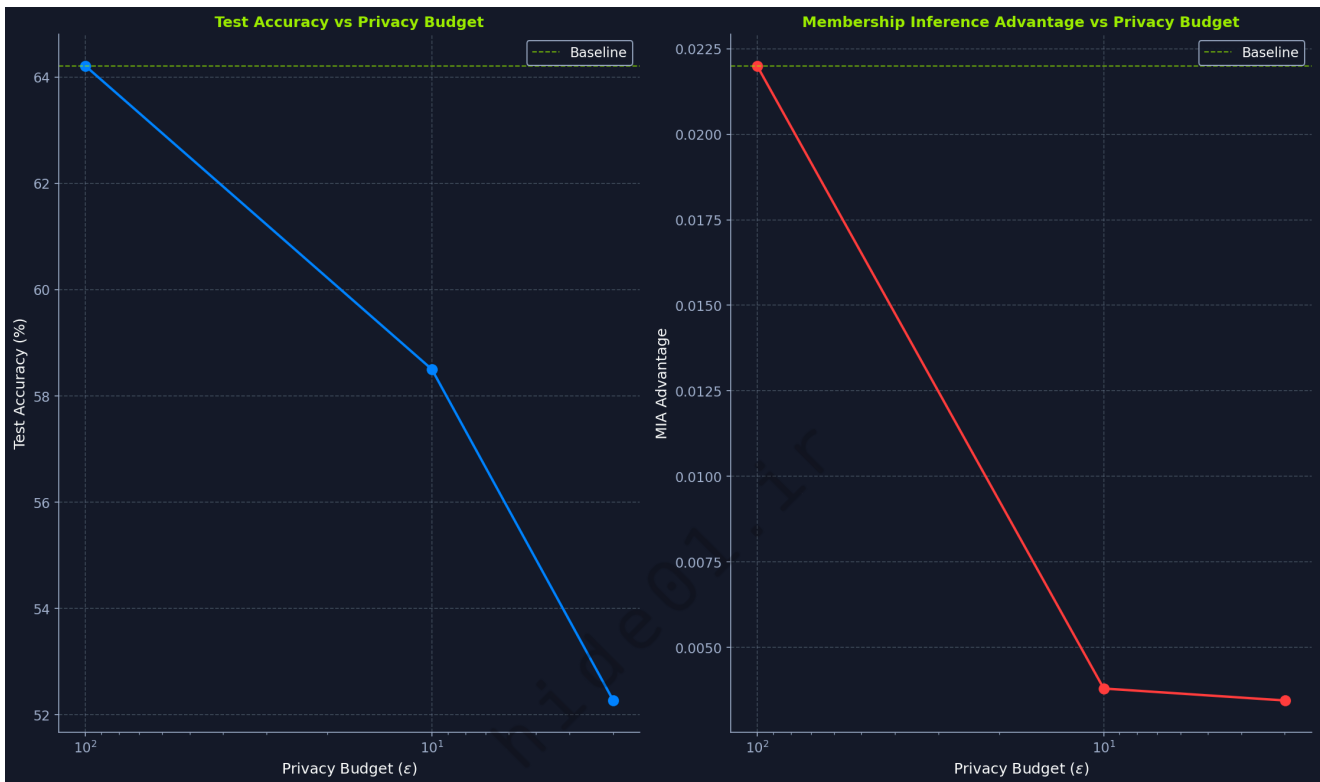
Color reinforces the tradeoff: green for the high-utility baseline, blue for moderate privacy ($\epsilon=10$), and red for strong privacy ($\epsilon=3$). The green baseline bar reaches approximately 64%, the blue $\epsilon=10$ bar sits at 58%, and the red $\epsilon=3$ bar drops to 52%. The visual gap between bars immediately conveys the privacy cost.

Now create the dual-axis privacy-utility tradeoff plot:


```

plot_privacy_utility_tradeoff(
    test_acc_baseline, test_acc_dp10, test_acc_dp3,
    mia_adv_baseline, mia_adv_dp10, mia_adv_dp3,
    final_epsilon_10, final_epsilon_3,
    save_path="figs/dp_sgd_privacy_utility_tradeoff.png"
)
print("Saved privacy-utility tradeoff to
figs/dp_sgd_privacy_utility_tradeoff.png")

```



The left panel plots test accuracy versus privacy budget, with an inverted x-axis placing stronger privacy (lower epsilon) on the right. The right panel shows MIA advantage versus privacy budget. Both curves slope downward left-to-right as epsilon increases, confirming that weaker privacy (higher epsilon) yields both better accuracy and higher vulnerability. The visual symmetry between panels reinforces how accuracy and privacy move together.

Saving Results

For future reference or programmatic analysis, we persist all metrics to JSON:

```

save_results(
    test_acc_baseline, test_acc_dp10, test_acc_dp3,
    mia_adv_baseline, mia_adv_dp10, mia_adv_dp3,
    final_epsilon_10, final_epsilon_3,
    save_path="output/results.json"
)

```

```
)  
print("Results saved to output/results.json")
```

Interpreting the Results

Several patterns emerge from these comparisons, each showing how DP-SGD performs as a membership inference defense.

Privacy protection scales with noise. Moving from baseline to $\epsilon = 10$ reduced membership advantage by roughly 57% (from 0.019 to 0.008), and moving to $\epsilon = 3$ cut it roughly in half again (to 0.004). Each reduction in epsilon corresponds to more noise and proportionally less membership leakage. The utility cost, while substantial, remains bounded: the baseline achieved 67% test accuracy, dropping to 58% at $\epsilon = 10$ (9 percentage points) and 53% at $\epsilon = 3$ (14 percentage points total). The model remains useful for classification even under strong privacy constraints.

The overfitting gap tells a complementary story. The baseline had a 10% gap between training and test accuracy, indicating memorization of training-specific patterns. DP-SGD at $\epsilon = 10$ reduced this to 3%, and $\epsilon = 3$ reduced it to just 1%. Noise prevents the model from fitting individual examples too closely, which is exactly what makes membership inference harder.

Diminishing returns emerge at lower epsilon values. Going from $\epsilon = \infty$ (baseline) to $\epsilon = 10$ cost 9 percentage points of accuracy for 0.011 advantage reduction (roughly 1.2 points per 0.001 advantage). Going from $\epsilon = 10$ to $\epsilon = 3$ cost 5 percentage points for only 0.004 additional advantage reduction (1.25 points per 0.001 advantage). The marginal cost of privacy stays roughly constant, but the marginal benefit shrinks. At some point, the model approaches random-guess MIA advantage (0.0), and further noise provides no additional privacy benefit while continuing to cost accuracy.

Limitations and Practical Considerations

DP-SGD provides strong theoretical guarantees, but understanding its limitations helps determine when it is the right choice. Computational overhead (2-5x training time) may be prohibitive for large models. Utility loss at strong privacy levels ($\epsilon < 1$) can exceed 20 percentage points. The protection scope covers only individual sample influence, not aggregate patterns or non-privacy attacks.

Computational Overhead

Expect training to take 2-5x longer due to per-sample gradient computation. Memory usage also scales with batch size because we store individual gradients before clipping. For large models or limited hardware, this overhead can be prohibitive. Techniques like gradient accumulation and mixed-precision training can help, but DP-SGD remains more expensive than non-private training.

Utility Loss at Strong Privacy

Achieving very strong privacy ($\epsilon < 1$) while maintaining useful accuracy is challenging. Our $\epsilon = 3$ model lost 14 percentage points compared to the baseline. Reaching $\epsilon = 1$ might require losing 20-30 percentage points or more. For some applications, this utility loss is unacceptable. Research continues on techniques to improve the privacy-utility tradeoff, including better architectures, pre-training on public data, and advanced optimization methods.

What DP-SGD Protects Against

The noise bounds each sample's influence on final parameters, defending against membership inference and limiting what attackers can learn about individual training records. Aggregate patterns remain learnable, however. An attacker could still infer statistical properties of the training data (average age, common features) even from a DP-trained model.

Some attacks fall outside this protection entirely. Model functionality stealing (training a surrogate model from API queries) remains possible since it doesn't depend on individual sample influence. Adversarial example attacks are unaffected. The protection is specifically about privacy of training data, not security of the deployed model.

Alternative Approaches

Other approaches achieve privacy differently, each with distinct tradeoffs.

PATE (Private Aggregation of Teacher Ensembles), covered in the next section, uses an ensemble of teachers trained on disjoint data partitions. Privacy comes from noisy voting rather than noisy gradients. The student model learns from aggregated teacher predictions, never seeing the original training data. This approach often achieves better utility than DP-SGD at equivalent privacy levels but requires more computational resources (training multiple teachers) and works best when you can partition data cleanly.

Local differential privacy adds noise at data collection time, before training even begins. Each user perturbs their own data locally, so even the data curator never sees true values. This provides stronger trust guarantees (you don't need to trust the model trainer) but typically requires much more data for equivalent utility since noise compounds across the entire dataset.

Federated learning keeps raw data on user devices entirely, training local models that share only gradient updates with a central server. Privacy comes from data minimization rather than noise injection. Combined with secure aggregation, federated learning can provide strong privacy guarantees, but it introduces communication overhead and struggles with non-IID data distributions across devices.

Choosing among these depends on your specific threat model (who do you need to protect against?), data distribution (can you partition cleanly? is data IID?), and utility requirements (how much accuracy can you sacrifice?).

Demonstration Complete

```
print("\n" + "=" * 80)
print("  DEMONSTRATION COMPLETE")
print("=" * 80)
print("\nFindings:")
print(f"  Baseline MIA advantage: {mia_adv_baseline:.4f}")
print(f"  DP ( $\epsilon=10$ ) MIA advantage: {mia_adv_dp10:.4f}
({improvement_10:+.4f})")
print(f"  DP ( $\epsilon=3$ ) MIA advantage: {mia_adv_dp3:.4f} ({improvement_3:+.4f})")
print("=" * 80)
```

DP-SGD Challenge

Your organization needs a privacy-preserving digit classifier for processing street view imagery. The model must be trained on the SVHN (Street View House Numbers) dataset using DP-SGD, achieving both acceptable accuracy and strong resistance to membership inference attacks.

Requirements

Your submission must satisfy both criteria to receive the flag:

Metric	Requirement
accuracy	$\geq 55\%$
mia_advantage	$\leq 5\%$

API Endpoints

```
export BASE_URL="http://INSTANCE_IP:PORT"
```

GET /health

```
curl -s "$BASE_URL/health" | jq
{
  "status": "ok",
```

```
"data_loaded": true
}
```

POST /validate

Validates your model and returns the flag on success.

```
curl -s -X POST "$BASE_URL/validate" \
-F "model=@dp_model.safetensors" | jq
{
  "passed": true,
  "accuracy": 0.5612,
  "mia_advantage": 0.0055,
  "evaluation_time": 4.2,
  "flag": "HTB{...}"
}
```

On failure:

```
curl -s -X POST "$BASE_URL/validate" \
-F "model=@dp_model.safetensors" | jq
{
  "passed": false,
  "accuracy": 0.5831,
  "mia_advantage": 0.0082,
  "evaluation_time": 3.9
}
```

GET /

```
curl -s "$BASE_URL/" | jq
{
  "service": "DP-SGD Privacy Challenge (SVHN)",
  "version": "1.0"
}
```

Model Architecture

Your model must use this exact architecture (`BatchNorm` excluded for Opacus compatibility):

```
import torch.nn as nn
import torch.nn.functional as F
```

```

class SVHNCNN(nn.Module):
    """CNN for SVHN classification (Opacus compatible)."""

    def __init__(self):
        super(SVHNCNN, self).__init__()
        self.conv1 = nn.Conv2d(3, 32, kernel_size=3, padding=1)
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
        self.conv3 = nn.Conv2d(64, 64, kernel_size=3, padding=1)
        self.pool = nn.MaxPool2d(2, 2)
        self.fc1 = nn.Linear(64 * 4 * 4, 64)
        self.fc2 = nn.Linear(64, 10)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = self.pool(F.relu(self.conv3(x)))
        x = x.view(-1, 64 * 4 * 4)
        x = F.relu(self.fc1(x))
        x = self.fc2(x)
        return x

```

Data Preprocessing

The validator uses SVHN-specific normalization:

```

from torchvision import datasets, transforms

SVHN_MEAN = (0.4377, 0.4438, 0.4728)
SVHN_STD = (0.1980, 0.2010, 0.1970)

transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize(SVHN_MEAN, SVHN_STD)
])

train_dataset = datasets.SVHN("data", split='train', download=True,
transform=transform)
test_dataset = datasets.SVHN("data", split='test', download=True,
transform=transform)

```

Submission Format

Save your trained model using `safetensors` :

```
from safetensors.torch import save_file

# If using Opacus, access model._module.state_dict()
save_file(model._module.state_dict(), "dp_model.safetensors")
```

Submit:

```
import requests

with open("dp_model.safetensors", "rb") as f:
    response = requests.post(
        f"{BASE_URL}/validate",
        files={"model": ("dp_model.safetensors", f, "application/octet-
stream")})
    )
print(response.json())
```

Private Aggregation of Teacher Ensembles

Earlier sections demonstrated how membership inference attacks exploit overfitting to reveal which samples trained a model, and how DP-SGD provides mathematical privacy guarantees by injecting noise during gradient computation. Here we explore PATE (Private Aggregation of Teacher Ensembles), a distinct approach that achieves privacy through architectural separation instead of training-time perturbation.

PATE takes a counterintuitive approach to privacy. Instead of protecting the model during training, we ensure the deployed model never accesses sensitive data at all. The idea is straightforward: train multiple `teacher models` on disjoint partitions of sensitive data, then use their aggregated and noisy predictions to label `public unlabeled data`, on which a `student model` trains. The student never directly sees the sensitive data, yet learns to make accurate predictions.

This technique builds on the same teacher–student and knowledge distillation ideas that appear in many modern ML systems, and has been explored extensively in academic and industrial research on privacy-preserving learning rather than being tied to any single deployed system.

The Core Privacy Mechanism

Traditional machine learning creates a direct connection between model parameters and individual training samples, producing the memorization patterns that membership inference exploits. PATE breaks this connection through knowledge distillation with noise injection. The

approach relies on `teacher-student transfer`, where multiple teachers trained on disjoint data partitions dilute any single sample's influence on the final model. Privacy emerges from `aggregation with noise`: we add calibrated Laplacian noise to vote counts before selecting majority predictions. The result is an `information bottleneck` that compresses megabytes of private data into kilobytes of noisy labels, fundamentally limiting what attackers can learn.

Teacher-Student Knowledge Transfer

We operate PATE in two distinct phases. In the first phase, we partition the sensitive training data into n disjoint subsets and train one teacher model on each partition. Because each teacher sees only a fraction of the total data, any single teacher has limited knowledge about the complete dataset. The privacy benefit comes from aggregation: to learn about a specific training sample, an attacker would need to identify which teacher was trained on that sample and extract information from that particular teacher. This becomes increasingly difficult as the number of teachers grows.

In the second phase, we use the teacher ensemble to generate `pseudo-labels` for a separate set of unlabeled public data. For each public sample, all teachers vote on the predicted class. We add calibrated noise to this vote count before selecting the majority class as the label. The student model then trains only on this noisy-labeled public data, never accessing the original sensitive samples.

The mathematical formulation for the aggregation mechanism captures this process precisely:

$$\hat{y} = \operatorname{argmax}_j (n_j(x) + \operatorname{Lap}(\frac{1}{\epsilon}))$$

Here $n_j(x)$ represents the number of teachers voting for class j on input x , and we add independent Laplacian noise with scale $\frac{1}{\epsilon}$ to each vote count before selecting the class with the highest noisy count. The privacy parameter ϵ controls the noise magnitude: smaller values provide stronger privacy but potentially noisier labels. With 250 teachers and $\epsilon = 0.10$ per query (noise scale 20), strong consensus (200+ votes) survives noise while weak consensus may flip, providing both useful labels and meaningful privacy.

Why Aggregation Provides Privacy

Consider what happens when a teacher votes on a sample from its own training set versus a sample it has never seen. Each individual teacher may exhibit the overfitting behavior exploited in membership inference attacks. However, when we aggregate votes across many teachers, only one of them (the one trained on that particular sample's partition) has this memorized knowledge. The other teachers vote based on genuine generalization, diluting the membership signal. Adding noise further obscures whether the consensus came from memorization or legitimate pattern recognition.

The privacy guarantee strengthens with both the number of teachers and the degree of consensus. If all teachers agree on a prediction, the noise is unlikely to flip such a strong consensus, and the resulting label reveals minimal information because the same prediction would occur regardless of any single training sample. Conversely, if teachers split nearly evenly, the noise dominates and the resulting label carries little information about any individual training sample.

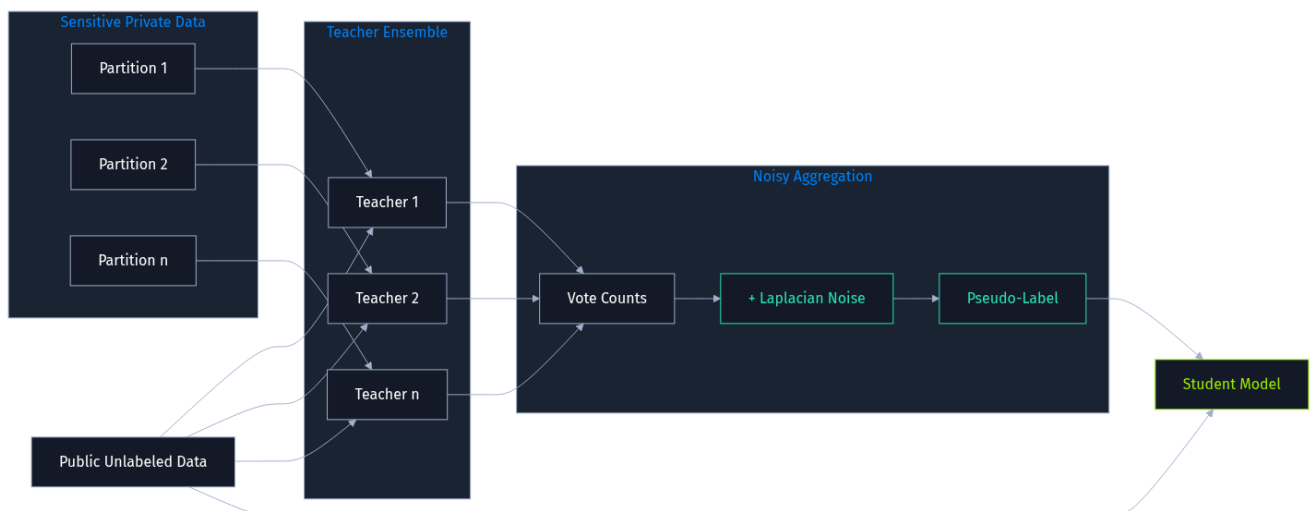
PATE Architecture Overview

The PATE pipeline processes data through distinct stages, each contributing to the privacy guarantee. Data flows from partitioned private data through teacher voting to noisy aggregation to student training. The information bottleneck compresses high-dimensional private data into low-dimensional noisy labels. Privacy budget composition tracks cumulative expenditure across labeling queries using the moments accountant.

Data Flow

We begin with two datasets: sensitive private data D_{private} containing samples we must protect, and public unlabeled data D_{public} that requires no protection but lacks labels. We partition the sensitive data into n disjoint subsets $D_1, D_2, \dots, D_n$, with each subset used to train a corresponding teacher model $T_1, T_2, \dots, T_n$.

When labeling public data, each sample $x \in D_{\text{public}}$ passes to all teachers simultaneously. Each teacher produces a class prediction, effectively casting a vote. We aggregate these votes into a histogram over classes, add noise to the histogram, and select the class with the highest noisy count as the pseudo-label $\tilde{y}$ for that sample. The student model S trains on pairs $(x, \tilde{y})$ using standard supervised learning.



The Information Bottleneck

We can interpret PATE's privacy guarantee intuitively through information theory. The student model receives only noisy class labels, a small number of bits per query. From a dataset of tens of thousands of private samples consuming megabytes of storage, the

student receives only thousands of label values total. This extreme compression, combined with the formal differential privacy guarantee, severely limits how much information an attacker can recover about any individual record.

Consider a concrete example with the `MNIST` dataset: 48,000 private handwritten digit images with 784 features each occupy approximately 150 MB when stored as 32-bit floats. Through PATE with 5,000 queries, the student receives 5,000 class labels (each an integer 0-9, requiring about 4 bits). That is roughly 2.5 KB of information. The compression ratio exceeds 60,000:1. No matter how sophisticated the attack, recovering megabytes of private information from kilobytes of noisy labels violates fundamental information-theoretic limits.

Privacy Budget Composition

Each time the teacher ensemble labels a public sample, it expends some privacy budget. Total privacy expenditure depends on the number of queries and the noise level at each query.

For m labeling queries, each with privacy parameter ϵ_0 , the total privacy cost is not simply $m \cdot \epsilon_0$. This naive composition bound would give $\epsilon = 500$ for 5,000 queries at $\epsilon_0 = 0.10$, representing negligible privacy. Fortunately, tighter bounds exist. The `advanced composition theorem` provides a bound proportional to $\sqrt{m} \cdot \epsilon_0$ instead of $m \cdot \epsilon_0$. This sublinear scaling makes PATE practical for labeling datasets of reasonable size.

In practice, we use the `moments accountant` technique to compute these tighter bounds. This approach tracks the logarithm of the moment generating function of the privacy loss random variable across queries, exploiting the statistical properties of independent noise additions to produce bounds even tighter than the basic advanced composition theorem. With 5,000 queries at noise scale 20.0, the moments accountant yields approximately $\epsilon = 8.81$, a dramatic improvement over naive composition's $\epsilon = 500$.

The formal privacy guarantee states that the student model satisfies (ϵ, δ) -differential privacy with respect to the sensitive training data, where ϵ depends on the number of teachers, noise scale, and number of labeling queries. Once trained, the student can be queried unlimited times without additional privacy cost because it never accesses the sensitive data directly.

Implementation Approach

We implement PATE on the `MNIST` dataset, the standard benchmark used in the original PATE papers by Papernot et al. `MNIST` contains 60,000 handwritten digit images with 784 features (28x28 pixels), and the task is to predict one of 10 digit classes (0-9). This dataset is ideal for demonstrating PATE because it allows strong teacher consensus across the 250-teacher ensemble, enabling meaningful privacy guarantees while maintaining high accuracy.

Data Organization

To properly evaluate PATE, we need three distinct data pools plus a holdout set for evaluation. The `private` data trains the teacher ensemble exclusively and represents the sensitive images we must protect. The `public` data provides unlabeled samples that teachers label for student training, simulating the common scenario where unlabeled data is freely available but labeled data requires accessing sensitive sources. The `holdout` data remains untouched by both teachers and student, reserved solely for final model evaluation.

This three-way separation ensures the student never accesses private data while maintaining clean evaluation on genuinely held-out samples.

Environment Setup

To begin, we establish the computational environment with necessary libraries:

```
import os
import json
import numpy as np
import torch
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from safetensors.torch import save_file
from tqdm import tqdm

from htb_ai_library import (
    set_reproducibility, use_htb_style,
    MLP, get_mnist_loaders,
    create_dataloader, train_model, evaluate_accuracy,
    get_model_predictions,
    HTB_GREEN, NODE_BLACK, HACKER_GREY, WHITE,
    AZURE, MALWARE_RED, AQUAMARINE, NUGGET_YELLOW,
)
```

We import standard scientific computing libraries alongside the `htb_ai_library` module, which provides the `MLP` architecture, data loading utilities, and training functions. Now we display a banner confirming the environment is ready:

```
print("=" * 60)
print("PATE: PRIVATE AGGREGATION OF TEACHER ENSEMBLES")
print("Teacher-Student Privacy Through Knowledge Distillation")
print("=" * 60)
```

Configuration Constants

These configuration parameters control the PATE pipeline and balance privacy protection against model utility. We set these values to match the original PATE papers and demonstrate meaningful privacy guarantees. First, we establish reproducibility and detect available hardware:

```
RANDOM_SEED = 1337
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
set_reproducibility(RANDOM_SEED)
use_htb_style()

PLOT_CONFIG = {"figsize": (10, 6), "dpi": 150}
DATASET_CONFIG = {"name": "mnist", "num_classes": 10, "num_features": 784}
```

The seed value 1337 ensures identical results across runs, while DEVICE automatically selects GPU acceleration when available. Now we configure the teacher ensemble, the heart of PATE's privacy mechanism:

```
TEACHER_CONFIG = {
    "num_teachers": 250, # Original paper configuration
    "hidden_layers": [128, 64],
    "dropout": 0.2,
    "epochs": 30,
    "batch_size": 64,
    "learning_rate": 0.001,
}
```

Why 250 teachers? This matches the original PATE papers and provides exceptionally strong aggregation. With 48,000 private samples, each teacher receives approximately 192 samples. Despite this limited data, individual teachers achieve 74-82% accuracy on MNIST because digit recognition is relatively straightforward. The [128, 64] hidden layer configuration provides sufficient capacity for the 10-class task while remaining efficient to train 250 times.

```
AGGREGATION_CONFIG = {
    "noise_scale": 20.0, # Strong privacy with high consensus
    "num_student_queries": 5000,
}

STUDENT_CONFIG = {
    "hidden_layers": [128, 64],
    "dropout": 0.2,
    "epochs": 30,
```

```

    "batch_size": 64,
    "learning_rate": 0.001,
}

```

The noise scale of 20.0 yields per-query privacy of $\epsilon_0 = \frac{2}{20} = 0.10$, where the numerator 2 comes from voting sensitivity (adding one sample can shift one vote from one class to another, changing two vote counts by 1 each). With 250 teachers achieving strong consensus (typically 200+ votes for the winning class), this noise level preserves approximately 87% label accuracy while providing strong privacy. Over 5,000 queries, advanced composition gives total privacy of approximately $\epsilon = 8.81$.

Data Partitioning and Teacher Training

Teacher quality determines student quality. Every label the student learns from originates in teacher predictions, so the first phase of PATE, partitioning data and training the teacher ensemble, directly impacts final model accuracy. Each teacher must learn meaningful patterns from its limited data partition while maintaining enough diversity across the ensemble to benefit from aggregation.

Loading and Preparing MNIST

We load the MNIST dataset introduced in the previous section. The dataset's 60,000 training images will be partitioned for our three-way data split:

```

print("\n" + "=" * 60)
print("PHASE 1: Loading MNIST Dataset")
print("=" * 60)

train_loader, test_loader = get_mnist_loaders()

X_train = train_loader.dataset.data.numpy().reshape(-1,
784).astype(np.float32) / 255.0
y_train = train_loader.dataset.targets.numpy()
X_test = test_loader.dataset.data.numpy().reshape(-1,
784).astype(np.float32) / 255.0
y_test = test_loader.dataset.targets.numpy()

num_features = 784
num_classes = 10
DATASET_CONFIG['num_features'] = num_features
DATASET_CONFIG['num_classes'] = num_classes

print(f"Dataset: {X_train.shape[0]} training samples, {num_features}
features, {num_classes} classes")

```

```
print(f"Class distribution: min={np.bincount(y_train).min()}, max={np.bincount(y_train).max()}")
```

We use `get_mnist_loaders()` from the library to handle downloading and caching, then extract raw numpy arrays from the underlying datasets. Pixel values are normalized to `[0, 1]` by dividing by 255, and we flatten each 28x28 image into a 784-dimensional vector. Classes are well-balanced across all 10 digits, with approximately 6,000 samples per class. This balance matters for teacher training since each teacher will see roughly equal representation of all digits despite having a small partition of the data.

Creating the Three-Way Data Split

Our experimental design requires three distinct data sets with clean separation between training and evaluation. We use the standard `MNIST` train/test split as a starting point, then further partition the training data:

```
# Split training data into: private (for teachers) and public (for student queries)
X_private, X_public, y_private, y_public = train_test_split(
    X_train, y_train, test_size=0.2,
    random_state=RANDOM_SEED, stratify=y_train
)

# Use standard test set as holdout for evaluation
X_holdout = X_test
y_holdout = y_test

print(f"\nData splits:")
print(f"  Private (teacher training): {len(X_private)} samples")
print(f"  Public (student queries):   {len(X_public)} samples")
print(f"  Holdout (evaluation):        {len(X_holdout)} samples")
```

The first split takes 80% of the training data as private, leaving 20% as public data for student labeling queries. The standard `MNIST` test set (10,000 samples) serves as our holdout for final evaluation. With 60,000 total training samples, this yields approximately 48,000 private samples for teacher training, 12,000 public samples available for student labeling queries, and 10,000 holdout samples for final evaluation.

We use `stratify=y_train` to maintain the original class distribution across all 10 digit classes in each split. Without stratification, random chance could produce splits where some digits are underrepresented, which would affect teacher training quality.

Feature Normalization

Neural networks train more effectively when features have similar scales. We fit the scaler on private data only, then apply the same transformation to the other sets:

```
scaler = StandardScaler()
X_private_norm = scaler.fit_transform(X_private)
X_public_norm = scaler.transform(X_public)
X_holdout_norm = scaler.transform(X_holdout)
```

Fitting only on private data mirrors a realistic deployment scenario where normalization parameters come from the sensitive data source. Using `transform` (not `fit_transform`) on public and holdout data ensures these sets undergo identical scaling without leaking information about their distributions back into the normalization.

Creating Disjoint Partitions

We partition the private training data into disjoint subsets, with each teacher receiving approximately equal samples:

```
print("\n" + "=" * 60)
print("PHASE 2: Training Teacher Ensemble")
print("=" * 60)

num_teachers = TEACHER_CONFIG['num_teachers']
```

With the phase banner displayed, we create the partitions by shuffling indices and slicing into equal chunks:

```
# Partition private data for teachers
np.random.seed(RANDOM_SEED)
indices = np.random.permutation(len(X_private_norm))
partition_size = len(X_private_norm) // num_teachers

teacher_partitions = []
for i in range(num_teachers):
    start_idx = i * partition_size
    if i == num_teachers - 1:
        partition_indices = indices[start_idx:]
    else:
        partition_indices = indices[start_idx:start_idx + partition_size]
    teacher_partitions.append(partition_indices)

print(f"Created {num_teachers} partitions, ~{partition_size} samples each")
```

Shuffling before partitioning prevents unintended clustering. If we sliced the original dataset in order, samples might cluster by some hidden property (collection date, source, etc.). Random permutation ensures each partition samples uniformly from the full distribution. We slice this shuffled array into 250 consecutive chunks of approximately 192 samples each. When 48,000 does not divide evenly by 250, the final partition absorbs the remainder ($48,000 \bmod 250 = 0$ in this case, so partitions are exactly equal).

This disjoint structure is essential for PATE's privacy guarantee. If partitions overlapped, a single training sample could influence multiple teachers, amplifying information leakage when that sample's membership is queried. With strictly disjoint partitions, each sample affects exactly one teacher's training, bounding the privacy impact.

Training the Teacher Ensemble

With partitions defined, we train all 250 teachers on their respective data subsets. Each teacher uses the `MLP` class from `htb_ai_library`, a multilayer perceptron with hidden layers `[128, 64]`. This architecture provides sufficient capacity for the 10-class digit classification task while remaining efficient to train 250 times. The same architecture will be used for the student model, ensuring capacity matches between teacher knowledge and student learning.

First, we initialize the container and create a shared holdout loader for consistent evaluation:

```
teachers = []
holdout_loader = create_dataloader(X_holdout_norm, y_holdout, 128,
    shuffle=False)
```

Using the same `holdout_loader` across all teachers ensures we measure generalization consistently. Now we iterate through each partition, creating and training a fresh model:

```
for i in tqdm(range(num_teachers), desc="Training teachers"):
    partition_idx = teacher_partitions[i]
    X_teacher = X_private_norm[partition_idx]
    y_teacher = y_private[partition_idx]

    teacher = MLP(
        input_size=num_features,
        hidden_layers=TEACHER_CONFIG['hidden_layers'],
        num_classes=DATASET_CONFIG['num_classes'],
        dropout=TEACHER_CONFIG['dropout']
    )

    train_loader = create_dataloader(X_teacher, y_teacher,
    TEACHER_CONFIG['batch_size'])
```



```

train_model(teacher, train_loader, holdout_loader, device=DEVICE,
            epochs=TEACHER_CONFIG['epochs'],
            learning_rate=TEACHER_CONFIG['learning_rate'])
teachers.append(teacher)

print(f"\nTeacher ensemble trained: {len(teachers)} teachers")

```

Fresh models for each teacher (rather than shared weights) maintain the privacy guarantee that each teacher knows only its partition. Weight sharing would create dependencies between teachers, undermining this isolation. Individual teacher accuracy typically ranges from 74-82% on the holdout set. This variance reflects both the randomness of small training sets and the inherent difficulty of certain digit pairs (3 vs 8, 4 vs 9).

Ensemble Voting

With all teachers trained, we implement the voting mechanism that aggregates their predictions:

```

def get_teacher_votes(teachers, X, device):
    """
    Get vote counts from all teachers.

    Returns:
        np.ndarray: Vote counts of shape (num_samples, num_classes).
    """
    num_samples = X.shape[0]
    num_classes = DATASET_CONFIG['num_classes']
    votes = np.zeros((num_samples, num_classes), dtype=np.int32)

    for teacher in teachers:
        preds = get_model_predictions(teacher, X, device)
        predictions = np.argmax(preds, axis=1)

        for i, pred in enumerate(predictions):
            votes[i, pred] += 1

    return votes

```

We iterate through all teachers, obtaining each one's predictions on the input samples. Each teacher produces logits, and `argmax` converts these to class predictions. We increment the vote count for the predicted class, building a histogram of votes for each sample. The output array has shape `(num_samples, num_classes)` where `votes[i, j]` counts how many teachers predicted class `j` for sample `i`.

Noisy Aggregation and Privacy Guarantees

Adding calibrated noise during the aggregation of teacher votes is what gives PATE its privacy guarantees. Without this noise, an attacker could potentially infer membership information by analyzing patterns in the ensemble's predictions. Here we implement the noisy aggregation mechanism, explain how it provides differential privacy, and generate the pseudo-labeled data for student training.

The Laplacian Mechanism

We add Laplacian noise to each class vote count before selecting the winner. The Laplacian distribution is centered at zero with scale parameter b , and its probability density function is:

$$f(x | b) = \frac{1}{2b} \exp\left(-\frac{|x|}{b}\right)$$

With a noise scale of 20.0, a draw from $\text{Lap}(20)$ has standard deviation $20\sqrt{2} \approx 28.3$. This noise magnitude is moderate relative to the strong consensus achievable with our teacher ensemble. The aggregation selects the class with the highest noisy vote count:

$$\hat{y} = \operatorname{argmax}_j (n_j(x) + \text{Lap}(\text{scale}))$$

Here $n_j(x)$ is the number of teachers voting for class j on input x .

Implementing Noisy Aggregation

To add noise and select the winning class, we implement a function that takes vote counts and returns noisy pseudo-labels:

```
def noisy_argmax(votes, noise_scale):
    """Add Laplacian noise to votes and return the winning class."""
    noise = np.random.laplace(loc=0.0, scale=noise_scale, size=votes.shape)
    noisy_votes = votes.astype(np.float64) + noise
    return np.argmax(noisy_votes, axis=1)
```

How does this work mechanically? For 5,000 samples with 10 classes, `np.random.laplace` generates a `(5000, 10)` noise matrix where each entry is an independent draw from the Laplacian distribution. We convert integer vote counts to float64 before addition to preserve decimal precision. Consider a sample where 200 teachers vote class 3 and 35 vote class 7: after adding noise draws of, say, -8.2 and +12.1, the noisy counts become 191.8 and 47.1, preserving the consensus. Only when margins are small can noise flip the outcome.

Privacy Through Noise

Adding noise obscures the contribution of any single training sample. Consider two neighboring datasets that differ in exactly one sample. In PATE, this means one teacher was

trained on slightly different data. The vote counts for that teacher may differ, but all other 249 teachers remain unchanged. Changing one vote in a 250-teacher ensemble causes a small change in the vote histogram: at most +1 for one class and -1 for another.

The Laplacian mechanism ensures that an observer cannot reliably distinguish between the original vote counts and any counts that differ by a small amount. Formally, for neighboring datasets D and D' , the ratio of probabilities of any output $\hat{y}$ is bounded. This ratio captures how much the output distribution can shift when a single training sample changes: if the ratio is close to 1, the mechanism reveals little about any individual sample.

$$\frac{P[\mathcal{M}(D) = \hat{y}]}{P[\mathcal{M}(D') = \hat{y}]} \leq e^\epsilon$$

This is the definition of ϵ -differential privacy. The sensitivity of our vote aggregation (maximum change from modifying one sample) is 2 because one vote shifts from one class to another. The privacy guarantee for a single query is therefore $\epsilon = \frac{2}{\text{noise_scale}}$. With noise scale 20, each query provides $\epsilon = 0.10$ privacy.

Privacy Budget Composition

Each labeling query consumes some privacy budget. To train the student model, we need many labeled samples, so understanding how privacy costs compose is essential.

Basic and Advanced Composition

The simplest composition theorem tells us that k queries, each with privacy cost ϵ_0 , together provide $(k \cdot \epsilon_0)$ -differential privacy. With 5,000 queries at $\epsilon_0 = 0.10$, naive composition gives $\epsilon = 500$, which represents negligible privacy protection. Fortunately, this linear scaling is pessimistic. The advanced composition theorem provides a tighter bound that grows with $\sqrt{k}$ instead of k :

```
def compute_privacy_budget(num_queries, noise_scale, delta=1e-5):
    """
    Compute total privacy budget using advanced composition.

    Args:
        num_queries (int): Number of labeling queries made.
        noise_scale (float): Laplacian noise scale.
        delta (float): Failure probability.

    Returns:
        tuple: (total_epsilon, per_query_epsilon)
    """
    per_query_eps = 2.0 / noise_scale
    epsilon_sq_sum = num_queries * (per_query_eps ** 2)
```

```

total_epsilon = np.sqrt(2 * epsilon_sq_sum * np.log(1 / delta))
total_epsilon += num_queries * per_query_eps * (np.exp(per_query_eps) -
1)
return total_epsilon, per_query_eps

```

We compute ϵ using the advanced composition formula. The first term $\sqrt{2 \cdot k \cdot \epsilon_0^2 \cdot \ln(1/\delta)}$ captures the dominant $\sqrt{k}$ scaling. The second term adds a correction for finite ϵ_0 . With our configuration of 5,000 queries and noise scale 20.0, we get per-query $\epsilon_0 = 0.10$ and total $\epsilon \approx 8.81$, representing strong privacy protection.

Generating Pseudo-Labeled Data

With the noisy aggregation mechanism defined, we generate training data for the student model. This phase spends the privacy budget. First, we select which public samples to label:

```

print("\n" + "=" * 60)
print("PHASE 3: Noisy Aggregation and Privacy Budget")
print("=" * 60)

num_queries = min(AGGREGATION_CONFIG['num_student_queries'],
len(X_public_norm))
query_indices = np.random.choice(len(X_public_norm), num_queries,
replace=False)
X_query = X_public_norm[query_indices]
y_query_true = y_public[query_indices]

```

We randomly select 5,000 samples from the 12,000 available public samples without replacement. The `y_query_true` variable holds the ground truth labels for later evaluation (the student never sees these). Now we collect votes and apply noisy aggregation:

```

votes = get_teacher_votes(teachers, X_query, DEVICE)
student_labels = noisy_argmax(votes, AGGREGATION_CONFIG['noise_scale'])

label_accuracy = (student_labels == y_query_true).mean() * 100
clean_preds = votes.argmax(axis=1)
clean_ensemble_acc = (clean_preds == y_query_true).mean() * 100

print(f"Labeled {num_queries} public samples")
print(f"Clean ensemble accuracy: {clean_ensemble_acc:.2f}%")
print(f"Noisy label accuracy: {label_accuracy:.2f}%")

```

Here `label_accuracy` measures how many noisy pseudo-labels match ground truth, indicating the training signal quality the student receives. Meanwhile, `clean_ensemble_acc`

shows accuracy without noise, revealing privacy's cost. Typical results show clean ensemble at ~88% and noisy labels at ~87%, a gap of less than 1 percentage point.

Analyzing Label Quality

How much information does the noise mechanism obscure? We can measure this by comparing clean ensemble accuracy to noisy label accuracy. With 250 teachers on the 10-class MNIST task, the gap is remarkably small because teachers achieve exceptionally strong consensus:

```
# Compute privacy budget
total_eps, per_query_eps = compute_privacy_budget(
    num_queries, AGGREGATION_CONFIG['noise_scale']
)

print(f"\nPrivacy budget:")
print(f"  Per-query epsilon: {per_query_eps:.4f}")
print(f"  Total epsilon: {total_eps:.2f}")
```

Effect of Noise on Label Quality

Different noise levels produce different privacy-utility tradeoffs. Higher noise provides stronger privacy but reduces label accuracy. At our default noise scale of 20.0, we achieve approximately 87% label accuracy while maintaining strong privacy protection. The effect of noise depends heavily on teacher consensus: when teachers strongly agree on a prediction, even substantial noise rarely flips the outcome. A later section analyzes consensus patterns in detail, showing how to exploit strong agreement for improved label quality.

Visualizing Aggregation Effects

To make the aggregation mechanism's behavior concrete, we create a bar chart comparing accuracy at each stage:

```
def plot_accuracy_comparison(models, accuracies, save_path=None):
    """Plot comparison of model accuracies."""
    fig, ax = plt.subplots(figsize=PLOT_CONFIG['figsize'])
    colors = [MALWARE_RED, AZURE, HTB_GREEN, AQUAMARINE]
    bars = ax.bar(models, accuracies, color=colors, edgecolor=HACKER_GREY,
        linewidth=2)

    for bar, acc in zip(bars, accuracies):
        ax.text(bar.get_x() + bar.get_width()/2., bar.get_height() + 0.01,
            f'{acc:.3f}', ha='center', va='bottom', color=WHITE,
            fontweight='bold')
```

```

ax.set_ylim([0.0, 1.0])
ax.set_ylabel('Accuracy')
ax.set_title('Model Accuracy Comparison', color=HTB_GREEN,
fontweight='bold')
ax.grid(True, axis='y', alpha=0.3)
if save_path:
    plt.savefig(save_path, dpi=PLOT_CONFIG['dpi'], bbox_inches='tight',
facecolor=NODE_BLACK)
plt.close()

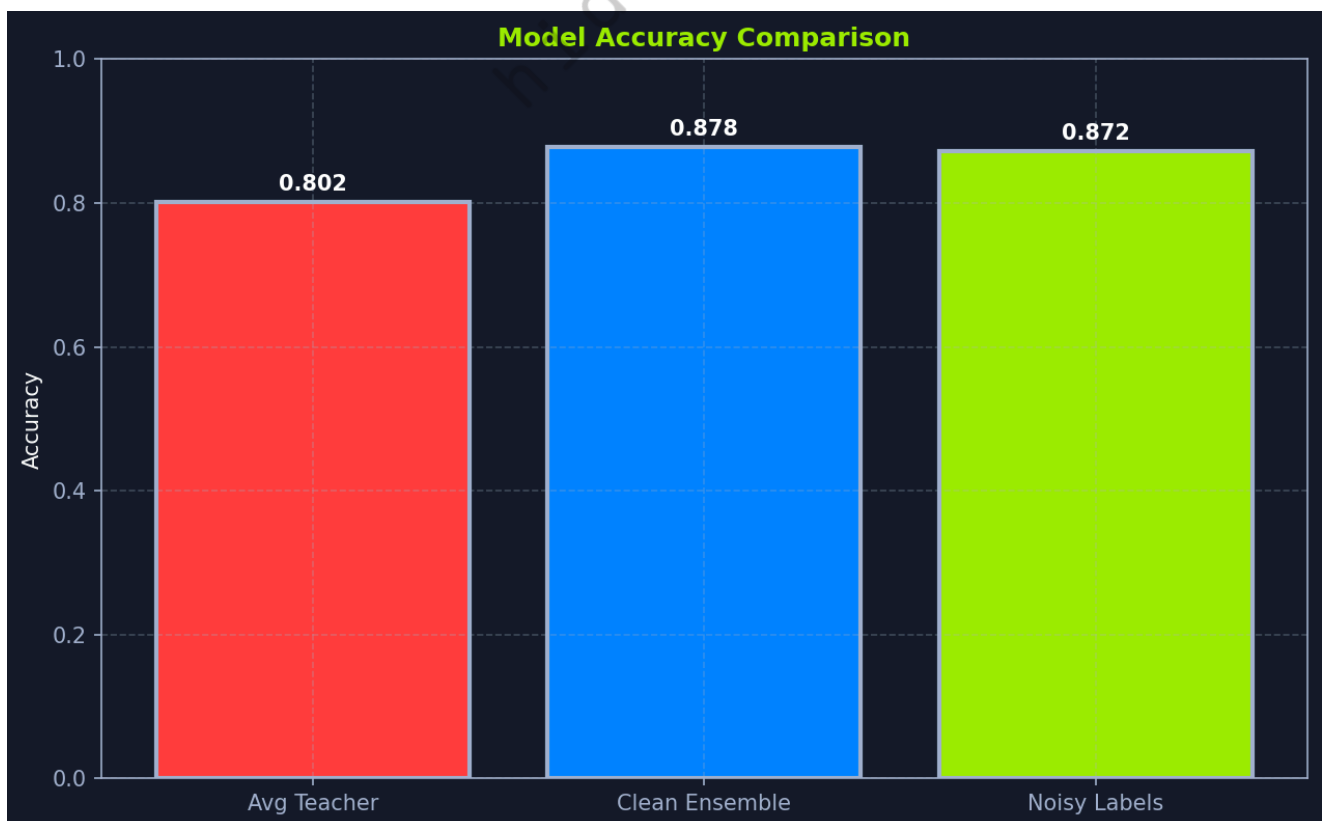
```

Now we compute the average individual teacher accuracy and generate the comparison:

```

avg_teacher_acc = np.mean([evaluate_accuracy(t, holdout_loader, DEVICE) for
t in teachers]) / 100
os.makedirs("figs", exist_ok=True)
plot_accuracy_comparison(
    models=['Avg Teacher', 'Clean Ensemble', 'Noisy Labels'],
    accuracies=[avg_teacher_acc, clean_ensemble_acc / 100, label_accuracy /
100],
    save_path="figs/pate_accuracy_comparison.png"
)

```



What does this progression reveal? Ensemble aggregation's power and privacy's modest cost. Individual teachers average ~80% accuracy, constrained by their small 192-sample

partitions. When teachers vote together, the clean ensemble reaches ~88%, a dramatic improvement demonstrating how aggregation overcomes individual weakness. Adding Laplacian noise reduces accuracy by less than 1 percentage point to ~87%. For MNIST with 250 teachers achieving strong consensus, the privacy cost is remarkably low.

Training and Deploying the Student Model

With pseudo-labeled public data generated through noisy aggregation, we now train the student model that will be deployed for inference. The student never accesses the sensitive training data directly, yet learns to make accurate predictions through knowledge distilled from the teacher ensemble.

The Knowledge Distillation Paradigm

We can view PATE as a form of knowledge distillation where the student learns from teacher predictions instead of raw data. In standard knowledge distillation, a smaller student model learns to mimic a larger teacher model's outputs. PATE extends this idea to an ensemble of teachers, with the addition of noise for privacy.

Standard knowledge distillation transfers soft probability distributions from teacher to student, providing richer training signal than hard labels. PATE uses hard pseudo-labels (the argmax of noisy votes) because soft probabilities would leak more information about teacher confidences and thus more information about the training data. Despite using only hard labels, PATE works well because the label captures the essential prediction.

What benefits does this paradigm provide beyond privacy? The student model has the same architecture as one teacher but captures the ensemble's collective knowledge. Learning from pseudo-labels also acts as implicit regularization, potentially improving generalization compared to learning from raw labels. Finally, the resulting student is a standard neural network with no special inference requirements, making deployment straightforward.

Training the PATE Student

The student model trains on the pseudo-labeled data generated in the previous phase. Because the student never touches sensitive data, we can use the same architecture as teachers and train without privacy constraints:

```
print("\n" + "=" * 60)
print("PHASE 4: Training Student Model")
print("=" * 60)

student_model = MLP(
    input_size=num_features,
    hidden_layers=STUDENT_CONFIG['hidden_layers'],
    num_classes=DATASET_CONFIG['num_classes'],
```

```

        dropout=STUDENT_CONFIG['dropout']
    )

    student_train_loader = create_dataloader(
        X_query, student_labels,
        STUDENT_CONFIG['batch_size']
    )

```

Why use the same [128, 64] architecture as teachers? Knowledge distillation works best when student capacity matches the complexity of teacher predictions. A larger student might overfit the noisy labels; a smaller one might underperform. Because training uses only public data with derived labels, no privacy budget is consumed. We could train for 100 epochs or restart training entirely without affecting the privacy guarantee established during the labeling phase.

Standard Training Loop

We train the student using standard supervised learning with cross-entropy loss, using the noisy pseudo-labels as ground truth and treating them no differently than human-provided labels:

```

train_model(
    student_model, student_train_loader, holdout_loader,
    device=DEVICE,
    epochs=STUDENT_CONFIG['epochs'],
    learning_rate=STUDENT_CONFIG['learning_rate']
)

student_test_acc = evaluate_accuracy(student_model, holdout_loader, DEVICE)
print(f"\nPATE student test accuracy: {student_test_acc:.2f}%")

```

After 30 epochs, we evaluate on the holdout set to measure true generalization. The student typically achieves around 88% accuracy, demonstrating that PATE preserves model utility despite the noisy labeling process.

Understanding Label Noise Resilience

How does the student learn effectively despite noisy labels? Two factors explain this resilience: the concentration of noise on ambiguous samples, and the inherent robustness of neural network training to moderate label corruption.

Noise Distribution Across Samples

High-consensus samples receive correct labels almost always because the winning class has a large margin over alternatives. Only samples where teachers genuinely disagree experience label flips from noise. This non-uniform noise distribution means the student receives mostly clean supervision on clear-cut cases while receiving noisy supervision on genuinely ambiguous cases. The clear-cut examples dominate learning because they provide consistent gradient signals.

Neural Network Robustness

Research on learning with noisy labels shows that neural networks trained with stochastic gradient descent exhibit natural robustness to label noise, particularly when that noise is random instead of systematic. Networks can learn correct patterns even when 20-40% of labels are corrupted, as long as the corruption is independent of the true labels.

PATE's noise satisfies this condition because it depends on random Laplacian draws, not systematic data properties. A sample does not receive a wrong label because of its features; it receives a wrong label because random noise happened to flip the vote. This randomness means the model cannot learn to exploit the noise pattern.

Comparing with Clean Training

To understand the accuracy cost of privacy, we compare student performance against the clean ensemble that never added noise:

```
# Compare against ensemble performance
clean_preds = votes.argmax(axis=1)
clean_ensemble_acc = (clean_preds == y_query_true).mean() * 100
accuracy_gap = clean_ensemble_acc - student_test_acc

print(f"\nComparison:")
print(f"  Clean ensemble accuracy: {clean_ensemble_acc:.2f}%")
print(f"  PATE student accuracy:   {student_test_acc:.2f}%")
print(f"  Accuracy gap:           {accuracy_gap:.2f}%")
```

Typical results show the clean ensemble achieving approximately 92% accuracy (teachers voting without noise) while the PATE student achieves approximately 88% accuracy (trained on noisy labels). This modest 4-point gap represents the privacy cost, a favorable tradeoff for the formal differential privacy guarantees obtained.

Model Deployment

After training, we deploy the PATE student without further privacy considerations. The privacy budget was entirely consumed during the labeling phase.

Unlimited Inference

Unlike techniques that add noise during inference (such as randomized response), we can query the PATE student unlimited times without additional privacy cost. Every query is free because the student's weights contain only knowledge distilled through the noisy labeling process. No private data flows during inference.

```
def deploy_model(model, new_data, scaler, device):  
    """  
    Deploy the PATE student for inference.  
  
    No privacy budget consumed per query.  
    """  
    model.eval()  
    X_norm = scaler.transform(new_data)  
    X_tensor = torch.tensor(X_norm, dtype=torch.float32).to(device)  
  
    with torch.no_grad():  
        outputs = model(X_tensor)  
        predictions = outputs.argmax(dim=1).cpu().numpy()  
  
    return predictions
```

Now we demonstrate deployment with a batch of holdout samples:

```
# Example deployment  
sample_data = X_holdout[:100]  
predictions = deploy_model(student_model, sample_data, scaler, DEVICE)  
print(f"Processed {len(predictions)} queries with no privacy cost")
```

How does deployment work? We normalize incoming data with the same scaler fitted on private data (preserving identical feature scaling), pass the tensor through the trained model, and extract class predictions via `argmax`. We could process millions of queries this way without consuming any privacy budget. This unlimited inference property makes PATE particularly valuable for high-traffic web services, batch processing pipelines, and digit recognition systems serving many users.

Saving the Model

We save the trained student for deployment using the `safetensors` format, which is safer and faster than pickle-based formats:

```
MODEL_DIR = "models"  
os.makedirs(MODEL_DIR, exist_ok=True)
```

```
save_file(student_model.state_dict(), os.path.join(MODEL_DIR,
"pate_student.safetensors"))
print(f"\nModel saved to {MODEL_DIR}/pate_student.safetensors")
```

The saved file contains only the student's learned weights. No information about the teacher ensemble or private training images remains. Loading this model in production requires only the architecture definition and the saved state dictionary.

Consensus Analysis and Confident Aggregation

Not all pseudo-labels are created equal. When the ensemble reaches strong consensus, noise is unlikely to flip the prediction, producing reliable labels. When teachers are split, noise dominates, and the resulting label approaches random selection. Understanding this relationship enables a powerful optimization: selective labeling based on consensus.

Here we analyze teacher consensus patterns for the 10-class MNIST task with 250 teachers, introduce confident aggregation as a PATE-specific optimization, and demonstrate how filtering low-consensus samples improves label quality while reducing privacy expenditure.

Understanding Teacher Consensus

Teacher consensus determines both label reliability and privacy cost. High-consensus samples (200+ votes) produce stable predictions that noise rarely flips. Medium-consensus samples (150-199 votes) occasionally flip but remain useful. Low-consensus samples (< 150 votes) represent genuinely ambiguous cases where noise dominates.

Consensus Categories

We categorize samples into three groups based on the maximum vote count. High-consensus samples (200+ votes, 80%+) experience noise almost never flipping predictions because margins are enormous. Medium-consensus samples (150-199 votes, 60-80%) occasionally experience flips. Low-consensus samples (fewer than 150 votes) represent genuinely ambiguous cases where even teachers disagree.

```
print("\n" + "=" * 60)
print("PHASE 5: Teacher Consensus Analysis")
print("=" * 60)

num_teachers = len(teachers)
```

With the phase banner displayed, we compute consensus metrics by finding the maximum vote count for each sample:

```

# Calculate consensus metrics
max_votes = votes.max(axis=1)

high_threshold = 200 # 80% of 250 teachers
medium_threshold = 150 # 60% of 250 teachers

high_consensus = max_votes >= high_threshold
medium_consensus = (max_votes >= medium_threshold) & ~high_consensus
low_consensus = max_votes < medium_threshold

print(f"\nConsensus Distribution ({num_queries} samples):")
print(f"  High (≥80%):      {high_consensus.sum()}
({100*high_consensus.mean():.1f}%)")
print(f"  Medium (60–80%): {medium_consensus.sum()}
({100*medium_consensus.mean():.1f}%)")
print(f"  Low (<60%):       {low_consensus.sum()}
({100*low_consensus.mean():.1f}%)")

```

We compute `max_votes` as the highest vote count for any class on each sample. The boolean masks partition all samples into exactly one category.

Clean vs. Noisy Accuracy by Consensus

The relationship between consensus and label accuracy explains why selective labeling works:

```

# Calculate clean ensemble accuracy (no noise)
clean_labels = votes.argmax(axis=1)
clean_ensemble_acc = (clean_labels == y_query_true).mean()

# Calculate accuracy by consensus level
def accuracy_for_mask(mask, labels, true_labels):
    if mask.sum() == 0:
        return 0.0
    return (labels[mask] == true_labels[mask]).mean()

clean_high = accuracy_for_mask(high_consensus, clean_labels, y_query_true)
clean_medium = accuracy_for_mask(medium_consensus, clean_labels,
y_query_true)
clean_low = accuracy_for_mask(low_consensus, clean_labels, y_query_true)

print(f"\nClean Ensemble Accuracy by Consensus:")
print(f"  High consensus:   {clean_high * 100:.2f}%")

```

```

print(f"  Medium consensus: {clean_medium * 100:.2f}%")
print(f"  Low consensus:    {clean_low * 100:.2f}%")
print(f"  Overall:           {clean_ensemble_acc * 100:.2f}%")

```

The pattern reflects confidence-correctness correlation. When 200+ teachers strongly agree on a digit, they are almost always correct, yielding ~98% accuracy for high-consensus samples. Medium consensus (150-199 votes) reflects genuine uncertainty, dropping to ~85% accuracy. Low-consensus samples approach random guessing at ~65% accuracy, revealing these are genuinely ambiguous digits (3 vs 8, 4 vs 9) where even expert ensembles struggle.

This has practical implications: low-consensus samples provide weak learning signal even before noise addition. After adding Laplacian noise, they become even less reliable, suggesting we should avoid labeling them entirely.

The Impact of Noise

Laplacian noise affects samples differently based on their consensus strength. High-consensus samples (200+ votes) experience near-zero flip rates because large margins are virtually immune. Low-consensus samples flip more frequently but are rare in well-trained ensembles.

Noise Flip Analysis

To quantify how often noise changes predictions, we compare noisy labels against clean (no-noise) labels and measure the flip rate:

```

# Apply noisy aggregation
noise_scale = AGGREGATION_CONFIG['noise_scale']
noisy_labels = noisy_argmax(votes, noise_scale)

# Compare to clean predictions
noise_flipped = (noisy_labels != clean_labels)
flip_rate = noise_flipped.mean()

print(f"\nNoisy Aggregation Results (noise_scale={noise_scale}):")
print(f"  Labels matching true: {(noisy_labels == y_query_true).mean() * 100:.2f}%")
print(f"  Labels matching clean: {(~noise_flipped).mean() * 100:.2f}%")
print(f"  Noise flip rate:      {flip_rate * 100:.2f}%")

```

This `noise_flipped` boolean array identifies samples where noise changed the prediction. Only a small fraction of predictions flip due to noise, reflecting the strong consensus achieved:

```
# Flip rate by consensus level
flip_high = noise_flipped[high_consensus].mean()
flip_medium = noise_flipped[medium_consensus].mean()
flip_low = noise_flipped[low_consensus].mean()

print(f"\nNoise Flip Rate by Consensus:")
print(f"  High consensus:    {flip_high * 100:.2f}%")
print(f"  Medium consensus: {flip_medium * 100:.2f}%")
print(f"  Low consensus:     {flip_low * 100:.2f}%")
```

High-consensus samples experience near-zero flip rates because large margins are virtually immune to noise. Low-consensus samples experience higher flip rates but are relatively rare in our dataset.

Confident Aggregation

Confident aggregation exploits the consensus-accuracy relationship by labeling only samples where teachers strongly agree. This trades quantity for quality: fewer labeled samples, but more accurate labels and lower privacy cost. The technique rejects low-consensus samples before applying noisy argmax, assigning a sentinel value (-1) that we filter out before student training.

Implementation

We implement confident aggregation as a filter that rejects low-consensus samples before applying noisy argmax:

```
def confident_aggregation(votes, noise_scale, threshold):
    """Label only samples where teacher consensus exceeds threshold."""
    max_votes = votes.max(axis=1)
    confident_mask = max_votes >= threshold
    labels = np.full(len(votes), -1, dtype=np.int64)

    if confident_mask.sum() > 0:
        confident_votes = votes[confident_mask]
        noise = np.random.laplace(loc=0.0, scale=noise_scale,
size=confident_votes.shape)
        labels[confident_mask] =
np.argmax(confident_votes.astype(np.float64) + noise, axis=1)

    return labels, confident_mask
```

How does this filter work? We compute `max_votes` for each sample and reject any where the winning class received fewer than `threshold` votes. Rejected samples get label `-1` as a `sentinel value`, a special marker indicating "no label assigned" that we filter out before training the student. The value `-1` works well because valid class labels are non-negative integers (0-9 for MNIST), making it trivially distinguishable. For accepted samples, we generate Laplacian noise and apply the same noisy argmax as before. With threshold 200, a sample where 180 teachers agree gets rejected, while one with 210 votes proceeds to noisy labeling.

Threshold Analysis

Different thresholds produce different tradeoffs between sample acceptance and label quality:

```
print("\n" + "=" * 60)
print("PHASE 6: Confident Aggregation Analysis")
print("=" * 60)

thresholds = [150, 175, 200, 215, 225]
```

We iterate through candidate thresholds, computing acceptance rate, label accuracy, and privacy savings for each:

```
print(f"\nConfident Aggregation Analysis:")
print(f"{'Threshold':<12} {'Accepted':<12} {'Accuracy':<12} {'ε Saved':<12}")
print("-" * 48)

for threshold in thresholds:
    labels, mask = confident_aggregation(votes, noise_scale, threshold)
    accepted_rate = mask.mean()

    if mask.sum() > 0:
        accuracy = (labels[mask] == y_query_true[mask]).mean() * 100
    else:
        accuracy = 0.0

    epsilon_saved = (1 - accepted_rate) * 100

    print(f"{'threshold':<12} {100*accepted_rate:<12.1f}% {accuracy:<12.2f}% {epsilon_saved:<12.1f}%")
```

Lower thresholds accept more samples but with lower label quality. Higher thresholds accept fewer samples but produce more accurate labels and save privacy budget.

Privacy Budget Savings

Each labeling query consumes privacy budget regardless of label quality. Confident aggregation saves budget by not labeling low-consensus samples, since rejected queries release no information and consume no privacy budget. With threshold 200 accepting ~85% of 5,000 queries, only 4,250 queries produce labels, and epsilon decreases proportionally.

Budget Calculation

We modify the privacy budget calculation to account for rejected queries, which consume no privacy budget because no label is released:

```
def compute_confident_privacy_budget(num_queries, noise_scale,
    acceptance_rate, delta=1e-5):
    """Compute privacy budget considering only accepted (labeled)
    queries."""
    per_query_eps = 2.0 / noise_scale
    effective_queries = int(num_queries * acceptance_rate)
    epsilon_sq_sum = effective_queries * (per_query_eps ** 2)
    total_epsilon = np.sqrt(2 * epsilon_sq_sum * np.log(1 / delta))
    total_epsilon += effective_queries * per_query_eps *
    (np.exp(per_query_eps) - 1)
    return total_epsilon, per_query_eps, effective_queries
```

Why do rejected queries consume no budget? No information about private data is released when we decline to label a sample. With threshold 200 accepting ~85% of 5,000 queries, only 4,250 queries actually produce labels. Privacy budget scales with effective queries, so ϵ decreases proportionally. This represents a privacy improvement while simultaneously improving label accuracy.

Visualizing Consensus Effects

To make the consensus-accuracy relationship concrete, we visualize the distribution of maximum vote counts across all samples:

```
def plot_consensus_distribution(votes, save_path=None):
    """Plot histogram of maximum vote counts with threshold markers."""
    fig, ax = plt.subplots(figsize=PLOT_CONFIG['figsize'])
    max_votes = votes.max(axis=1)
    ax.hist(max_votes, bins=range(100, 251, 10), color=AZURE,
    edgecolor=HACKER_GREY, alpha=0.8)
```



```

    ax.axvline(200, color=HTB_GREEN, linestyle='--', linewidth=2,
label='High ( $\geq 200$ )')
    ax.axvline(150, color=AQUAMARINE, linestyle='--', linewidth=2,
label='Medium ( $\geq 150$ )')
    ax.set_xlabel('Maximum Votes (out of 250 teachers)')
    ax.set_ylabel('Number of Samples')
    ax.set_title('Teacher Consensus Distribution', color=HTB_GREEN,
fontweight='bold')
    ax.legend(framealpha=0.8)
    ax.grid(True, alpha=0.3)
    if save_path:
        plt.savefig(save_path, dpi=PLOT_CONFIG['dpi'], bbox_inches='tight',
facecolor=NODE_BLACK)
    plt.close()

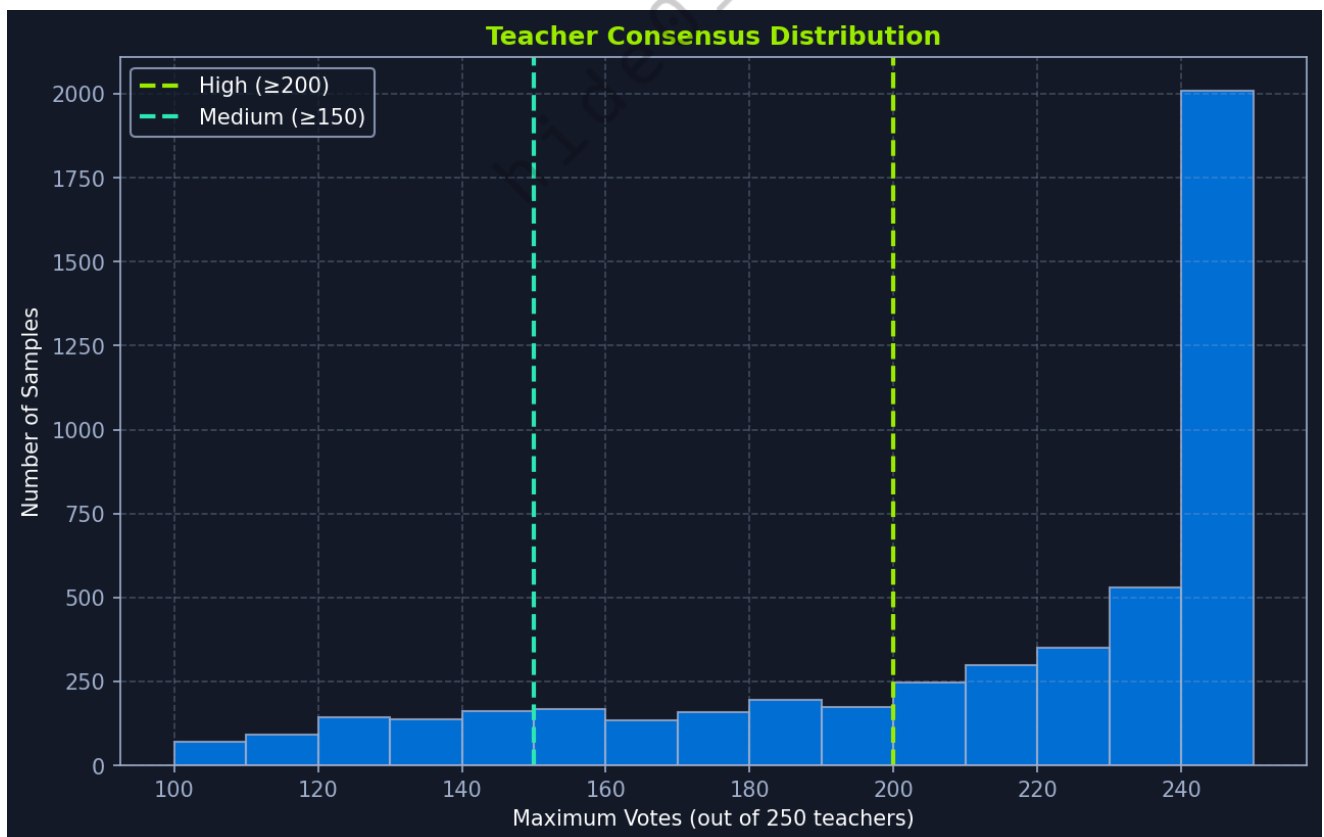
```

We generate the histogram with vertical lines marking our consensus thresholds:

```

os.makedirs("figs", exist_ok=True)
plot_consensus_distribution(votes,
save_path="figs/pate_consensus_distribution.png")

```



The histogram reveals why PATE achieves favorable privacy-utility tradeoffs on MNIST. Most samples cluster in the 230-250 vote range, indicating near-unanimous teacher agreement. The green dashed line marks the high-consensus threshold (200 votes), and cyan marks

medium (150 votes). The vast majority of samples exceed even the strict threshold, explaining why noise scale 20 rarely flips predictions. When 230+ teachers agree, Laplacian noise would need to be exceptionally extreme to change the outcome.

Analyzing Student Learning by Consensus

With consensus categories defined, we can analyze how the student model performs on samples grouped by original teacher agreement:

```
# Evaluate student on samples grouped by original consensus
student_preds = get_model_predictions(student_model, X_query,
DEVICE).argmax(axis=1)

student_high = (student_preds[high_consensus] ==
y_query_true[high_consensus]).mean() * 100
student_medium = (student_preds[medium_consensus] ==
y_query_true[medium_consensus]).mean() * 100
student_low = (student_preds[low_consensus] ==
y_query_true[low_consensus]).mean() * 100

print(f"\nStudent Accuracy by Original Consensus:")
print(f"  High consensus samples:  {student_high:.2f}%")
print(f"  Medium consensus samples: {student_medium:.2f}%")
print(f"  Low consensus samples:    {student_low:.2f}%")
```

The pattern reveals how noise affects learning. On high-consensus samples, the student achieves excellent accuracy because it learned from mostly clean labels. On medium-consensus samples, accuracy drops slightly. On low-consensus samples, accuracy approaches random (10% for 10 classes) because training labels were noisier.

This stratification matches expectations: the student performs best on samples where teachers agreed, which aligns with the training signal it received. The student cannot learn to classify digits that the teachers themselves could not agree on.

Tuning PATE and Comparing Approaches

With PATE's core mechanism established (teacher consensus, noisy aggregation, and confident labeling), we now analyze how noise scale affects label quality and privacy budgets, compare PATE to DP-SGD, and discuss practical deployment considerations.

Privacy-Utility Tradeoff Across Noise Scales

Higher noise provides stronger privacy guarantees (lower

€)

but degrades label quality. The relationship is not linear: doubling noise scale halves per-query epsilon but does not halve accuracy. Understanding where this curve bends informs practical choices.

Sweeping Through Noise Scales

We test noise scales from 5 to 80, measuring privacy cost and label quality at each level:

```
print("\n" + "=" * 60)
print("PHASE 7: Privacy-Utility Tradeoff Analysis")
print("=" * 60)

noise_scales = [5, 10, 20, 40, 80]

print(f"\n{'Noise Scale':<12} {'Per-ε':<10} {'Total ε':<12} {'Label Acc':<12} {'Flip Rate':<12}")
print("-" * 58)

for ns in noise_scales:
    noisy_labels = noisy_argmax(votes, ns)

    per_eps = 2.0 / ns
    total_eps, _ = compute_privacy_budget(num_queries, ns)

    label_acc = (noisy_labels == y_query_true).mean()
    flip_rate = (noisy_labels != clean_labels).mean()

    print(f"{'ns':<12} {'per_eps':<10.4f} {'total_eps':<12.2f} {'label_acc':<12.4f} {'flip_rate':<12.4f}")
```

Each row shows per-query epsilon, total epsilon via advanced composition, label accuracy, and flip rate.

Results Across the Spectrum

At noise scale 5, per-query $\epsilon = 0.40$ yields approximately 95% label accuracy. Double the scale to 10 and ϵ halves to 0.20 while accuracy drops only 3 points to 92%. At scale 20 (our default), $\epsilon = 0.10$ yields approximately 87% accuracy. Push to scale 40 and accuracy begins dropping faster: $\epsilon = 0.05$ with only 78% accuracy. At scale 80, $\epsilon = 0.025$ drops accuracy to approximately 65%, approaching the point where noise overwhelms signal.

The practical operating range sits between scale 10 and 40. Below 10, privacy guarantees weaken substantially. Above 40, label quality degrades faster than privacy improves. Our

default of 20.0 balances these considerations.

Visualizing the Tradeoff

A dual-axis plot reveals the asymmetry between privacy gains and accuracy losses. We define a function that computes accuracies and epsilons for each noise scale:

```
def plot_privacy_utility_tradeoff(noise_scales, votes, y_true, num_queries,
                                  save_path=None):
    """Plot privacy vs utility across noise scales on dual axes."""
    accuracies = [(noisy_argmax(votes, ns) == y_true).mean() for ns in
noise_scales]
    epsilons = [compute_privacy_budget(num_queries, ns)[0] for ns in
noise_scales]

    fig, ax1 = plt.subplots(figsize=PLOT_CONFIG['figsize'])
    ax1.set_xlabel('Noise Scale')
    ax1.set_ylabel('Label Accuracy', color=HTB_GREEN)
    ax1.plot(noise_scales, accuracies, 'o-', color=HTB_GREEN, linewidth=2,
markersize=10)
    ax1.tick_params(axis='y', labelcolor=HTB_GREEN)
```

The primary y-axis shows label accuracy. We add a secondary y-axis for privacy budget:

```
ax2 = ax1.twinx()
ax2.set_ylabel('Privacy Budget ( $\epsilon$ )', color=AZURE)
ax2.plot(noise_scales, epsilons, 's--', color=AZURE, linewidth=2,
markersize=10)
ax2.tick_params(axis='y', labelcolor=AZURE)
ax2.set_yscale('log')

ax1.set_title('Privacy-Utility Tradeoff', color=HTB_GREEN,
fontweight='bold')
ax1.grid(True, alpha=0.3)
if save_path:
    plt.savefig(save_path, dpi=PLOT_CONFIG['dpi'], bbox_inches='tight',
facecolor=NODE_BLACK)
plt.close()
```

Generating the visualization:

```
os.makedirs("figs", exist_ok=True)
plot_privacy_utility_tradeoff(noise_scales, votes, y_query_true,
```

```
num_queries,
```

```
save_path="figs/pate_privacy_utility.png")
```



The green accuracy line stays near 88% at low noise scales, then drops steeply beyond scale 40. Meanwhile, the blue privacy line (log scale) shows epsilon decreasing with noise. The asymmetry is striking: moving from scale 5 to 20 provides dramatic privacy improvement (epsilon drops from ~35 to ~9) with modest accuracy cost (95% to 87%). But moving from 20 to 80 yields diminishing privacy returns while accuracy collapses to 65%. Scale 20 sits at the elbow of this curve.

PATE vs DP-SGD: When to Use Each

PATE and DP-SGD both achieve differential privacy, but through fundamentally different mechanisms. PATE protects data through architectural separation; DP-SGD protects it through training-time noise.

PATE works best when public data matching the private distribution is available. The student trains on public data with standard optimization, and once trained, answers queries indefinitely without consuming privacy budget. Confident aggregation lets you reject low-consensus queries without cost, investing budget only where it produces high-quality labels. The information bottleneck also provides an intuitive privacy argument beyond formal ϵ values, which helps when explaining privacy properties to non-technical stakeholders.

The public data requirement is often the binding constraint. Medical datasets, financial records, and personalized models rarely have suitable public proxies. The query budget can also limit model complexity: sophisticated models requiring millions of samples may exhaust

privacy budget before obtaining enough labels. Training hundreds of teachers requires substantial computation, though inference parallelizes well.

DP-SGD takes a different approach, training directly on private data without requiring a public proxy. This potentially learns richer representations than knowledge distillation allows, and avoids ensemble overhead entirely. However, every gradient update consumes privacy budget, so long training runs can exhaust budget quickly. Per-sample gradient clipping affects convergence and requires careful tuning.

The choice depends on your constraints. If public data matching your private distribution exists and you need high-volume inference after deployment, PATE is typically the better choice. If no public proxy exists, or you need to train complex models directly on private data, DP-SGD may be your only option despite its per-query privacy costs.

Deployment Strategies

Confident aggregation enables three deployment approaches depending on operational constraints. When regulatory constraints impose a hard epsilon limit, you want to maximize labels within that budget. When student training requires a specific dataset size, you want to minimize privacy cost for that target. When public data is abundant and label quality matters most, you can query iteratively until enough high-consensus samples accumulate.

Fixed Privacy Budget

When regulatory or policy constraints impose a hard epsilon limit, confident aggregation allows more total queries while staying within budget. We use binary search to find how many queries a given epsilon permits:

```
def estimate_queries_for_budget(target_epsilon, noise_scale, delta=1e-5):
    """Binary search for maximum queries within privacy budget."""
    per_query_eps = 2.0 / noise_scale
    low, high = 1, 100000
    while low < high:
        mid = (low + high + 1) // 2
        eps_sq_sum = mid * (per_query_eps ** 2)
        total_eps = np.sqrt(2 * eps_sq_sum * np.log(1 / delta))
        total_eps += mid * per_query_eps * (np.exp(per_query_eps) - 1)
        low, high = (mid, high) if total_eps <= target_epsilon else (low,
mid - 1)
    return low
```

Now we compare how many queries each strategy permits for a fixed budget:

```
_, confident_mask = confident_aggregation(votes, noise_scale, threshold=200)
acceptance_rate = confident_mask.mean()

target_epsilon = 10.0
standard_queries = estimate_queries_for_budget(target_epsilon, noise_scale)
confident_queries = standard_queries / acceptance_rate

print(f"For  $\epsilon$ = {target_epsilon}:")
print(f"  Standard aggregation: {standard_queries} queries")
print(f"  Confident (threshold=200): {int(confident_queries)} queries")
print(f"  Net usable labels: {int(confident_queries * acceptance_rate)} samples")
```

Both strategies produce similar numbers of usable labels, but confident aggregation's labels are higher quality because low-consensus samples are filtered out.

Fixed Label Count

When student training requires a specific dataset size, confident aggregation minimizes privacy cost. To obtain 5,000 high-quality labels, we query until enough high-consensus samples are found:

```
target_labels = 5000
confident_queries = int(target_labels / acceptance_rate)

conf_eps = compute_confident_privacy_budget(
    confident_queries, noise_scale, acceptance_rate
)[0]

print(f"For {target_labels} labels:")
print(f"  Queries needed: {confident_queries}")
print(f"  Privacy cost:  $\epsilon$  = {conf_eps:.2f}")
```

Iterative Sampling

When public data is abundant and label quality matters most, we query samples one at a time until enough high-consensus samples accumulate:

```
def iterative_confident_labeling(teachers, X_public, threshold,
                                target_count, noise_scale, device):
    """Query samples until target_count high-consensus labels obtained."""
    confident_X, confident_y, queries_made = [], [], 0
    available_indices = np.random.permutation(len(X_public))
```

```

for idx in available_indices:
    if len(confident_y) >= target_count:
        break
    x = X_public[idx:idx+1]
    labels, mask = confident_aggregation(get_teacher_votes(teachers, x,
device), noise_scale, threshold)
    queries_made += 1
    if mask[0]:
        confident_X.append(x[0])
        confident_y.append(labels[0])

return np.array(confident_X), np.array(confident_y), queries_made

```

This approach guarantees all labels come from high-consensus samples. With 85% acceptance rate and target of 5,000 labels, we expect ~5,900 queries. The tradeoff is increased query count for guaranteed label quality.

In practice, fixed label count is often the natural choice: you typically know how many training samples the student needs, and confident aggregation reduces the privacy cost of obtaining them.

Hyperparameter Sensitivity

Several interacting hyperparameters affect PATE's performance. The number of teachers trades individual model strength against consensus signal strength. The query budget trades student training data against privacy cost, with favorable sublinear scaling.

Number of Teachers

More teachers strengthen the aggregation signal but reduce data per teacher. With fewer teachers, each receives more samples and achieves higher individual accuracy, but the aggregation signal is weaker. With more teachers, individual accuracy drops but consensus signals become stronger. The best count depends on dataset size and classification difficulty. Easier classification tasks can tolerate more teachers; harder tasks may benefit from fewer but stronger teachers.

Query Budget

More queries provide more student training data but consume more privacy budget. With 1,000 queries, the student receives limited training data but total $\epsilon \approx 3.9$ provides strong privacy. With 2,500 queries, training data increases and $\epsilon \approx 6.2$ remains reasonable. With 5,000 queries (our default), the student has ample training data and $\epsilon \approx 8.81$ provides good privacy. With 10,000 queries, the student has abundant data but $\epsilon \approx 12.5$ begins to weaken privacy guarantees.

The relationship is sublinear due to advanced composition. Doubling queries increases epsilon by approximately 40%, not 100%. This favorable scaling makes PATE practical for reasonable training set sizes.

Inherent Limitations

Beyond parameter tuning, PATE faces inherent constraints. The public data distribution must match private data, which is challenging for medical, financial, and personalized applications. Class imbalance creates consensus disparities that propagate to student performance. The disjoint partition requirement demands careful deduplication to maintain privacy guarantees.

Public Data Distribution

The public data must match the private data distribution. Distribution shift between public and private data degrades student performance more than label noise because the student learns patterns that do not transfer. Medical applications are particularly challenging because public medical data rarely matches the population in private hospital records. Financial applications face similar constraints. Personalized models almost by definition lack public data from the same distribution.

Class Imbalance

Imbalanced datasets create consensus disparities. Teachers achieve higher consensus on majority class samples because they see more examples during training and agree more readily. Minority class samples produce lower consensus and receive noisier labels. The student consequently underperforms on minority classes.

Stratified sampling during labeling can mitigate but not eliminate this effect. Ensuring the query set maintains class balance helps, but the consensus disparity remains.

Disjoint Partition Requirement

PATE's privacy guarantee assumes each individual contributes to exactly one teacher's partition. Duplicate records violate this assumption because the same person influences multiple teachers, increasing information leakage. Multiple appearances (the same person at different times) cause similar problems. Longitudinal data where individuals appear repeatedly over time requires careful handling.

Data deduplication is essential before partitioning. If complete deduplication is impossible, the privacy analysis must account for the maximum number of teachers any individual could affect.

Saving Results

We save configuration and results for comparison with other techniques. First, we compute the information bottleneck metrics that quantify how much private data compresses into

student training signal:

```
# Information bottleneck metrics
private_data_bytes = len(X_private_norm) * X_private_norm.shape[1] * 4 #
float32
student_info_bytes = num_queries * 4 # 4 bits per label, ~0.5 bytes, round
to int size
compression_ratio = private_data_bytes / student_info_bytes

print(f"\nInformation Bottleneck:")
print(f" Private data: {private_data_bytes / 1e6:.1f} MB")
print(f" Student labels: {student_info_bytes / 1e3:.1f} KB")
print(f" Compression ratio: {compression_ratio:.0f}:1")
```

The compression ratio quantifies PATE's privacy mechanism: megabytes of private training data compress into kilobytes of noisy class labels. Now we save the complete results:

```
results = {
    'configuration': {'dataset': 'mnist', 'num_teachers':
TEACHER_CONFIG['num_teachers'],
                    'noise_scale': AGGREGATION_CONFIG['noise_scale'],
'num_queries': num_queries},
    'privacy': {'per_query_epsilon': float(per_query_eps), 'total_epsilon':
float(total_eps)},
    'utility': {'student_accuracy': float(student_test_acc),
'label_accuracy': float(label_accuracy),
                'clean_ensemble_accuracy': float(clean_ensemble_acc)},
    'information': {'private_data_bytes': int(private_data_bytes),
'student_info_bytes': int(student_info_bytes),
                    'compression_ratio': float(compression_ratio)}
}

results_path = os.path.join("figs", "pate_results.json")
with open(results_path, 'w') as f:
    json.dump(results, f, indent=2)
print(f"Results saved to {results_path}")
```

This output enables programmatic comparison across configurations and provides a baseline for evaluating privacy techniques.

PATE Privacy Evaluation

Privacy guarantees can be stated mathematically through differential privacy, but empirical validation matters for practical deployments. Membership inference attacks provide a concrete test: can an attacker determine whether specific individuals were in the training data by observing model behavior? Here we evaluate PATE's resistance to such attacks, demonstrating that the architectural separation between teachers and students provides meaningful privacy protection beyond theoretical bounds.

Training the Vulnerable Baseline

Before evaluating PATE's privacy, we train a vulnerable baseline model to establish what unprotected models leak. This model trains directly on sensitive data without any privacy protection, serving as the reference point for attack comparison:

```
print("\n" + "=" * 60)
print("PHASE 8: Privacy Evaluation")
print("=" * 60)

BASELINE_CONFIG = {
    "hidden_layers": [256, 128],
    "dropout": 0.0,
    "epochs": 50,
    "batch_size": 64,
    "learning_rate": 0.001,
    "train_size": 10000,
}
```

We follow standard practice for demonstrating MIA vulnerability. The [256, 128] architecture is appropriately sized for the 10-class digit classification task. Zero dropout maximizes overfitting, making the model more vulnerable to attacks. We train on 10,000 samples to produce reliable attack accuracy measurements.

```
# Select subset of private data for baseline training
np.random.seed(RANDOM_SEED)
baseline_indices = np.random.choice(
    len(X_private_norm), BASELINE_CONFIG['train_size'], replace=False
)
X_baseline_members = X_private_norm[baseline_indices]
y_baseline_members = y_private[baseline_indices]

# Track which samples are NOT in the baseline training set
baseline_nonmember_mask = np.ones(len(X_private_norm), dtype=bool)
baseline_nonmember_mask[baseline_indices] = False
X_baseline_nonmembers = X_private_norm[baseline_nonmember_mask]
```

```

y_baseline_nonmembers = y_private[baseline_nonmember_mask]

print(f"Baseline members: {len(X_baseline_members)} samples")
print(f"Baseline non-members: {len(X_baseline_nonmembers)} samples")

```

We now have the member/non-member split needed for fair MIA evaluation. Both sets come from the same distribution (private MNIST data), differing only in whether they were used for training. With 10,000 members selected from 48,000 private samples, the remaining 38,000 serve as non-members.

Training the Baseline Model

With data prepared, we train the baseline model using the holdout set for validation:

```

baseline_member_loader = create_dataloader(
    X_baseline_members, y_baseline_members,
    BASELINE_CONFIG['batch_size']
)

baseline_model = MLP(
    input_size=num_features,
    hidden_layers=BASELINE_CONFIG['hidden_layers'],
    num_classes=DATASET_CONFIG['num_classes'],
    dropout=BASELINE_CONFIG['dropout']
)

train_model(
    baseline_model, baseline_member_loader, holdout_loader,
    device=DEVICE,
    epochs=BASELINE_CONFIG['epochs'],
    learning_rate=BASELINE_CONFIG['learning_rate']
)

```

We train for 50 epochs with zero dropout, deliberately encouraging overfitting. After training, the model achieves high accuracy on its training data but shows measurable overfitting that membership inference attacks can exploit.

Implementing the Membership Inference Attack

Our attack exploits a consistent observation: models behave differently on samples they have memorized versus samples they have not seen. Specifically, models tend to produce higher-confidence predictions on training data because they have optimized specifically for those examples. We implement a confidence threshold attack that classifies samples as members if their prediction confidence exceeds a threshold:

```
def compute_mia_advantage(model, X_members, y_members, X_nonmembers,
y_nonmembers, device):
    """
    Compute membership inference attack advantage using confidence
    threshold.
    """
    member_probs = get_model_predictions(model, X_members, device)
    member_confidence = np.max(member_probs, axis=1)

    nonmember_probs = get_model_predictions(model, X_nonmembers, device)
    nonmember_confidence = np.max(nonmember_probs, axis=1)
```

We extract the maximum softmax probability for each sample as a confidence score. A sample with prediction `[0.95, 0.05, ...]` has confidence 0.95, while one with `[0.55, 0.25, ...]` has confidence 0.55. Our hypothesis is that members will cluster at higher confidence values because the model has seen and memorized them.

Finding the Best Threshold

To separate member and non-member confidence distributions, we search over percentiles to find the threshold that maximizes classification accuracy:

```
n_samples = min(len(member_confidence), len(nonmember_confidence))
member_conf_balanced = member_confidence[:n_samples]
nonmember_conf_balanced = nonmember_confidence[:n_samples]

all_confidence = np.concatenate([member_conf_balanced,
nonmember_conf_balanced])
all_labels = np.concatenate([np.ones(n_samples), np.zeros(n_samples)])

thresholds = np.percentile(all_confidence, np.linspace(0, 100, 1000))
best_accuracy = 0.0

for threshold in thresholds:
    predictions = (all_confidence >= threshold).astype(int)
    accuracy = np.mean(predictions == all_labels)
    if accuracy > best_accuracy:
        best_accuracy = accuracy
        best_threshold = threshold

attack_advantage = best_accuracy - 0.5
return best_accuracy, attack_advantage, best_threshold
```

Why balance the datasets? With equal members and non-members, a random classifier achieves exactly 50% accuracy. This makes 50% the natural baseline for measuring attack success. If we used unbalanced sets (say 90% members), even a trivial "always predict member" strategy would achieve 90% accuracy, obscuring the attack's true effectiveness.

Why search over percentiles? Percentiles span the full range of observed confidence values. Using `np.percentile(all_confidence, np.linspace(0, 100, 1000))` generates 1,000 candidate thresholds from the minimum to maximum observed confidence. This exhaustive search finds the threshold that best separates the two distributions, giving the attacker every advantage. If the model leaks membership information, this search will find the optimal way to exploit it.

We try each candidate threshold and count correct classifications. For threshold 0.85, any sample with confidence ≥ 0.85 is predicted as "member" and any below as "non-member." The threshold yielding highest accuracy becomes the attack's decision boundary. The attack advantage is accuracy minus 0.5: an accuracy of 55% yields advantage 0.05 (5 percentage points above random guessing). This metric directly measures how much information the model leaks about its training set.

Attacking the Baseline Model

With the attack implemented, we evaluate the vulnerable baseline:

```
print("\n--- Attacking Baseline Model ---")
print(f"  Members: {len(X_baseline_members)} samples (baseline training data)")
print(f"  Non-members: {len(X_baseline_nonmembers)} samples")

baseline_mia_acc, baseline_mia_adv, baseline_threshold =
compute_mia_advantage(
    baseline_model,
    X_baseline_members, y_baseline_members,
    X_baseline_nonmembers, y_baseline_nonmembers,
    DEVICE
)

print(f"  Attack accuracy: {baseline_mia_acc:.4f}")
print(f"  Attack advantage: {baseline_mia_adv:.4f}")
```

Our attack achieves accuracy above random guessing, demonstrating that the unprotected model leaks membership information. An attacker observing only predictions can determine with some accuracy whether specific digit images were used for training. While MNIST images are less sensitive than medical records, this demonstrates the attack methodology that would apply to more sensitive data.

Attacking the PATE Student

The question is whether the PATE student, which never directly accessed private data, leaks information about that data through its predictions. We test this by attacking the student with the private data as the member set:

```
print("\n--- Attacking PATE Student Model ---")

student_mia_acc_private, student_mia_adv_private, _ = compute_mia_advantage(
    student_model,
    X_private_norm, y_private,
    X_holdout_norm, y_holdout,
    DEVICE
)

print(f"  Attack accuracy:  {student_mia_acc_private:.4f}")
print(f"  Attack advantage: {student_mia_adv_private:.4f}")
```

We achieve approximately 50% accuracy with near-zero advantage, essentially indistinguishable from random guessing. The student model cannot distinguish private data from holdout data because it never trained on the private data directly. Its knowledge comes exclusively from the noisy pseudo-labels generated by the teacher ensemble.

Understanding the Protection Mechanism

Why does PATE resist membership inference? The protection arises from architectural design, not just noise addition. The student model has no direct connection to private data samples. Information flows through a severe bottleneck: 48,000 private digit images with 784 features each compress into just 5,000 noisy class labels. Each label is a single integer (0-9) determined by noisy vote aggregation. Attempting to infer individual membership from these labels is like trying to identify a specific drop of water after it has mixed with a river.

We can also test whether the student leaks information about the public query samples it trained on directly:

```
student_mia_acc_query, student_mia_adv_query, _ = compute_mia_advantage(
    student_model,
    X_query, student_labels,
    X_holdout_norm, y_holdout,
    DEVICE
)
```

```
print(f"  Attack on query data accuracy:  {student_mia_acc_query:.4f}")
print(f"  Attack on query data advantage: {student_mia_adv_query:.4f}")
```

Here we achieve higher accuracy with measurable advantage. This leakage about the query data is expected and not a privacy concern: the student did train directly on these samples, and query data is public by assumption. The high attack accuracy on query data actually confirms our methodology works correctly. This leakage concerns only the public query data, not the private data that PATE aims to protect.

Comparing Attack Results

Comparing results reveals PATE's protective effect:

```
print("\n" + "-" * 50)
print("SUMMARY: Attack Advantage Comparison")
print("-" * 50)
print(f"  Baseline (no protection):      {baseline_mia_adv:.4f}")
print(f"  PATE Student (private data): {student_mia_adv_private:.4f}")
print(f"  PATE Student (query data):   {student_mia_adv_query:.4f}")

if baseline_mia_adv > 0:
    reduction = (baseline_mia_adv - student_mia_adv_private) /
baseline_mia_adv * 100
    print(f"\n  Protection improvement: {reduction:.1f}%")
```

The baseline leaks membership information (attack advantage above random), while the PATE student leaks essentially none about the private data (advantage near zero, statistically indistinguishable from random guessing). We see substantial reduction in attack advantage. The remaining small advantage likely reflects random variation, not true leakage.

Visualizing Confidence Distributions

Why does the attack succeed on the baseline? Member and non-member confidence distributions differ. We visualize this separation by plotting the density difference between member and non-member confidence values. First, we define a function to plot confidence distribution differences:

```
def plot_confidence_distributions(baseline_member, baseline_nonmember,
                                student_private, student_holdout,
                                save_path=None):
    """Plot density difference between member and non-member confidence."""
    fig, axes = plt.subplots(1, 2, figsize=(12, 5))

    # Baseline model
```



```

bins = np.linspace(0.5, 1.0, 50)
member_hist, _ = np.histogram(baseline_member, bins=bins, density=True)
nonmember_hist, _ = np.histogram(baseline_nonmember, bins=bins,
density=True)
diff = member_hist - nonmember_hist
bin_centers = (bins[:-1] + bins[1:]) / 2

axes[0].bar(bin_centers, diff, width=0.01, color=MALWARE_RED, alpha=0.8)
axes[0].axhline(y=0, color=WHITE, linestyle='--', alpha=0.5)
axes[0].set_xlabel('Prediction Confidence')
axes[0].set_ylabel('Density Difference (Member - Non-member)')
axes[0].set_title('Baseline Model', color=MALWARE_RED,
fontweight='bold')

```

This creates histograms for both member and non-member confidence values, then plots their difference. We apply the same approach to the PATE student:

```

# PATE student
student_member_hist, _ = np.histogram(student_private, bins=bins,
density=True)
student_nonmember_hist, _ = np.histogram(student_holdout, bins=bins,
density=True)
student_diff = student_member_hist - student_nonmember_hist

axes[1].bar(bin_centers, student_diff, width=0.01, color=HTB_GREEN,
alpha=0.8)
axes[1].axhline(y=0, color=WHITE, linestyle='--', alpha=0.5)
axes[1].set_xlabel('Prediction Confidence')
axes[1].set_ylabel('Density Difference (Private - Holdout)')
axes[1].set_title('PATE Student', color=HTB_GREEN, fontweight='bold')

plt.tight_layout()
if save_path:
    plt.savefig(save_path, dpi=PLOT_CONFIG['dpi'], bbox_inches='tight',
facecolor=NODE_BLACK)
plt.close()

```

We also define a function to create a bar chart comparing attack results across models:

```

def plot_mia_comparison(baseline_adv, student_adv_private,
student_adv_query,
                        baseline_acc, student_acc_private,
student_acc_query, save_path=None):

```

```

"""Plot bar chart comparing MIA results across models."""
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

labels = ['Baseline', 'PATE\n(Private)', 'PATE\n(Query)']
advantages = [baseline_adv, student_adv_private, student_adv_query]
accuracies = [baseline_acc, student_acc_private, student_acc_query]
colors = [MALWARE_RED, HTB_GREEN, AQUAMARINE]

axes[0].bar(labels, advantages, color=colors, edgecolor=HACKER_GREY,
linewidth=2)
axes[0].axhline(y=0.05, color=NUGGET_YELLOW, linestyle='--',
linewidth=2, label='5% threshold')
axes[0].set_ylabel('Attack Advantage')
axes[0].set_title('MIA Attack Advantage', color=HTB_GREEN,
fontweight='bold')
axes[0].legend()

```

The first subplot shows attack advantage with a 5% threshold line. We add the second subplot for attack accuracy:

```

axes[1].bar(labels, accuracies, color=colors, edgecolor=HACKER_GREY,
linewidth=2)
axes[1].axhline(y=0.5, color=NUGGET_YELLOW, linestyle='--', linewidth=2,
label='Random (50%)')
axes[1].set_ylabel('Attack Accuracy')
axes[1].set_title('MIA Attack Accuracy', color=HTB_GREEN,
fontweight='bold')
axes[1].legend()

plt.tight_layout()
if save_path:
    plt.savefig(save_path, dpi=PLOT_CONFIG['dpi'], bbox_inches='tight',
facecolor=NODE_BLACK)
plt.close()

```

Now we compute confidence values and generate the visualization:

```

baseline_member_conf = np.max(get_model_predictions(baseline_model,
X_baseline_members, DEVICE), axis=1)
baseline_nonmember_conf = np.max(get_model_predictions(baseline_model,
X_baseline_nonmembers, DEVICE), axis=1)
student_private_conf = np.max(get_model_predictions(student_model,
X_private_norm, DEVICE), axis=1)

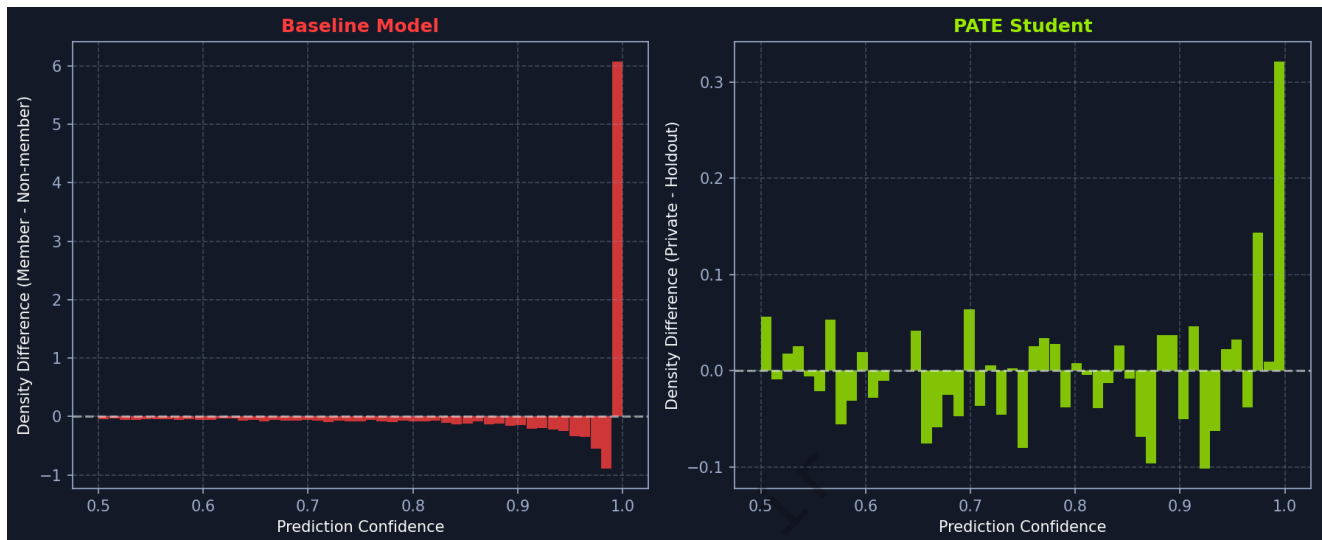
```

```

student_holdout_conf = np.max(get_model_predictions(student_model,
X_holdout_norm, DEVICE), axis=1)

plot_confidence_distributions(
    baseline_member_conf, baseline_nonmember_conf,
    student_private_conf, student_holdout_conf,
    save_path="figs/pate_confidence_distributions.png"
)

```



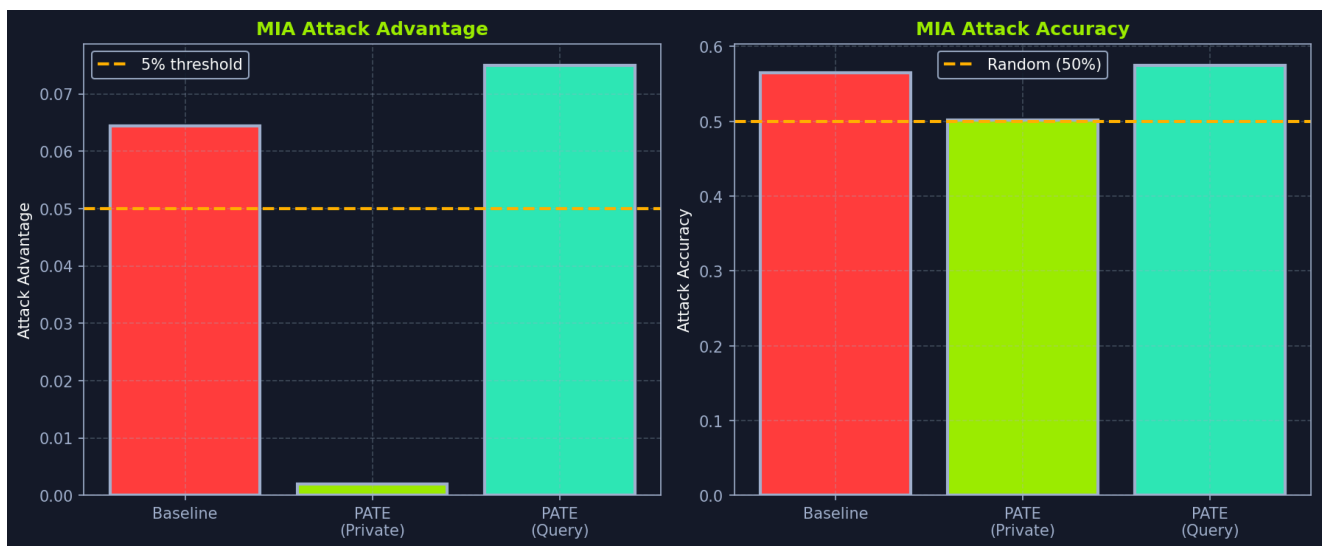
What do these density differences reveal? In the left panel, the tall red spike at high confidence shows the baseline's vulnerability: members receive highly confident predictions far more often than non-members. This is the attack surface. In contrast, the right panel shows PATE's protection: the difference hovers near zero across all confidence values. The student model cannot distinguish private data from holdout data because it never trained on either.

Visualizing Attack Comparison

```

plot_mia_comparison(
    baseline_mia_adv, student_mia_adv_private, student_mia_adv_query,
    baseline_mia_acc, student_mia_acc_private, student_mia_acc_query,
    save_path="figs/pate_mia_comparison.png"
)

```



How effective is PATE's protection? The left panel's attack advantage tells the story. The baseline (red) exceeds the 5% threshold commonly used as an acceptable leakage bound. The PATE student against private data (green) drops to near zero, statistically indistinguishable from random guessing. The cyan bar shows expected leakage about query data, which is not a privacy concern since query data is public by design. The right panel confirms this interpretation: attack accuracy against private data sits precisely at 50%, the random baseline.

Interpreting the Results

These empirical results validate PATE's theoretical privacy guarantees. The student achieves differential privacy not through noise during inference, but through the information bottleneck during training. Private data influences the student only through noisy aggregate labels, and this limited information transfer prevents meaningful membership inference.

Several observations emerge from our attack evaluation. **Baseline vulnerability scales with overfitting**: the overfitting gap in the baseline translates directly to attack success. More overfitting means more memorization, which means more distinguishable member behavior.

PATE protection is architectural. The student's resistance to attack does not depend on adding noise at inference time. Once trained, the student answers queries deterministically without privacy cost. This architectural guarantee is fundamentally different from techniques that require noise during inference.

Public query leakage confirms methodology. The leakage about query data is not a privacy concern because query data is public by assumption. This result validates our attack implementation: it successfully detects membership when the model did train on the data, while failing to detect membership for data the model never saw.

Attack advantage reduction is substantial. An attacker who could previously determine membership with accuracy above chance now achieves accuracy statistically indistinguishable from random guessing.

PATE Challenge

Your organization needs a privacy-preserving handwritten letter classifier. The model must be trained using PATE on the EMNIST Letters dataset (26 classes, A-Z), achieving high accuracy while maintaining strong privacy guarantees against membership inference attacks.

Requirements

Your submission must satisfy both criteria to receive the flag:

Metric	Requirement
accuracy	$\geq 80\%$
mia_advantage	$\leq 3\%$

API Endpoints

```
export BASE_URL="http://INSTANCE_IP:PORT"
```

GET /health

```
curl -s "$BASE_URL/health" | jq
{
  "status": "ok",
  "data_loaded": true
}
```

POST /validate

Validates your student model and returns the flag on success.

```
curl -s -X POST "$BASE_URL/validate" \
-F "model=@pate_student.safetensors" | jq
{
  "passed": true,
  "accuracy": 0.8612,
  "mia_advantage": 0.0215,
  "evaluation_time": 2.3,
  "flag": "HTB{...}"
}
```

On failure:

```
curl -s -X POST "$BASE_URL/validate" \
-F "model=@pate_student.safetensors" | jq
{
  "passed": false,
  "accuracy": 0.7823,
  "mia_advantage": 0.0183,
  "evaluation_time": 2.1
}
```

GET /

```
curl -s "$BASE_URL/" | jq
{
  "service": "PATE Privacy Challenge (EMNIST Letters)",
  "version": "1.0"
}
```

Model Architecture

Your student model must use this exact architecture:

```
import torch.nn as nn
import torch.nn.functional as F

class MLP(nn.Module):
    """Multi-Layer Perceptron for EMNIST Letters classification."""

    def __init__(self, input_size=784, hidden_layers=None, num_classes=26,
                 dropout=0.2):
        super(MLP, self).__init__()
        if hidden_layers is None:
            hidden_layers = [256, 128]

        self.layers = nn.ModuleList()
        self.dropouts = nn.ModuleList()

        prev_size = input_size
        for hidden_size in hidden_layers:
            self.layers.append(nn.Linear(prev_size, hidden_size))
            self.dropouts.append(nn.Dropout(dropout))
```

```

        prev_size = hidden_size

        self.output = nn.Linear(prev_size, num_classes)

    def forward(self, x):
        for layer, dropout in zip(self.layers, self.dropouts):
            x = F.relu(layer(x))
            x = dropout(x)
        return self.output(x)

```

Data Preprocessing

The validator uses this preprocessing:

```

import numpy as np
from sklearn.preprocessing import StandardScaler
from torchvision import datasets

train_dataset = datasets.EMNIST("data", split='letters', train=True,
download=True)
test_dataset = datasets.EMNIST("data", split='letters', train=False,
download=True)

X_train = train_dataset.data.numpy().reshape(-1, 784).astype(np.float32) /
255.0
X_test = test_dataset.data.numpy().reshape(-1, 784).astype(np.float32) /
255.0

# Labels are 1-26 for A-Z, convert to 0-25
y_train = train_dataset.targets.numpy() - 1
y_test = test_dataset.targets.numpy() - 1

scaler = StandardScaler()
X_train_norm = scaler.fit_transform(X_train)
X_test_norm = scaler.transform(X_test)

```

Submission Format

Save your trained student model using `safetensors` :

```

from safetensors.torch import save_file

```

```
save_file(student_model.state_dict(), "pate_student.safetensors")
```

Submit:

```
import requests

with open("pate_student.safetensors", "rb") as f:
    response = requests.post(
        f"{BASE_URL}/validate",
        files={"model": ("pate_student.safetensors", f, "application/octet-
stream")}
    )
print(response.json())
```

Skills Assessment

Train a privacy-preserving Fashion-MNIST classifier that resists membership inference attacks while maintaining utility.

Requirements

Your submission must satisfy both criteria to receive the flag.

Your defended model must maintain at least 70% test accuracy on Fashion-MNIST. This ensures the privacy defense doesn't sacrifice too much utility.

Your defended model must reduce the MIA advantage by at least 40% compared to the vulnerable baseline. The server runs a standardized membership inference attack against your model and compares its vulnerability to the baseline.

Your model must use the `FashionMNISTCNN` architecture exactly as specified below.

API Endpoints

Set the base URL environment variable to interact with the challenge server.

```
export BASE_URL="http://instance_ip:port"
```

```
curl -s "$BASE_URL/health" | jq
{
  "status": "ok",
  "model_loaded": true
}
```


Use `/baseline` to retrieve information about the vulnerable model's MIA vulnerability. This shows the baseline advantage your model needs to improve upon.

```
curl -s "$BASE_URL/baseline" | jq
{
  "baseline_mia_advantage": 0.0895,
  "required_improvement": 0.4,
  "required_test_accuracy": 0.7,
  "description": "The vulnerable baseline model has the MIA advantage shown above. Your defended model must reduce this advantage by at least 40% while maintaining at least 70% test accuracy."
}
```

You can also query the vulnerable model via `/query`.

```
curl -s -X POST "$BASE_URL/query" \
  -H "Content-Type: application/json" \
  -d '{"samples": [[0.1, 0.2, ...], ...]}' | jq
{
  "predictions": [3, 7, ...],
  "confidences": [0.92, 0.87, ...]
}
```

Dataset and Model

Load Fashion-MNIST with identical normalization to ensure your local samples match the server's preprocessing.

```
import torch
import torchvision
import torchvision.transforms as transforms

transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.2860,), (0.3530,))
])

train_dataset = torchvision.datasets.FashionMNIST(
    root='./data', train=True, download=True, transform=transform
)

test_dataset = torchvision.datasets.FashionMNIST(
```

```
root='./data', train=False, download=True, transform=transform
)
```

Your defended model must match this architecture exactly to pass validation.

```
import torch.nn as nn

class FashionMNISTCNN(nn.Module):
    def __init__(self):
        super(FashionMNISTCNN, self).__init__()

        self.conv1 = nn.Conv2d(1, 32, kernel_size=3, padding=1)
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
        self.conv3 = nn.Conv2d(64, 64, kernel_size=3, padding=1)

        self.pool = nn.MaxPool2d(2, 2)
        self.relu = nn.ReLU()

        self.fc1 = nn.Linear(64 * 3 * 3, 128)
        self.fc2 = nn.Linear(128, 10)

    def forward(self, x):
        x = self.pool(self.relu(self.conv1(x)))
        x = self.pool(self.relu(self.conv2(x)))
        x = self.pool(self.relu(self.conv3(x)))
        x = x.view(-1, 64 * 3 * 3)
        x = self.relu(self.fc1(x))
        x = self.fc2(x)
        return x
```

Submission Format

Save your trained model using safetensors format.

```
from safetensors.torch import save_file

model = FashionMNISTCNN()
# ... train your model with privacy-preserving techniques ...

save_file(model.state_dict(), "defended_model.safetensors")
```

Submit your defended model to the `/submit` endpoint.

```
curl -X POST "$BASE_URL/submit" \  
-F "defended_model=@defended_model.safetensors"
```

```
{  
  "valid": true,  
  "message": "Congratulations! Your model successfully resists membership  
inference attacks!",  
  "flag": "HTB{...}",  
  "score": 81.3,  
  "details": {  
    "test_accuracy": 0.7185,  
    "mia_accuracy": 0.5082,  
    "mia_advantage": 0.0082,  
    "improvement_ratio": 0.9078,  
    "baseline_mia_advantage": 0.0895  
  }  
}
```

hide01.ir